

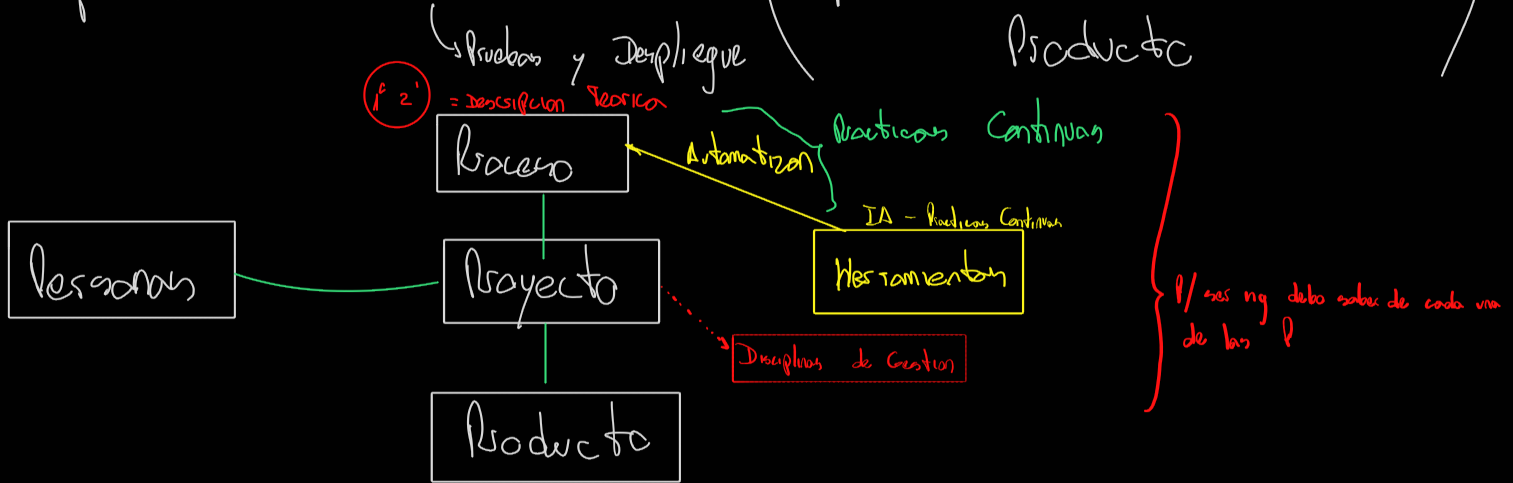
Definición Integradora de Software

El software es conocimiento o información representada en distintos niveles de Abstracción $\Rightarrow$ No es solo código ni un conjunto de programas

$\rightarrow$ Parte de lo General y se va refinando a lo largo del proceso de desarrollo de software hasta llegar al final del proceso que son cosas y unos

$\rightarrow$ Los resultados de las tareas de

Disciplinas Técnicas $\rightarrow$ 1ª' (enfocadas a la creación del Producto)



* No se puede hacer mg de software SOLO

$\rightarrow$ enfoque sistemático y organizado para resolver problemas

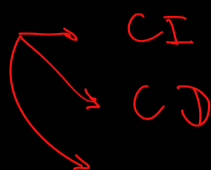
Proyecto tiene resultado único. También elaboración gradual y tareas interrelacionadas p/ cumplir con 1 objetivo?

$\rightarrow$ es una unidad de gestión de organización de recursos y el trabajo de las personas

* El resultado del proyecto es una versión del producto distribuido

Despliegue: Transparencia del producto al universo final

Prácticas Continuas



* el software es intangible

$\neq$
fabricación de productos tangibles

Proyecto Lider < PM (Estar a un nivel de gestión más alto)
Tiene a cargo a varias PL

(3ª')

DBD no está involucrado en Proyecto, Director de la BD sí

Administración
Gestión
(Management)

Planificación
Monitoreo

SCM → Método o Framework

Agilidad → Enfoque o Filosofía

SCM Manager ≠ Líder de Proyecto

Analistas Funcionales ≠ Product Owner

Representantes del Negocio
"Rol del Negocio" → No técnicos

Disciplinas de Soporte → Mantener la integridad del Producto
Transversales → a lo largo de todo el Proyecto o producto

Aseguramiento de calidad = Acompañamiento
≠ Testing

expectativas ≠ Necesidad → lo que se manifiesta y lo que no se manifiesta

Métrica = Valoración Objetiva a 3P (menos las personas ya que no se las mide)

SCM es la más tangible de las disciplinas de ing en software ∴ es una disciplina

* Las 3 disciplinas generan IC

* Configuración = Conjunto de IC del repositorio EN UN MOMENTO DE TIEMPO

IC = Items de configuración 1017'
→ tiene una versión asociada y momento (fecha)

* Propósito: - mantener la integridad de cada IC

- Atender la problemática inherente del SW que es
es que el SW cambia todo el tiempo cuando se encuentra en uso

Práctico:

SCM → Propósito: Gestionar cambios → integridad del producto

↓
Disciplina de soporte al PRODUCTO

Actividades Fundamentales / Básicas

1) Identificar Items de Configuración → ¿me interesa tener trazabilidad de «posible IC»

2) Control de Cambios

3) Reparto de estados (como fotos al repositorio en un momento)

4) Auditorías de Configuración

→ Física (¿Cumple con el Plan de SCM? ¿Está?)
→ Funcional (2º 23') → Matriz de Trazabilidad (2º 24')

→ siempre externas

→ Checklist (2º 27') → Herramienta q usan los Auditores

• 1) Regla de Nombrado y Repositorio (nombre repositorio) y Glosario

• 2)

Plan de SCM 1º 57 → Es un IC en si mismo (también con regla de Nombrado)

SCM en Pac Definidos → CCC (comité de control de cambios)

{ mínimo
Extendido } Depende del impacto del cambio

→ Base Line (2º 2') → "configuración estable"

SCM en Ambientes Agiles → Prácticas Continuas

→ conjunto de IC

→ Automatizadas (Reducción de tiempos)

→ Pipeline de despliegue

Iterativo e incremental es el ciclo de vida, el PUD es un proceso

TP5 → durante todo el ciclo 2º 8'

TP5 → durante todo el ciclo 2º 8'

* Version del IC

* Ramas → cuando unifico los clientes
Merge (integración) → conflictos

¿Cómo se marca la línea base?

2º 3º Buenas Prácticas

Las carpetas con por lo menos 2 IC
→ Reglas de IC
→ p/ Agregar
→ Reglas de Nombrado
→ Ubicación

Mover que viene Entrega TP → Definición como me convenga

guía de mr Evaluables > TP
guía de Trabajos hechos > Para chusquear

Estructura del repo sob carpetas, sin Arch

Carpetas p/ TP programable

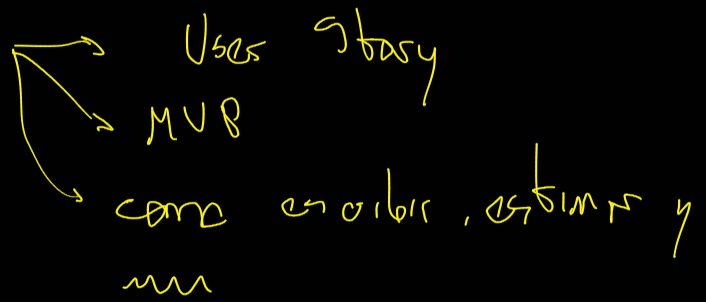
Carpetas p/ parte Administ → Cronograma
Como Mandar Mail

Carpetas Trabajos de investigación → consignar

Reglas de Nombrado no dudar Glorioso

↓
Poner extensión
Poner donde van guardados

Clase y clase puede faltar



gestion de Gantt

* La linea base puede estar conformada por un IC

* Entregable / Artefacto del SCM: Plan de SCM → es un IC

* Dato: En W. de Ing's esta Elicitacion → Especificacion → Validacion

Build = conjunto de paquetes de IC que se pasan en produccion o que van a Testing y si pasan se van a despliegue (produccion) convirtiendose en un Release



UTN-FRC
INGENIERÍA Y CALIDAD DE SOFTWARE

Ingeniería de Software

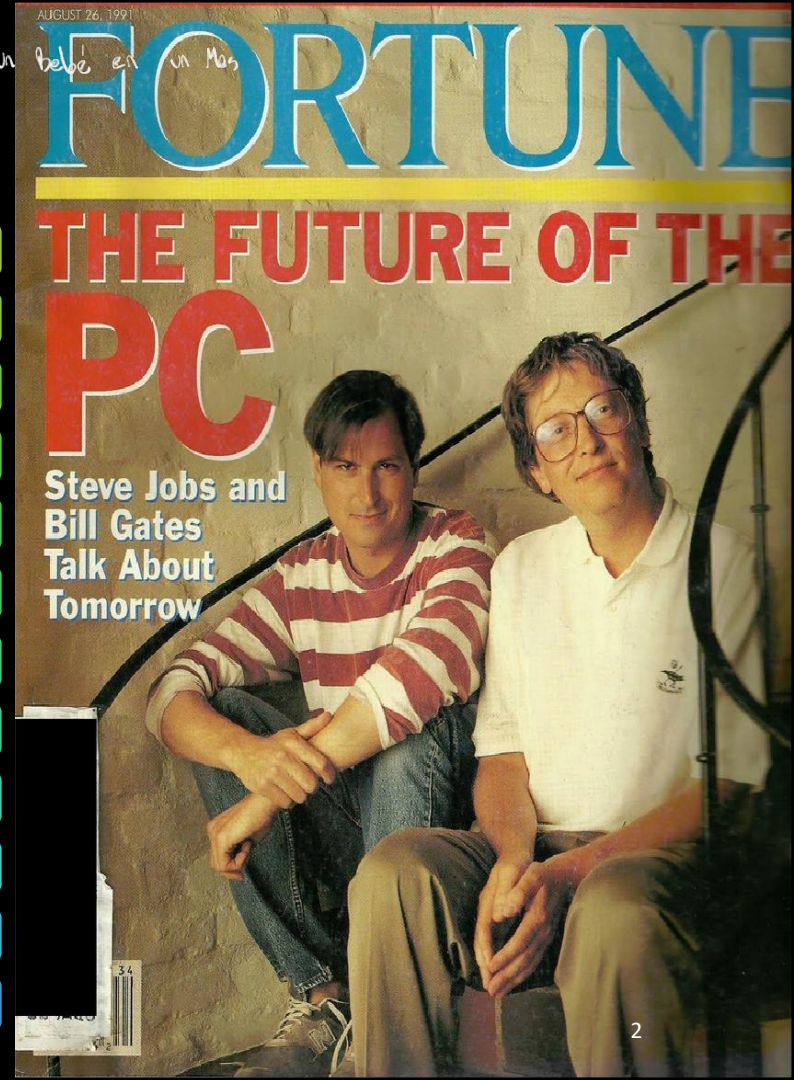


Universidad Tecnológica Nacional
Facultad Regional Córdoba
Cátedra de Ingeniería de Software
Docentes: Judith Meles – Laura Covaro

> Dice: Añadir personas a un proyecto retrasando lo retrasa aún más
(una Embarazada recién a M el un bebé ⇒ No sirve poner 9 Mujeres el tener un bebé en un Mes

Un poco de historia

- 1968 • Nace el termino – conferencia de la NATO
- 1975 • The Mythical Man-Month – Frederick Brooks
- 1978 • Tom DeMarco introduce Structured Analysis
- 80' • Primeros grandes errores de software conocidos
- 1987 • No silver bullet (Brooks). Características esenciales del software
- 1989 • Managing the Software Process – Watts Humphrey
- 1990 • Internet / Object Oriented
- 1991 • CMM 1.0
- 1993 • CMM 1.1
- 2000 • CMMI 1.0
- 2001 • Agile Manifiesto
- 2003 • Lean Software Development



GLOBAL TOP 5 BRANDS | TECH ASCENDANCY

RANK

1

2

3

4

5

2006



2012



2017



Las 10 empresas más grandes del mundo en 2019

- 1 **Apple** (881.980 millones de dólares)
- 2 **Microsoft** (803.090 millones de dólares)
- 3 **Amazon** (739.460 millones de dólares)
- 4 **Alphabet** (711.940 millones de dólares)
- 5 **Berkshire Hathaway** (536.530 millones de dólares)
- 6 **Johnson & Johnson** (396.210 millones de dólares)
- 7 **Alibaba** (387.610 millones de dólares)
- 8 **Facebook** (378.050 millones de dólares)
- 9 **JPMorgan Chase & Co** (368.560 millones de dólares)
- 10 **Tencent Holdings** (353.670 millones de dólares)

Las 10 empresas más grandes del mundo en 2026

- **NVIDIA** (EE. UU.): ~ \$4.7 billones
- **Alphabet** (Google, EE. UU.): ~ \$4.0 billones
- **Apple** (EE. UU.): ~ \$3.8 billones
- **Microsoft** (EE. UU.): ~ \$2.9 billones
- **Amazon** (EE. UU.): ~ \$2.6 billones
- **TSMC** (Taiwán): ~ \$1.9 billones
- **Broadcom** (EE. UU.): ~ \$1.8 billones
- **Saudi Aramco** (Arabia Saudita): ~ \$1.7 billones
- **Meta Platforms** (Facebook, EE. UU.): ~ \$1.6 billones
- **Tesla** (EE. UU.): ~ \$1.3 billones



Algunos problemitas con el desarrollo de software



La versión final del producto no satisface las necesidades del cliente.



No es fácil extenderlo y/o adaptarlo. Agregar más funcional en otra versión es casi una misión imposible



Mala documentación



Mala calidad.



Más tiempos y costos que los presupuestados

No importa

Errores informáticos más costosos de la historia

<https://www.javiergarzas.com/2013/05/top-7-de-errores-informaticos.html>

1 – La destrucción del Mariner I (1962). 18,5 millones de dólares.

El del Mariner I, una sonda espacial que se dirigía a Venus, se desvió de la trayectoria de vuelo prevista poco después del lanzamiento. Desde control se destruyó la sonda a los 293 segundos del despegue. La causa fue una fórmula manuscrita que se programó incorrectamente.

2 – La catástrofe del Hartford Coliseum (1978). 70 millones de dólares.

Apenas unas horas después de que miles de aficionados abandonaron el Hartford Coliseum, el techo se derrumbó por el peso de la nieve. La causa: cálculo incorrecto introducido en el software CAD utilizado para diseñar el coliseo.

3 – El gusano de Morris (1988). 100 millones de dólares.

El estudiante de posgrado Robert Tappan Morris fue condenado por el primer ataque con “gusanos” a gran escala en Internet. Los costos de limpiar el desastre se cifran en 100 millones de dólares. Morris, es hoy profesor en MIT.

4 – Error de cálculo de Intel (1994). 475 millones de dólares.

Un profesor de matemáticas descubrió y difundió que había un fallo en el procesador Pentium de Intel. La sustitución de chips costó a Intel 475 millones.

5 – Explosión del cohete Arian (1996). 500 millones de dólares.

En el 1996, el cohete Ariane 5 de la Agencia Espacial Europea estalló. El Ariane explotó porque un número real de 64 bits (coma flotante) relacionado con la velocidad se convirtió en un entero de 16 bits. [Te dejo un post que explica lo sucedido.](#)

6 – Mars Climate Orbiter (1999). 655 millones de dólares.

En 1999 los ingenieros de la NASA perdieron el contacto con la Mars Climate Orbiter en su intento que orbitase en Marte. La causa, un programa calculaba la distancia en unidades inglesas (pulgadas, pies y libras), mientras que otro utilizó unidades métricas.

7 – El error en los frenos de los Toyota (2010). 3 billones de dólares.

Toyota retiró más de 400.000 de sus vehículos híbridos en 2010, por un problema software, que provocaba un retraso en el sistema anti-bloqueo de frenos. Se estima entre sustituciones y demandas el error le costó a Toyota 3 billones de dólares.

8 – Las migraciones por el año 2000. 296,7 billones de dólares.

Se esperaba que el bug Y2K paralizase al mundo a la medianoche del 1 de enero 2000, ya que mucho software no había sido previsto para trabajar con el año 2000. El mundo no se acabó, pero se estima que se gastaron 296,7 billones de dólares para mitigar los daños.

ANNOUNCEMENT The Next web is coming back to New York! Learn more here



Chrysler is recalling 1.4 million vehicles after a Jeep was remotely hacked

by NADER LOPEZ 3d ago in INSIDER



http://thehacklab.io 0 Comments

All Time Favorites

The complete guide to backing up your computer properly

How to use personal drones legally. A beginner's guide

11 Android apps to make notifications more interesting

How to raise prices without ruining your business

How mobile is bridging brick and mortar's data gap

INFOBAE

Un error de software de algunos Airbus A350 requiere apagar los aviones cada 149 horas o actualizar el sistema

28 de junio de 2019

Compartir en Facebook

Compartir en Twitter



Hay una falla de software en los Airbus A350 que ya fue reparada en la actualización de software (Shutterstock)

MÁS LEÍDAS EN infobae.com

- 1 De ver en un libro, un medio de la selva se transformaron en un territorio más allá de la vida, la inspiración más de María Bui
- 2 La revolución sobre la fragilidad del mundo de María Bui, una historia y Argentina que pone en aprietos al padre Eugenio Canales
- 3 Según Ruffalo, los países que avanzarán en integrar el comercio de su región se están volviendo
- 4 Los ingresos fijos de la casa en un territorio más allá de la vida, la inspiración más de María Bui
- 5 Un medio que estuvo en Haití al comienzo de la pandemia, la historia del accidente de China. Una como el caso de la casa de la casa

Toyota deberá revisar 1,9 millones de autos Prius por problemas de software

Un error en el software que controla el funcionamiento del motor del auto podría hacer que se apague o funcione, pero a baja velocidad.

Compartir en Facebook Twitter LinkedIn Email Print



Toyota deberá revisar 1,9 millones de autos Prius por un error en el software que los controla.

Toyota anunció ayer un llamado a revisión de los 1,9 millones de autos Prius de última generación vendidos en todo el mundo debido a un problema de software que provoca la caída de su sistema híbrido.

Alrededor de la mitad de los Prius a ser revisados están en Japón y 750.000 en América del Norte, dijo una portavoz de Toyota. No se han reportado accidentes relacionados al defecto, agregó.

Toyota dijo que el problema se hallaba en el software usado para controlar el conmutador de potencia en un módulo que forma parte del sistema híbrido.

"La configuración del software podría causar una mayor tensión térmica en ciertos transistores dentro del conmutador de potencia, y estos transistores podrían deformarse o dañarse como consecuencia", dijo Toyota.

"Esto resultará en el encendido de varias luces de advertencia y probablemente cause que el vehículo entre en su modalidad protegida", sostuvo la automotriz.

Agregó que en esta modalidad el coche puede conducir pero a menor potencia máxima de manejo.

En pocos casos, el sistema híbrido podría apagarse, lo que causaría que el vehículo se detenga, probablemente mientras está siendo conducido, dijo Toyota.

Esta reciente medida sería el tercer llamado a revisión para el Prius actual, introducido hace cinco años, después de un llamado en junio del año pasado debido a problemas ligados al acumulador de freno. El Prius es uno de los modelos más vendidos de Toyota y se ha convertido en un símbolo de la tecnología híbrida de baja emisión.

Agencia Reuters

Un error de software deja para desguace 300 coches de Subaru

Un fallo en el software de los robots soldadores de Subaru provocan la retirada de 293 coches

Por Mario Contreras 2 años



Los coches se han convertido en ordenadores con ruedas y un motor. Se han convertido en una pieza central de nuestros coches que ahora, gracias a una más que dudosa seguridad, se pueden **hackear** a **palanca**. Si la programación de los coches es un problema, hay uno más grave con los robots que lo fabrican y Subaru tendrán que retirar unas 300 unidades del Ascent 2019 por un error de software.

Y sigue habiendo errores...

Definición de Software

Software, en general, **es** un set de programas y la documentación que acompaña.

- Existen tres tipos básicos de software. Estos son:
 - System software
 - Utilitarios
 - Software de Aplicación



Margaret Hamilton, lead software engineer, Project Apollo. Mostrando su código fuente..
<https://medium.com/@3fingeredfox/margaret-hamilton-lead-software-engineer-project-apollo-158754170da8>

Una definición integradora del software...



El software es conocimiento representado en distintos niveles de abstracción.



5 Razones para no comparar software y manufactura

✓ El software es menos predecible

→ pg es intangible

✓ No hay producción en masa,

casi ningún producto de software

es igual a otro. ⇒ Múltiples de clientes con diferentes

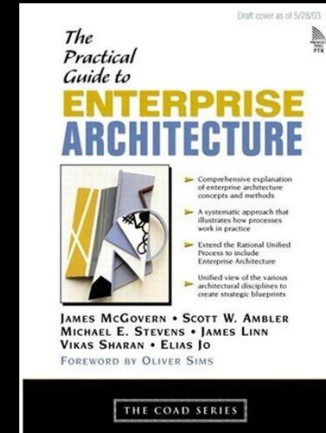
✓ No todas las fallas son errores

✓ El software no se gasta ⇒ No se designa en términos de Montecarlo

✓ El software no está gobernado por las leyes

de la física 😊

"The creation of software is an intellectual human endeavor. Creating good software relies on the personalities and the intellects of the members of the teams that create it. When applied to a different team of developers a process that delivers great software for one team of developers may fail to deliver anything at all for another team."
-- *The Practical Guide to S/W Arch.*



Dossier Estratégico para el Liderazgo Técnico

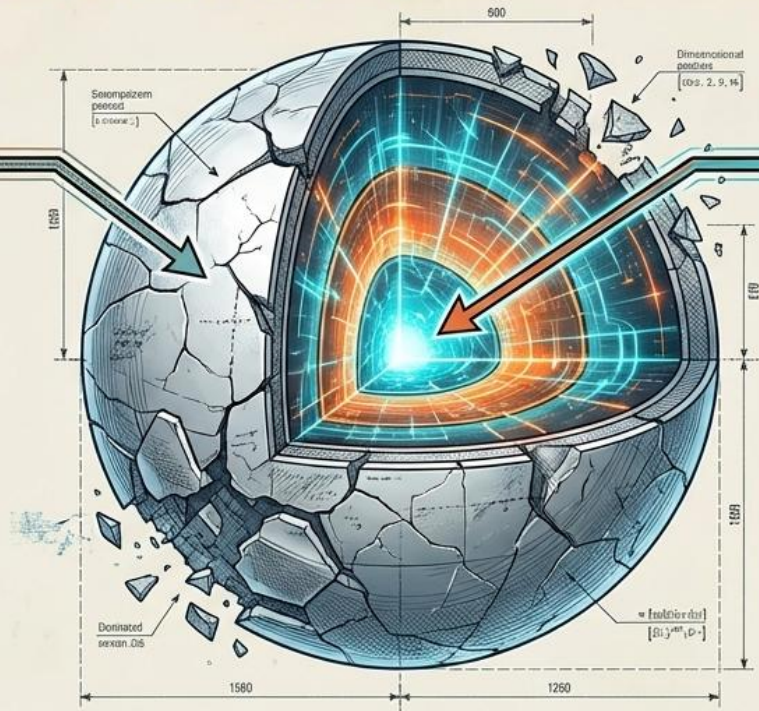
No Hay Balas de Plata

Esencia y Accidente en la Ingeniería de Software. Una reevaluación del ensayo fundacional de Frederick P. Brooks para la era moderna.

El Modelo Aristotélico del Software

El Accidente (La Representación)

Dificultades que acompañan la producción, pero no son inherentes.
(Ej: Sintaxis compleja, limitaciones de memoria, tiempos de compilación).
Ya casi los resolvimos todos.



La Esencia (El Concepto)

La construcción de estructuras conceptuales entrelazadas altamente complejas: algoritmos, flujos de datos, relaciones.
Esto es lo que queda hoy.

Escribir código no es el problema real. La parte difícil es la especificación, el diseño y la validación de la estructura conceptual.

Las 4 propiedades irreducibles del software complejo

La esencia del software resiste la automatización porque posee características únicas en la ingeniería humana.

1. Complejidad

Escalar software aumenta los elementos interactuando de forma no lineal. No hay piezas repetidas a nivel macro.

2. Conformidad

Debe adaptarse a interfaces humanas arbitrarias y sistemas sin principios unificadores, dictados por el entorno.

3. Mutabilidad

Es pura "materia de pensamiento", infinitamente maleable y constantemente sometida a la presión del cambio continuo.

4. Invisibilidad

El software carece de representación espacial o geométrica. No puede dibujarse en un plano. Priva a la mente de la abstracción visual.

La historia del software es la eliminación sistemática de lo accidental

Los mayores saltos de productividad históricos no alteraron la esencia; simplemente eliminaron barreras artificiales.

Esfuerzo Esencial (Diseño Conceptual)

Años 60: Lenguajes de Alto Nivel

Logro: Factor de 5x en productividad.

Resolución: Liberó al programador de lidiar con bits, registros y hardware.

Años 70: Time-Sharing

Logro: Preservación de la inmediatez.

Resolución: Eliminó el olvido de las minucias provocado por la compilación batch.

Años 80/90: Entornos Integrados (IDEs)

Logro: Herramientas unificadas.

Resolución: Facilitó la interconexión de programas y formatos.

Conclusión: Todos los grandes saltos atacaron exclusivamente el esfuerzo accidental.

El cuello de botella humano: Decidir el "Qué"

"La parte más difícil de construir software no es decirlo, sino decidir qué decir."



**Idea del Producto
(Mental)**

El Prompt

El nuevo lenguaje de alto nivel. Para que la IA funcione, el humano debe especificar con precisión matemática restricciones, estados y lógicas de negocio.

El límite fundamental:

Si el humano no comprende la complejidad esencial, el prompt será incompleto.

La IA no deduce la intención no escrita.

La programación automática históricamente solo funcionó en dominios con parámetros altamente restringidos.

El Embudo de la Especificación

El ancho de banda de ejecución de la IA es infinito; el ancho de banda conceptual del humano es el límite físico.

La Intención (El Qué)

Idea de negocio, reglas difusas,
integraciones legacy,
requerimientos de usuarios.

El Prompt

La traducción de ideas vagas a
restricciones formales requiere
esfuerzo humano puro.

La Representación (El Cómo)

La Representación (El Cómo)

Conclusión: El flujo total del sistema sigue estrangulado por la capacidad humana de diseño conceptual, sin importar cuán rápido se genere el código abajo.

Cómo atacar la verdadera Esencia

La IA es una excelente herramienta para lo accidental, pero la esencia requiere tácticas humanas orgánicas.



Buy vs. Build

Comprar frente a construir.
Evitar construir por completo.
La integración es más barata
que generar código nuevo
(incluso con IA).



Rapid Prototyping

Extraer requerimientos iterativamente.
La IA brilla aquí solo como
herramienta de descarte rápido
para refinar la especificación.



Grow, not build

Desarrollo incremental puro.
Hacer correr un esqueleto mínimo
e ir engrosando el sistema
orgánicamente, manteniendo
siempre un sistema funcional.



Grandes Diseñadores

El talento humano es irremplazable.
El salto de orden de magnitud
solo proviene de cultivar a las
mejores mentes de diseño
conceptual.

El diseño sigue siendo un acto humano

La IA nos dio la **velocidad de la luz** para recorrer el laberinto, pero no nos dio el mapa.



Las diferencias entre diseños buenos y grandes se acercan a un orden de magnitud.
Los grandes diseños provienen de grandes diseñadores, no de balas de plata.

Y cuando
nos va bien
es por...

1. Involucramiento del usuario 15.9 % → *hg's*
2. Apoyo de la Gerencia 13.0 %
3. Enunciado claro de los requerimientos 9.6 %
4. Planeamiento adecuado 8.2 %
5. Expectativas realistas 7.7 %
6. Hitos intermedios 7.7 %
7. Personas involucradas competentes 7.2 %

Y cuando nos va mal es por... *← Por el contrario*

1. Requerimientos incompletos 13.1 %
2. Falta de involucramiento del usuario 12.4 %
3. Falta de recursos 10.6 %
4. Expectativas poco realistas 9.3 %
5. Falta de apoyo de la Gerencia 8.7 %
6. Requerimientos cambiantes 8.1 %



Conclusión....

Saber programar NO es
suficiente!!!!

Ingeniería de Software

¿Qué es?

- Parmas definió a la ingeniería en software como “multi-person construction of multi-version software”
- *"La ingeniería de software es la aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento del software; es decir, la aplicación de la ingeniería al software".*
Definición oficial del IEEE (Instituto de Ingenieros Eléctricos y Electrónicos), formalizada en el SWEBOK (Software Engineering Body of Knowledge)
- *Sommerville: "La ingeniería de software es una disciplina de la ingeniería que se ocupa de todos los aspectos de la producción de software, desde la concepción inicial del sistema hasta su operación y mantenimiento".*

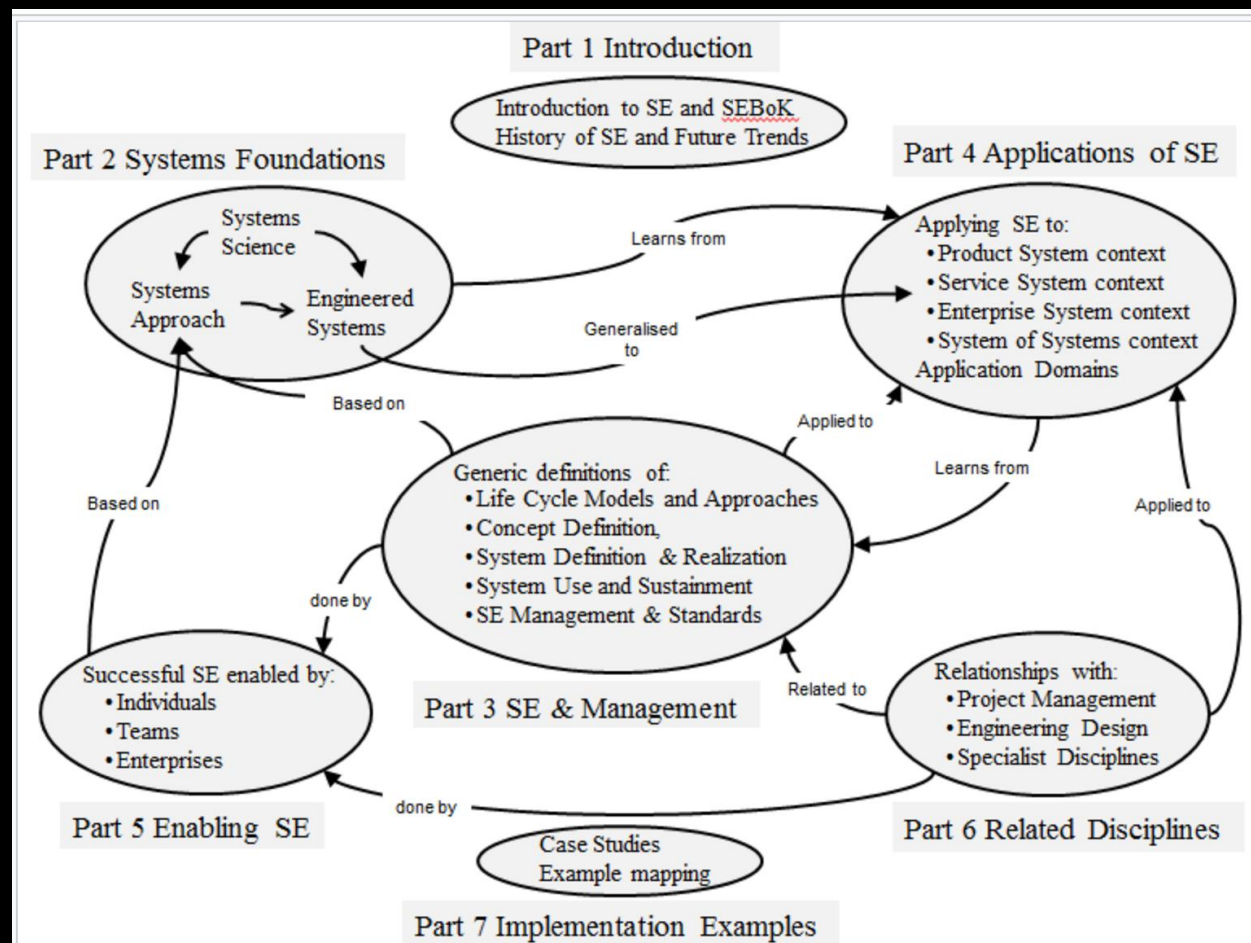




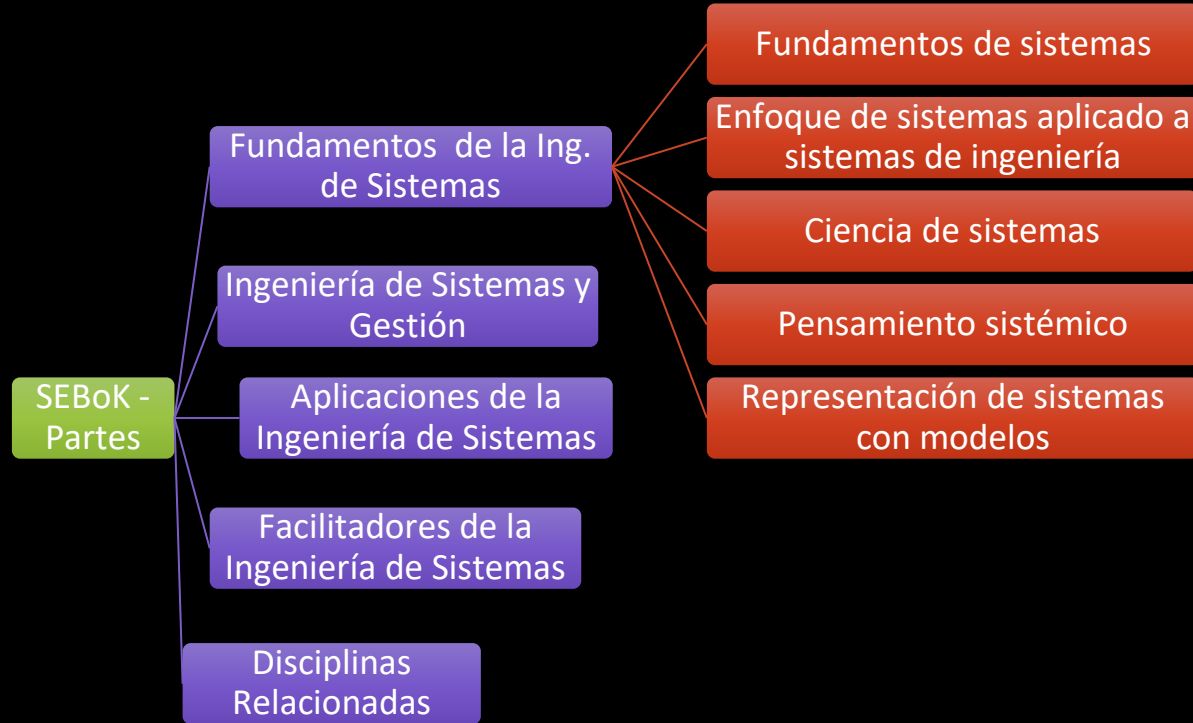
- Cuerpo de Conocimiento de la Ingeniería de Software
- Versión 3.0 del 2014 de la IEEE
- Está conformado por 15 áreas de conocimiento

Software Requirements
Software Design
Software Construction
Software Testing
Software Maintenance
Software Configuration Management
Software Engineering Management
Software Engineering Process
Software Engineering Models and Methods
Software Quality
Software Engineering Professional Practice
Software Engineering Economics
Computing Foundations
Mathematical Foundations
Engineering Foundations

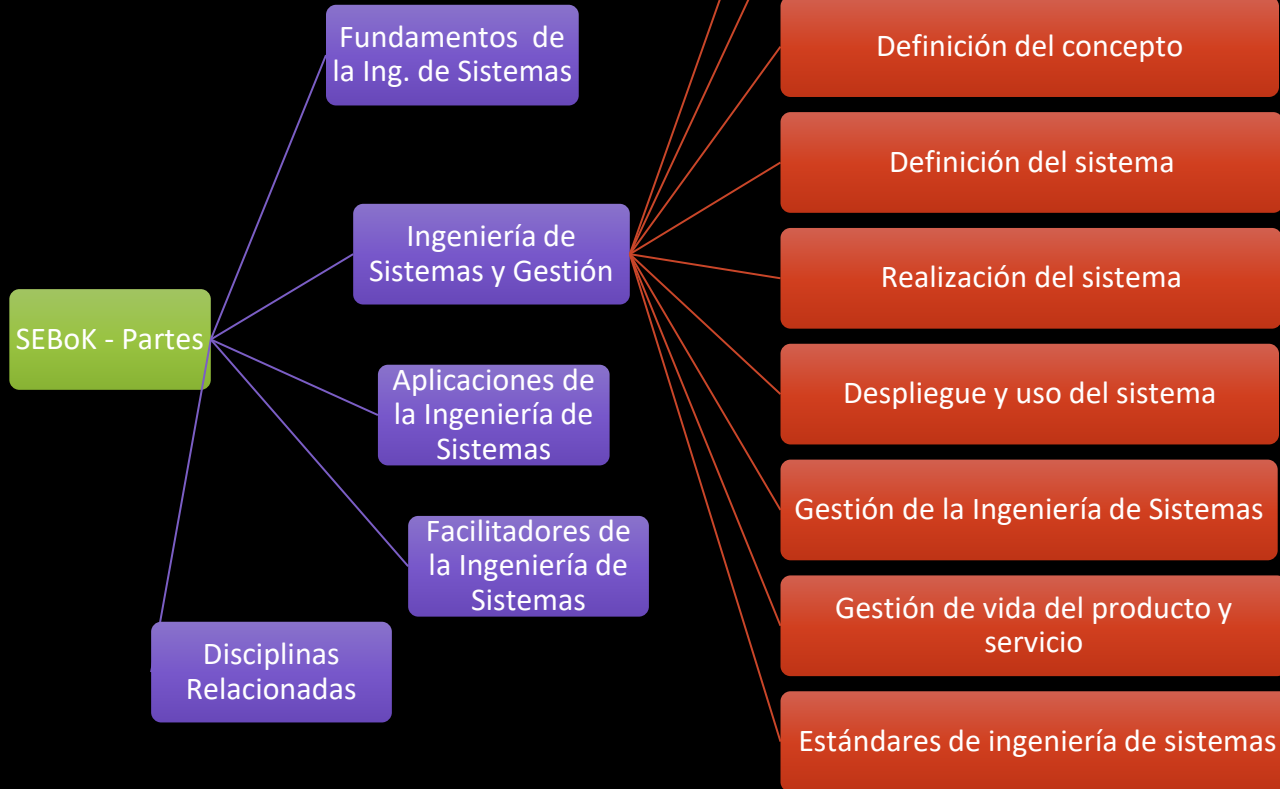
Cuerpo de Conocimiento de la Ingeniería de Sistemas- Versión 1.9.1 del 2018



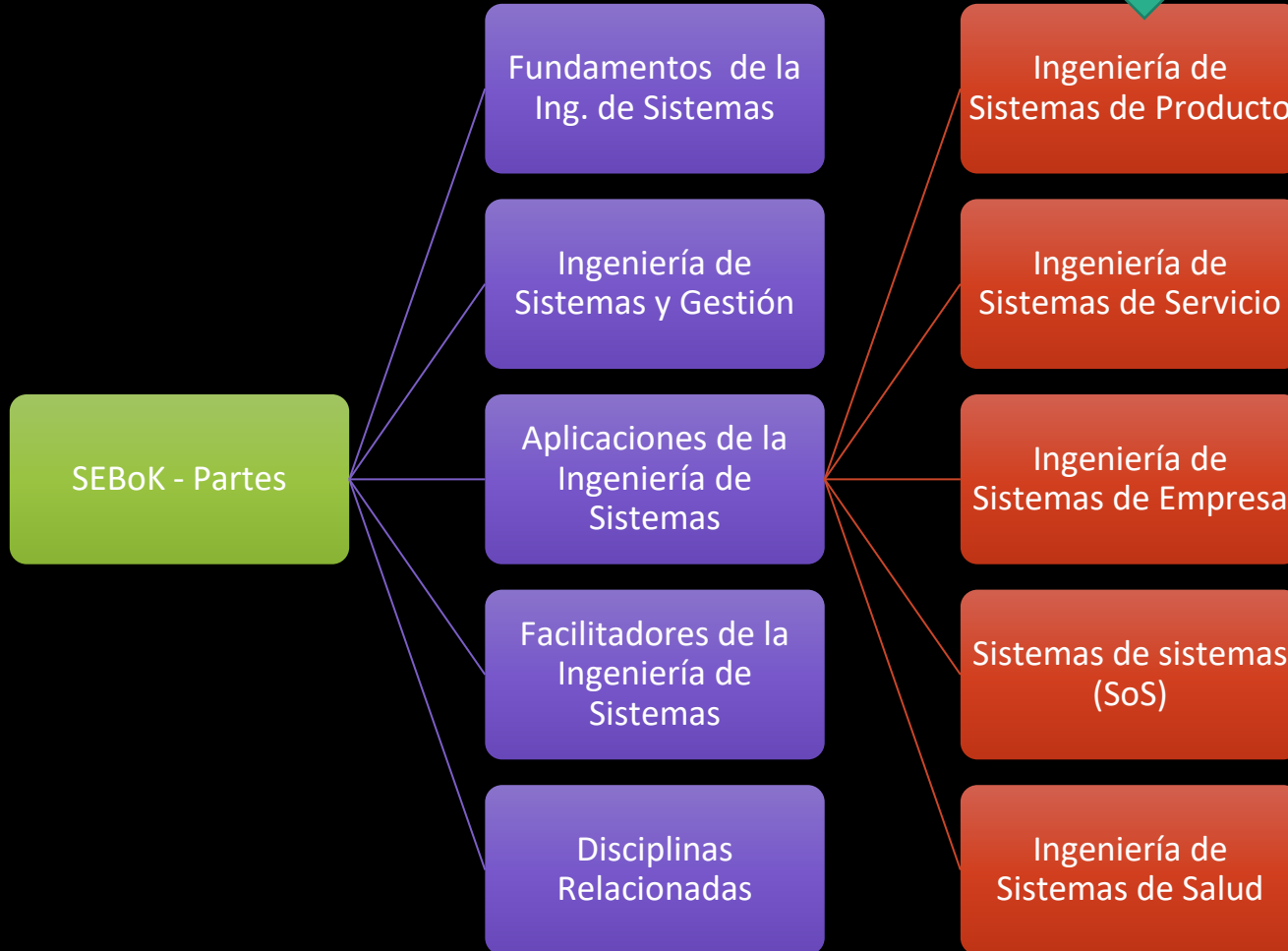
Áreas de Conocimiento

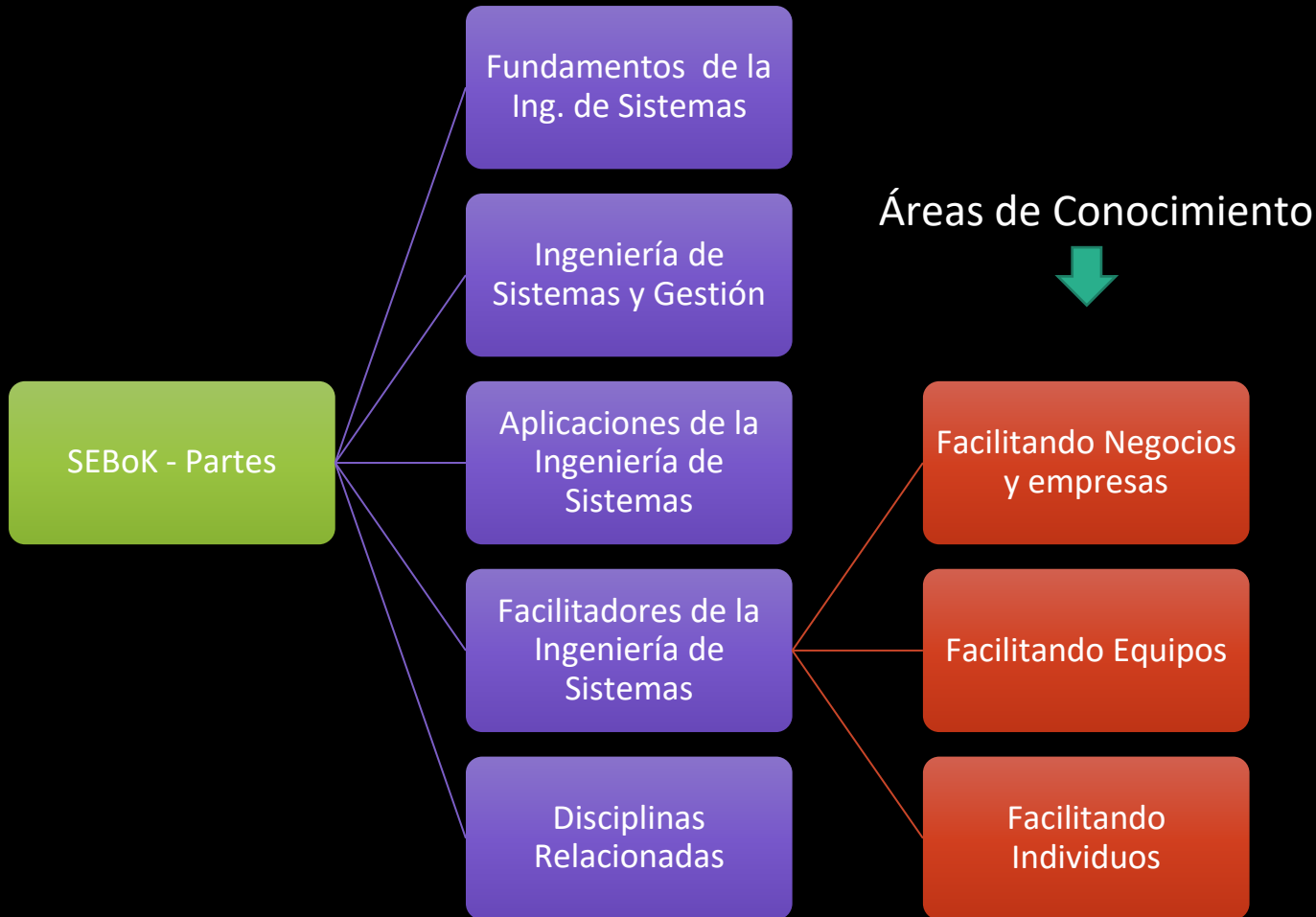


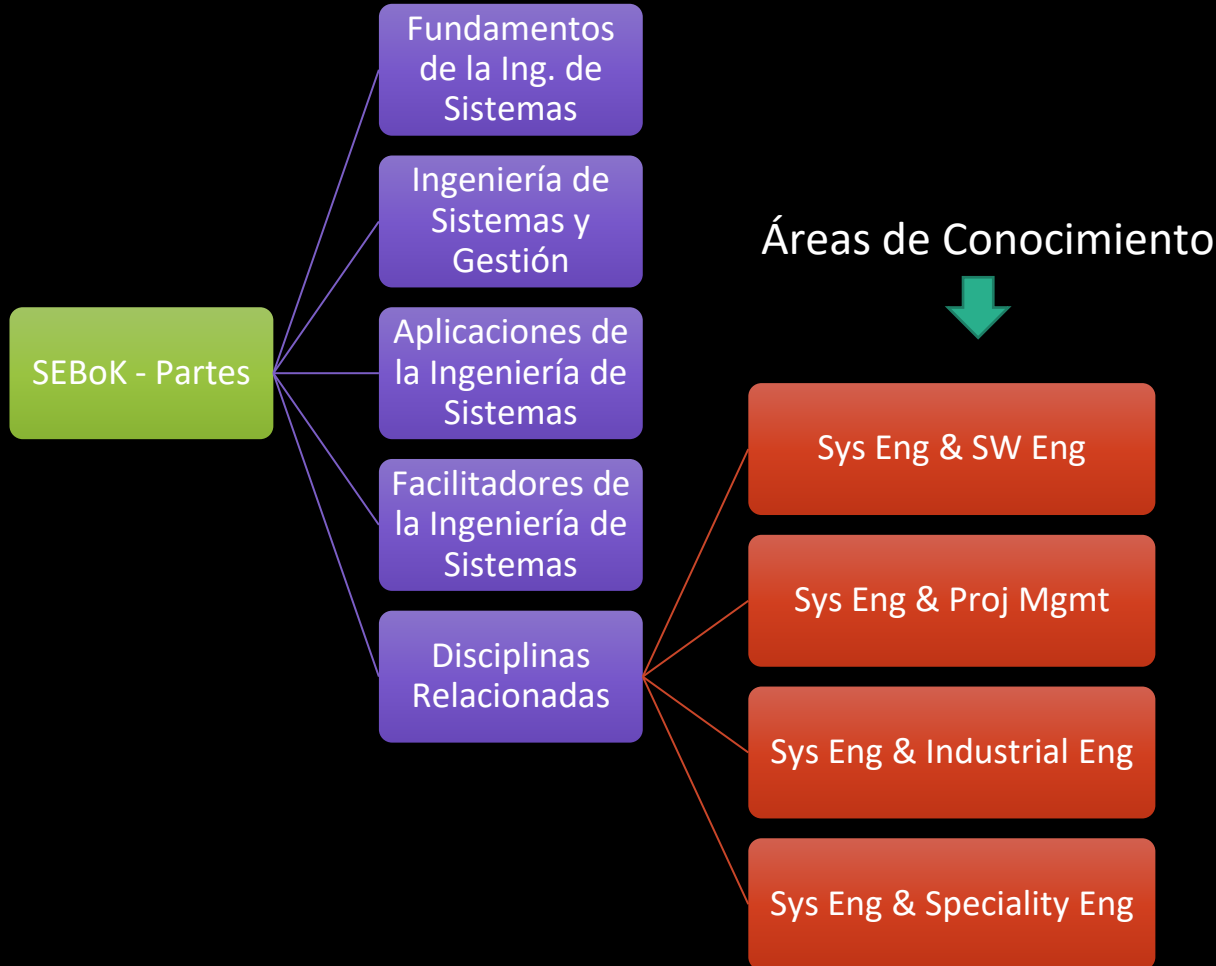
Áreas de Conocimiento →



Áreas de Conocimiento







Importante

Ingeniería y Calidad de Software: la materia en contexto



Disciplinas Técnicas

- Requerimientos
- Análisis y Diseño
- Construcción
- Prueba
- Despliegue

→ son las que nos permiten obtener como resultado el producto de software



Disciplinas de Gestión

- Planificación de Proyecto
- Monitoreo y Control de Proyectos



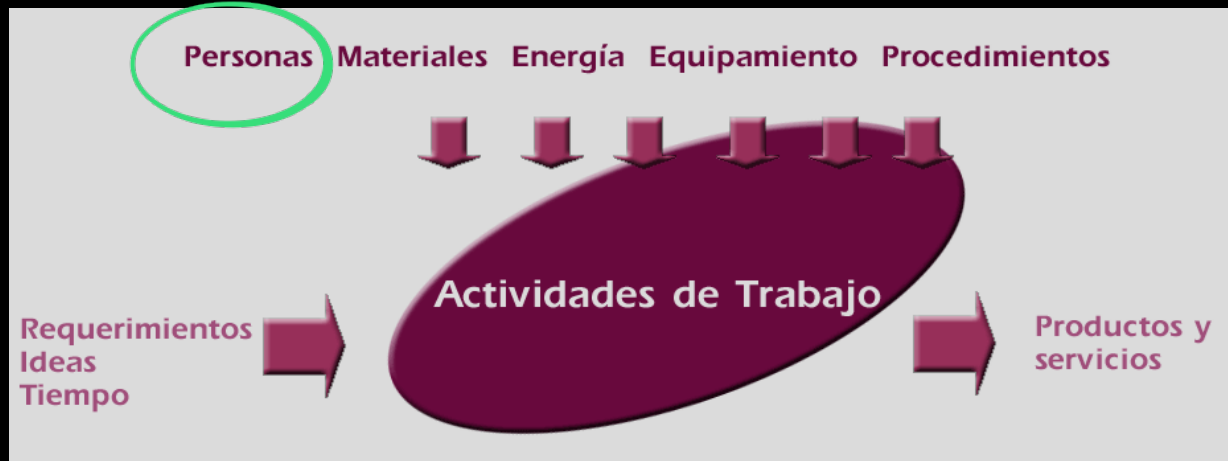
Disciplinas de Soporte

- Gestión de Configuración de Software
- Aseguramiento de Calidad
- Métricas

continúa

3 Disciplinas

El proceso de Software



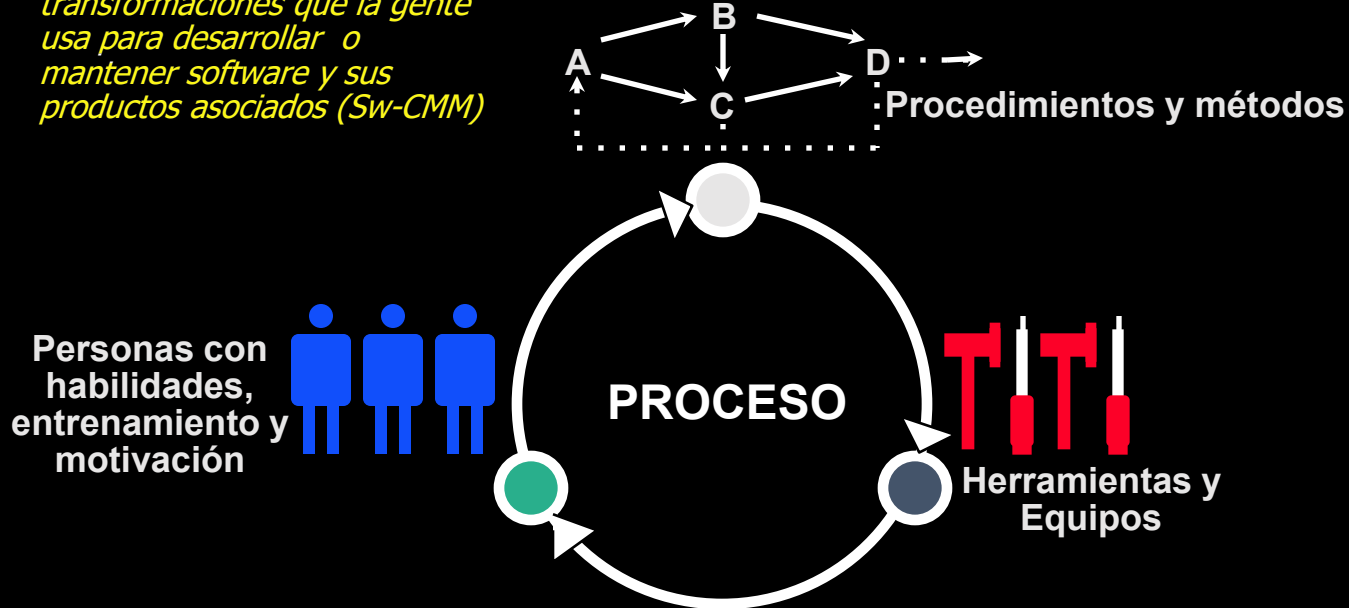
- Conjunto **estructurado** de actividades para desarrollar un sistema de software
- Estas actividades varían dependiendo de la organización y el tipo de sistema que debe desarrollarse.
- Debe ser explícitamente modelado si va a ser administrado.

Definición de un Proceso de Software

≠

Proceso: La secuencia de pasos ejecutados para un propósito dado (IEEE)

Proceso de Software: Un conjunto de actividades, métodos, prácticas, y transformaciones que la gente usa para desarrollar o mantener software y sus productos asociados (Sw-CMM)



Definido vs.

Empírico *basado en experiencia*

En la Universidad de California, en Irvine, simplemente sembraron pasto y esperaron 1 año, luego de eso se fijaron donde la gente había hecho “caminito”, entonces ahí construyeron las sendas peatonales



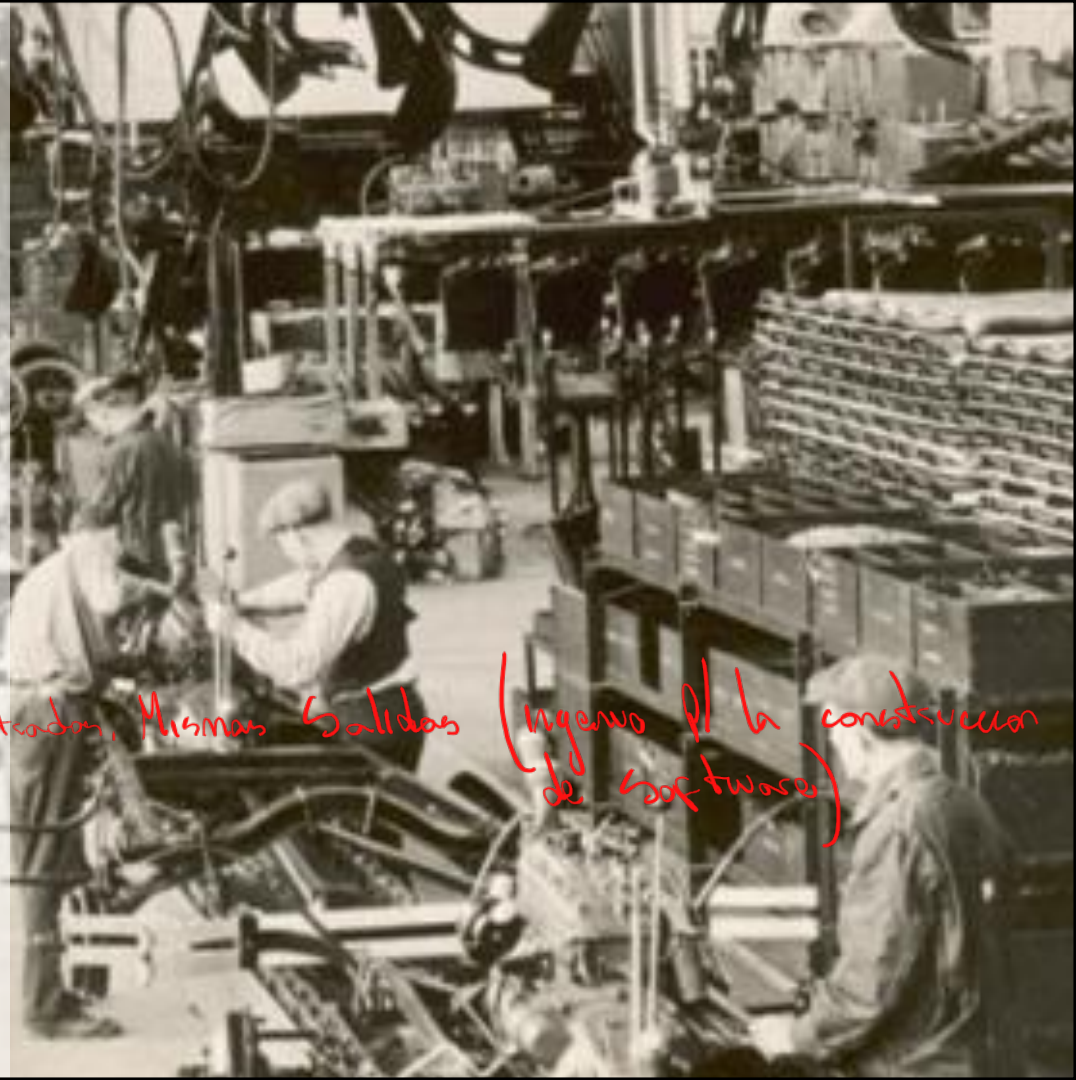
Definido (inspirados en las líneas de producción)

- Asume que podemos repetir el mismo proceso una y otra vez, indefinidamente, y obtener los mismos resultados. → A mismas Entradas, Mismas Salidas

La administración y control provienen de la predictibilidad del proceso definido.

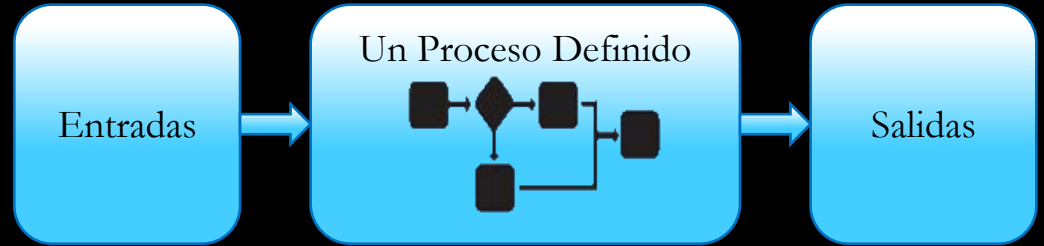
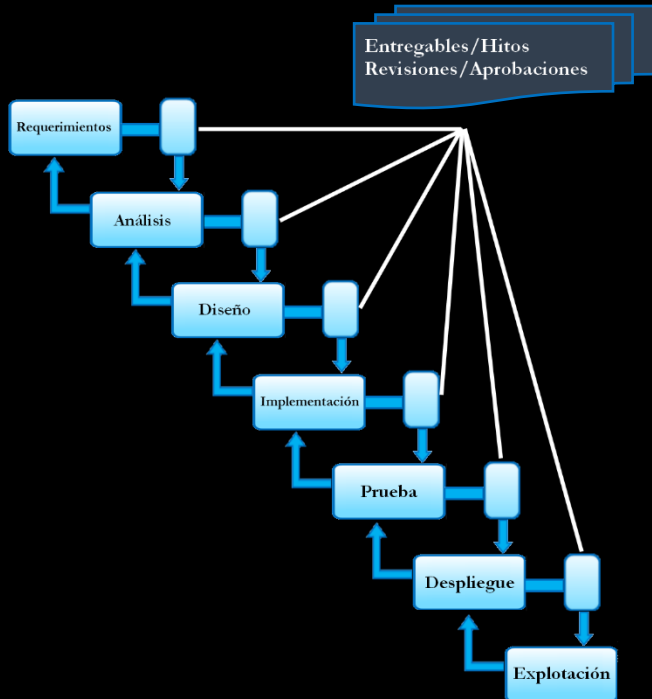
(Ingenio al la construcción de software)

An Assembly Line



Procesos Definidos

→ PUD



Procesos Empíricos



Basado en la Experiencia y el Aprendizaje

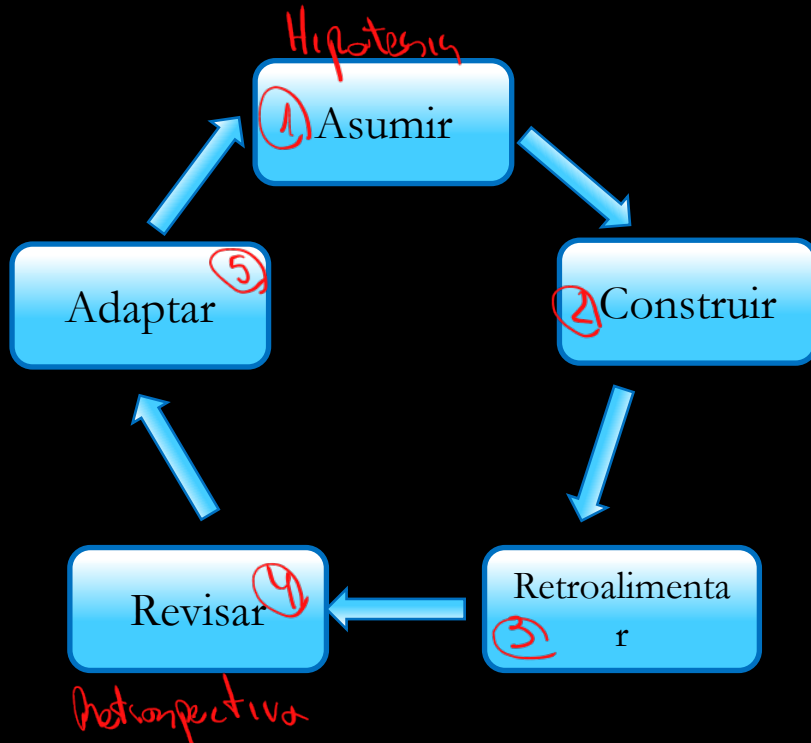
Empírico

- Asume procesos complicados con **variables cambiantes**.
Cuando se repite el proceso, se pueden llegar a obtener resultados diferentes.
- **La administración y control es a través de inspecciones frecuentes y adaptaciones**
- Son procesos que trabajan bien con procesos creativos y complejos. (a que suena??)

→ *Meditamos la cultura Organizacional*

→ *Tienen muy en cuenta a las personas*

Patrón de conocimiento en procesos empíricos



Ciclos de vida

⇒ Es una ABSTRACCION

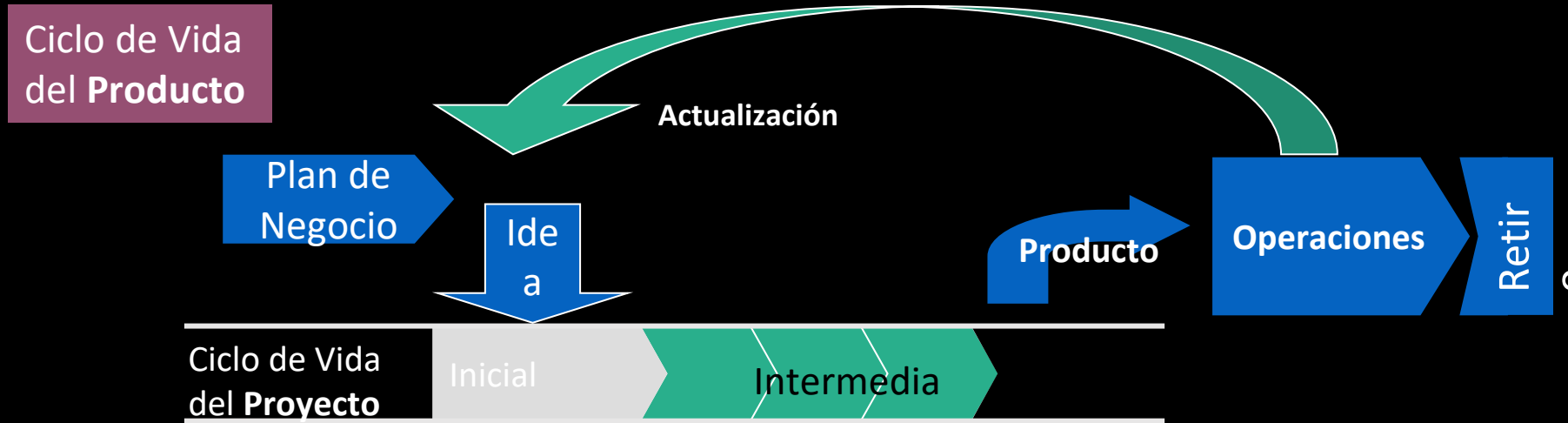
⇒ define : - Etapas
- Orden de las Etapas

La serie de pasos a través de los cuales el producto o proyecto progresa.

Los productos tienen su ciclo de vida.

Los proyectos también.

Relación: Ciclo de Vida del Proyecto y del Producto



Ciclos de Vida de proyectos de software

- Un ciclo de vida de un proyecto software es un representación de un proceso. Grafica una descripción del proceso desde una perspectiva particular
- Los modelos especifican
 - Las fases de proceso.
 - Ejemplo: requerimientos, especificación, diseño...
 - El orden en el cual se llevan a cabo

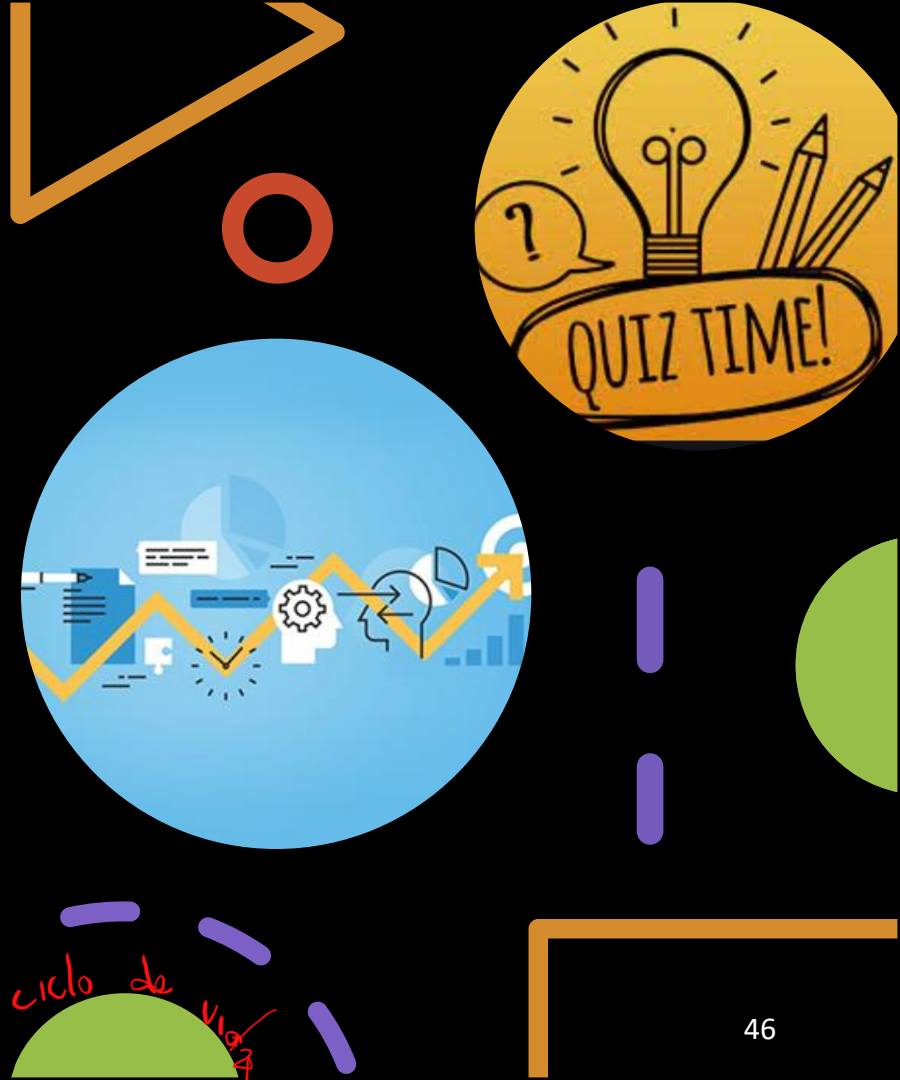
An abstract geometric composition on a black background. It features a large green semi-circle at the bottom left, a red circle on the left, a green semi-circle at the top center, and a red semi-circle at the top right. There are also several purple and orange geometric shapes, including a large orange L-shape at the top left, a purple L-shape at the bottom right, and several small purple and orange lines and shapes scattered throughout.

-
- An abstract geometric composition on a black background. It features a large green semi-circle at the bottom left, a red circle outline on the left, a green semi-circle at the top center, and a red semi-circle at the top right. There are also several purple and orange geometric shapes, including lines, arcs, and a square outline, scattered throughout the composition.

¿Qué ciclos de vida conocen?

¿Qué Relación hay entre procesos de Desarrollo y ciclos de vida?

np/ construir un producto se usa un proceso y ese proceso está dirigido por un tipo de ciclo de vida



→ El proceso es una implementación del ciclo de vida?

De ciclos de vida hay más?

Si! Capítulo 7 de Desarrollos de proyectos informáticos (Rapid Development) de Mcconell

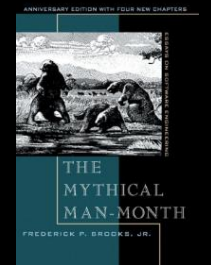
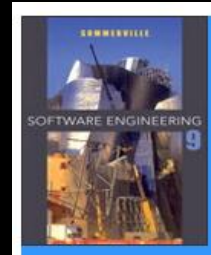
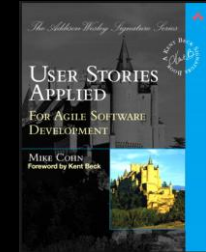
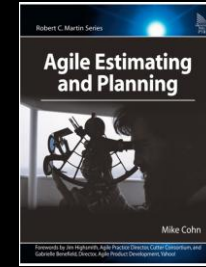
Referencias Bibliográficas Principales

■ Libros:

- Sommerville, Ian - INGENIERÍA DE SOFTWARE - Novena Edición (Editorial Addison-Wesley Año 2011). -
- Brooks, Frederick The mythical man-month (anniversary ed.), 1995 Addison-Wesley Longman Publishing Co., Inc. Boston, MA, USA ©1995
- Steve Mc Connell - Rapid Development Redmond, Wa.: Microsoft Press,
- Cohn, Mike – Agile Estimation and Planning – Editorial Prentice Hall 2006
- Cohn, Mike - User Stories Applied – Editorial Addison Wesley 2004.

■ Papers:

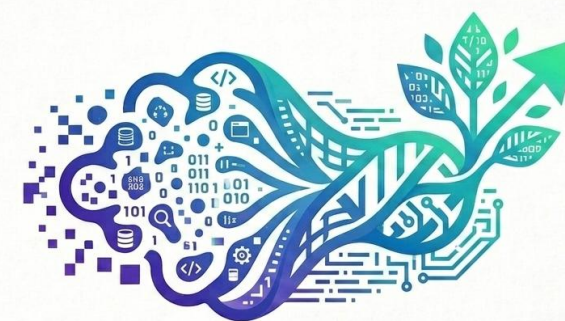
- Orphans Preferred (<http://www.stevemcconnell.com/psd/07-orphanspreferred.htm>)
- No Silver Bullet (<http://www.virtualschool.edu/mon/SoftwareEngineering/BrooksNoSilverBullet.html>)
- Software's Ten Essentials (<http://www.stevemcconnell.com/ieeesoftware/10Essentials.pdf>)
- <http://www.scrumguides.org/download.html>
- Dean Leffingwell and Pete Behrens – A user story primer (2009)
- Manifiesto Ágil <http://agilemanifesto.org/iso/es/>



Universidad Tecnológica Nacional
Facultad Regional Córdoba
Cátedra de Ingeniería y Calidad de Software
Docentes: Judith Meles- Laura Covaro

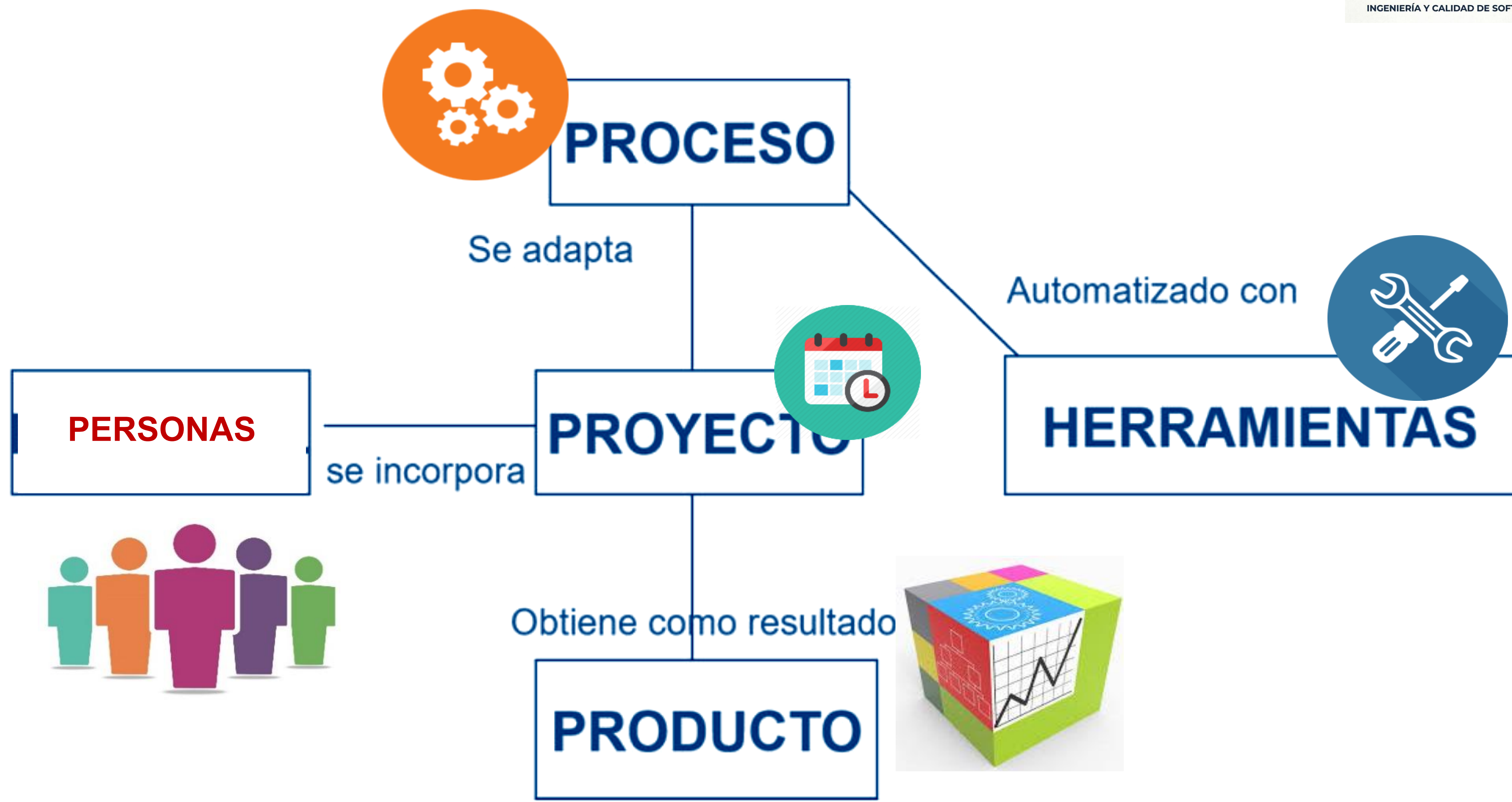
Software Configuration Management (SCM)

(o más allá del Commit, Update)



UTN-FRC
INGENIERÍA Y CALIDAD DE SOFTWARE

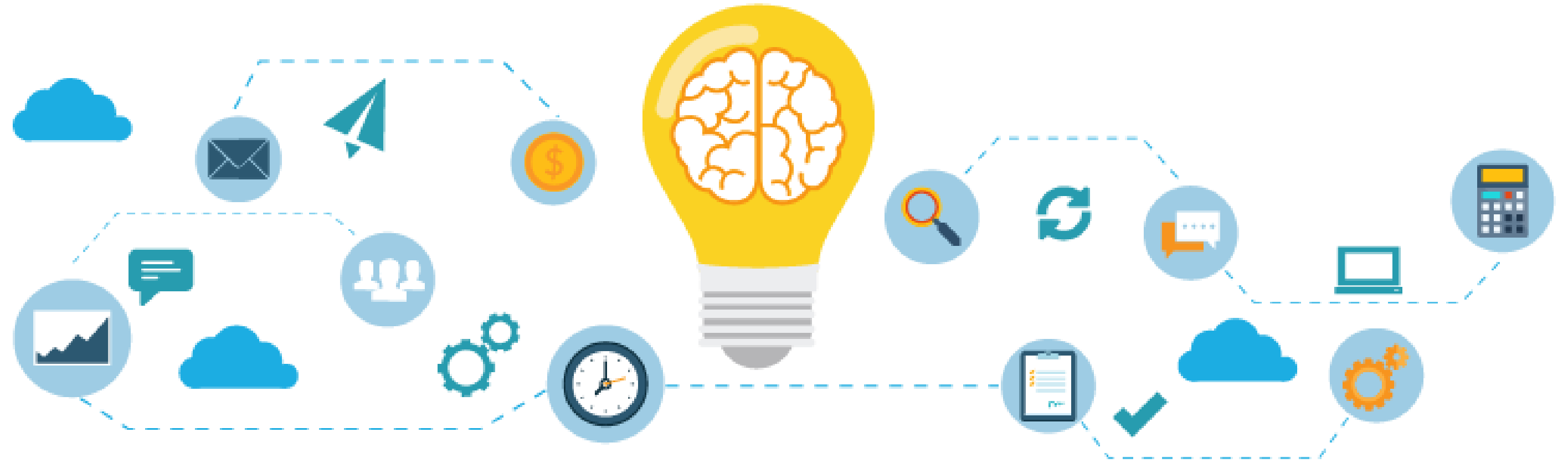
Software en contexto



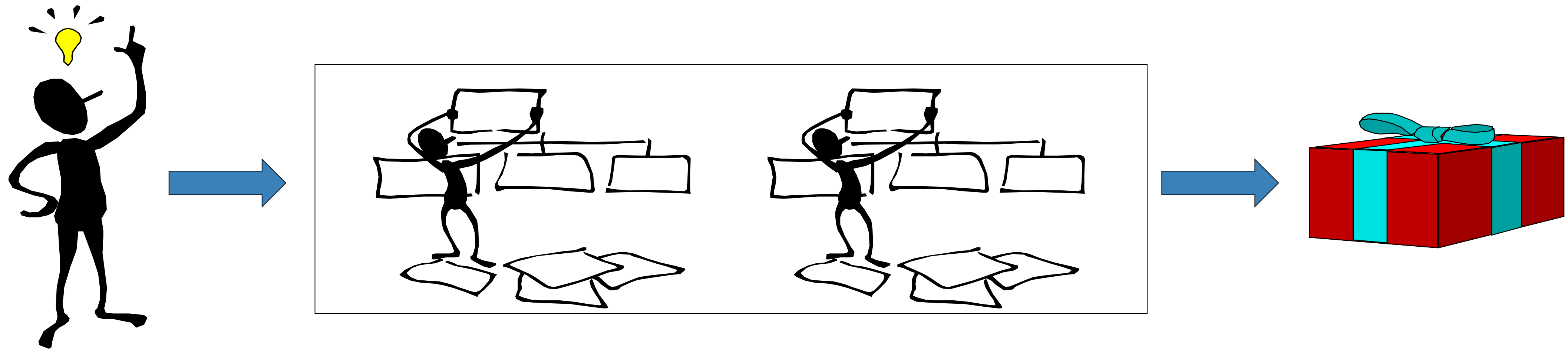
¿Cuándo pensamos en Software... en qué pensamos?

Conjunto de:

- *Programas*
- *Procedimientos*
- *Reglas*
- *Documentación*
- *Datos*



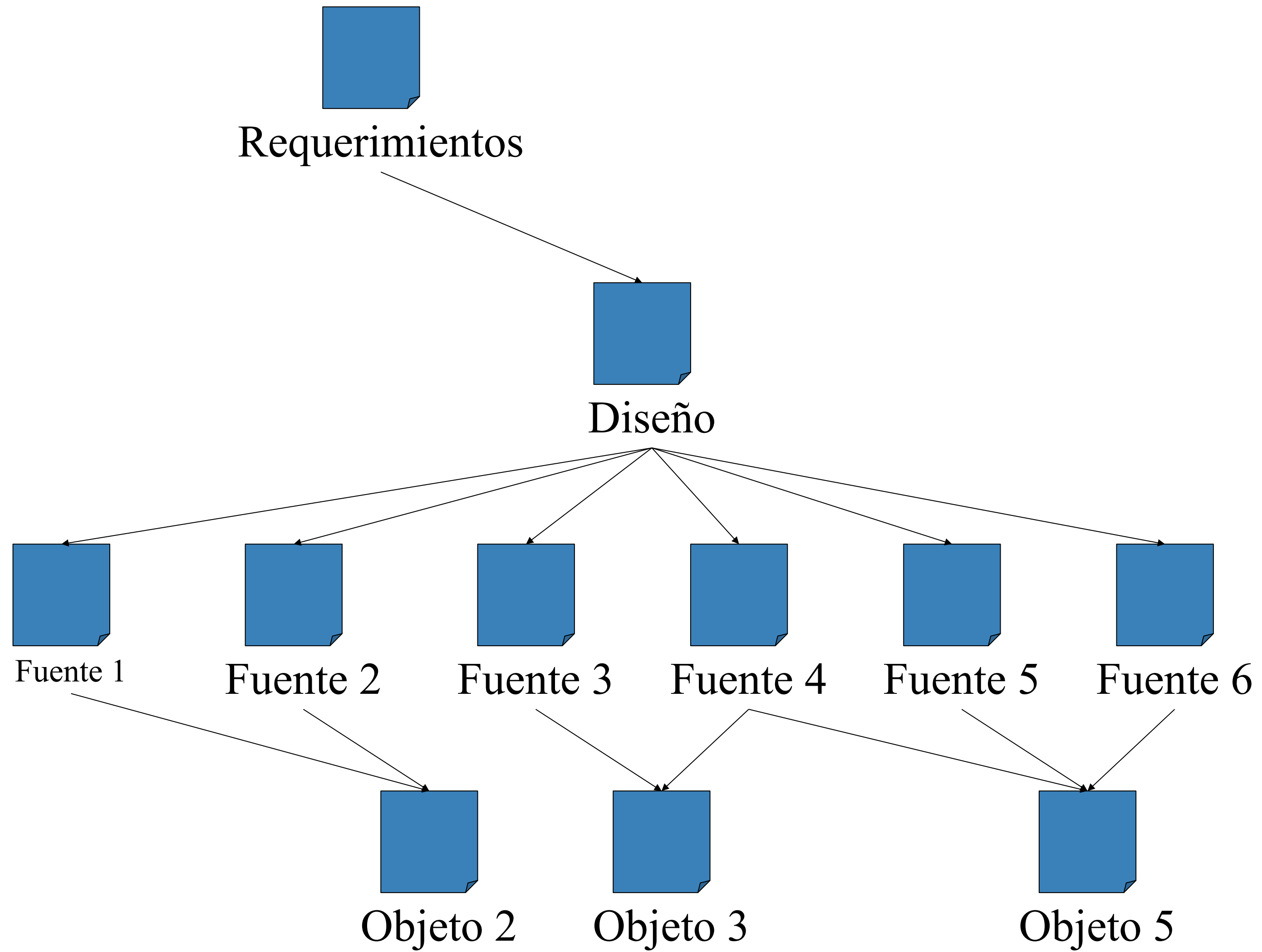
El Software



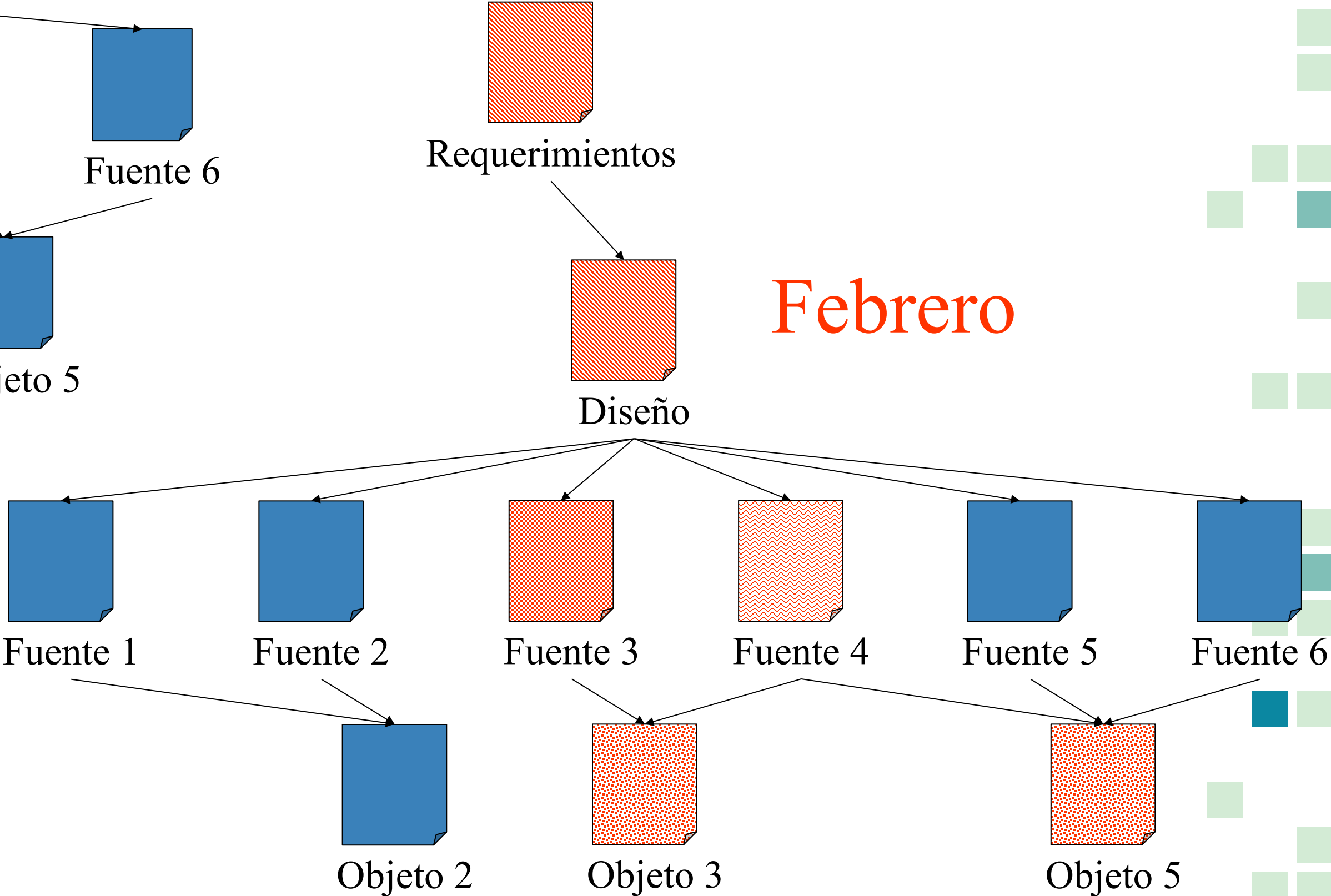
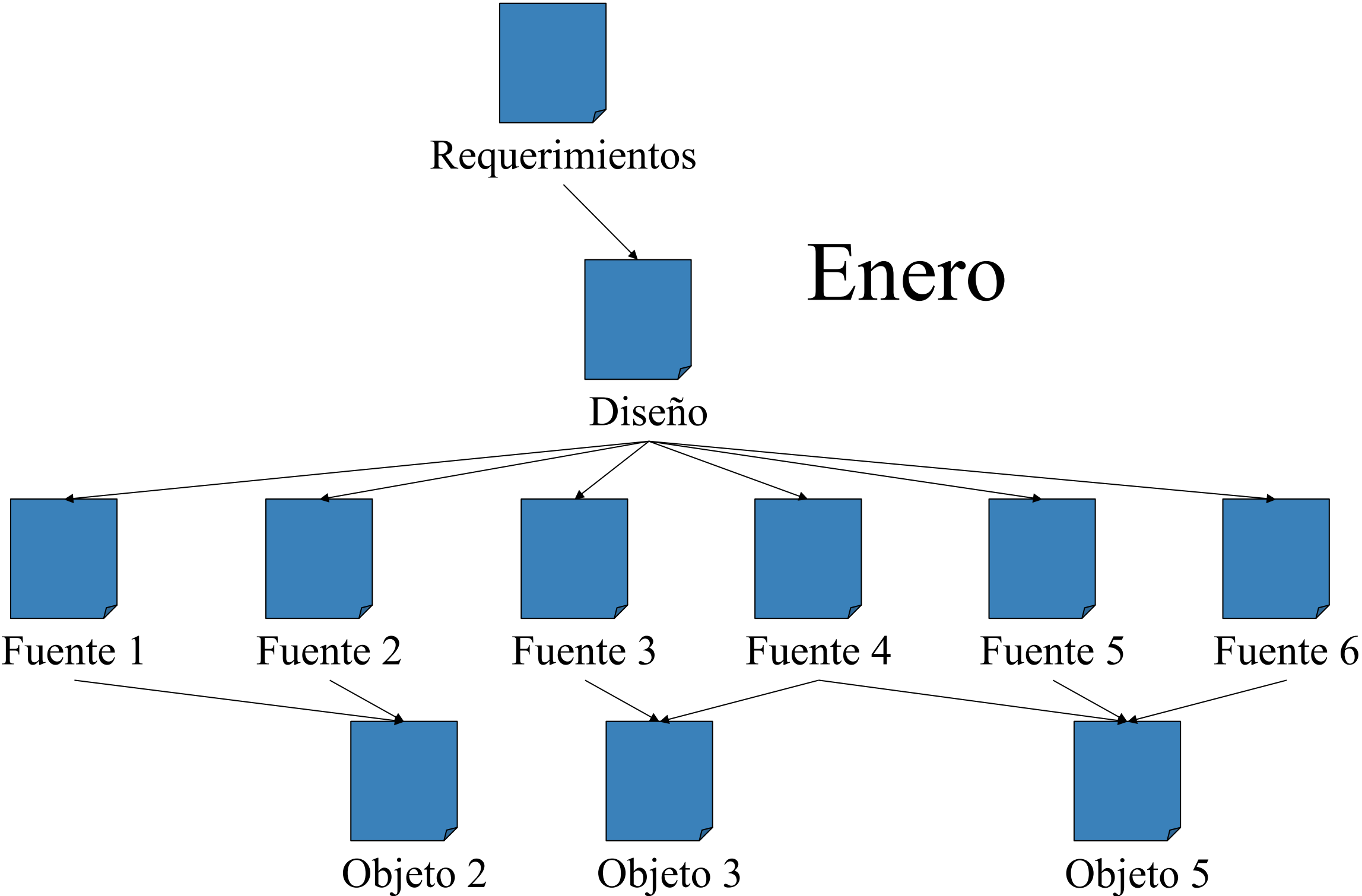
- Información:
 - estructurada con propiedades lógicas y funcionales.
 - creada y mantenida en varias formas y representaciones.
 - confeccionada para ser procesada por computadora en su estado más desarrollado

El software

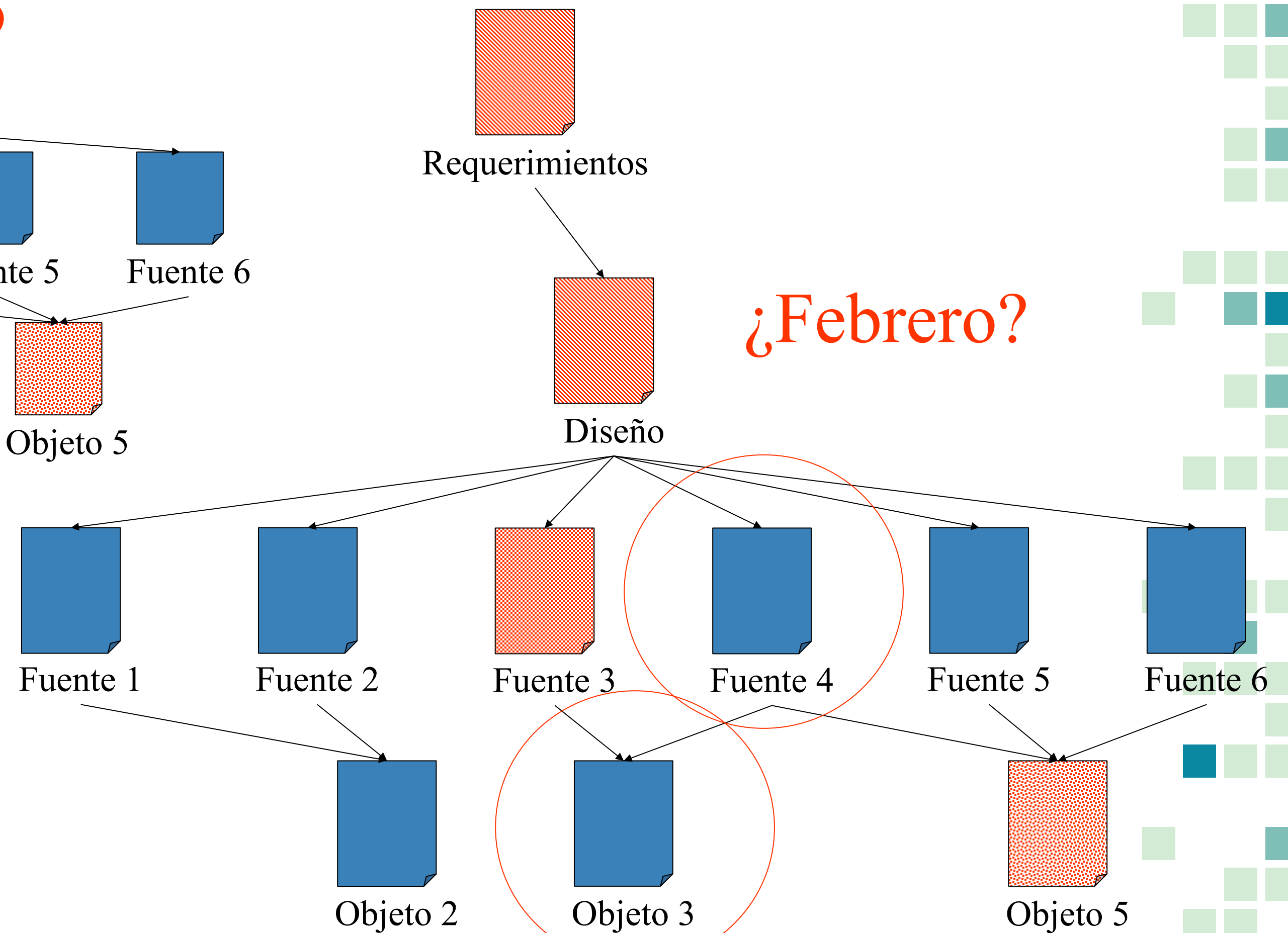
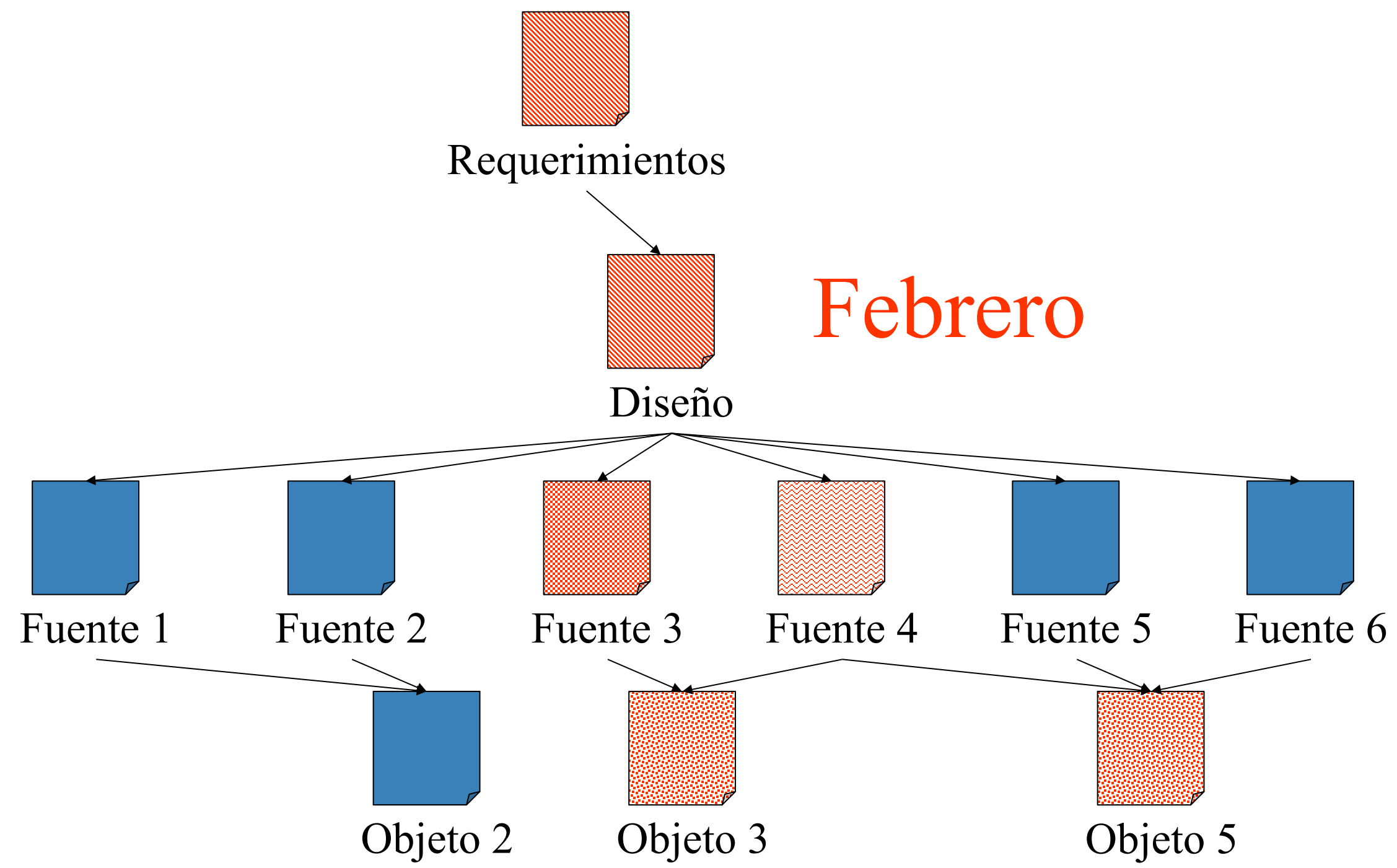
Enero



La evolución del software



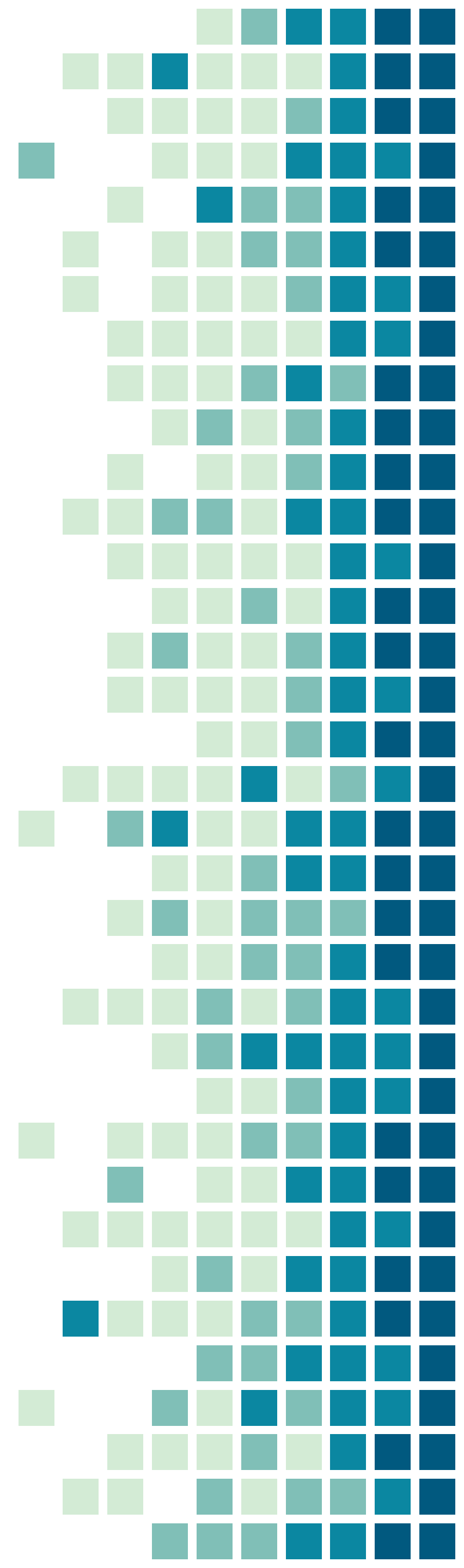
Problemas



Cambios en el Software

Tienen su origen en:

- ❖ Cambios del negocio y nuevos requerimientos
- ❖ Soporte de cambios de productos asociados
- ❖ Reorganización de las prioridades de la empresa por crecimiento
- ❖ Cambios en el presupuesto
- ❖ Defectos encontrados a corregir
- ❖ Oportunidades de mejora



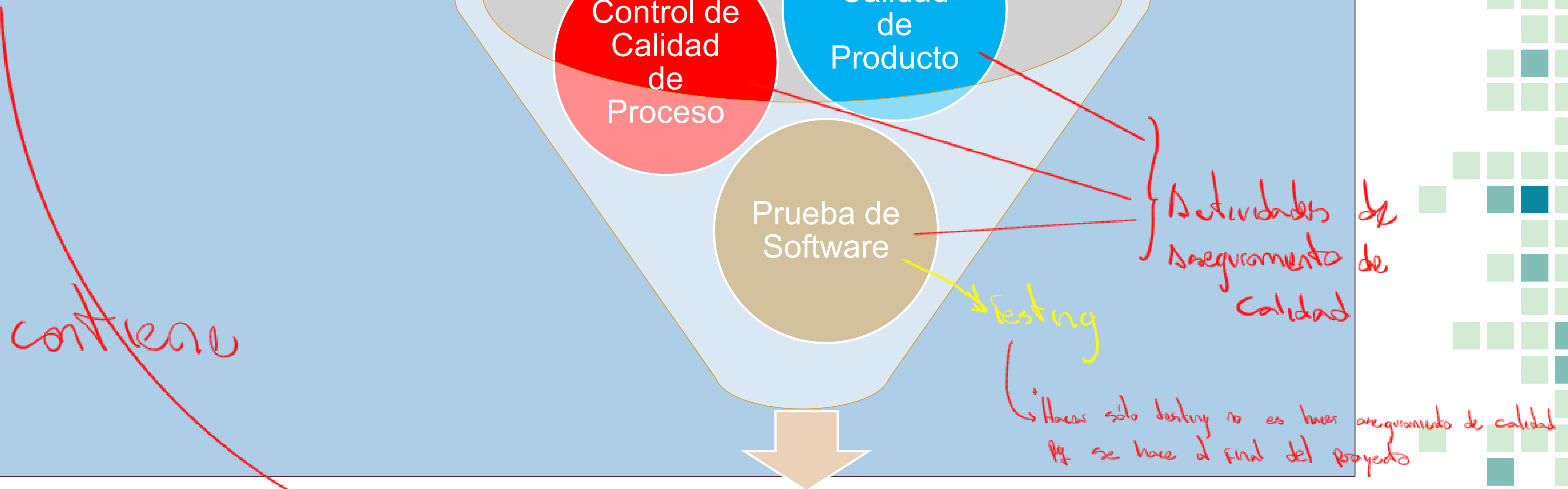
SCM como disciplina de soporte

Es una actividad “paragüas”, transversal a todo el proyecto, relevante para el producto a lo largo de su ciclo de vida.



Disciplinas de soporte del Software

Administración de Configuración de Software



Aseguramiento de Calidad de Software

Un poco de Historia



Tiene su origen a mediados de 1950s, cuando CM (por Configuration Management) originalmente utilizado para desarrollo de hardware y control de producción, fue utilizado en el desarrollo de software.

Definición

Una disciplina que aplica dirección y monitoreo administrativo y técnico a: identificar y documentar las características funcionales y técnicas de los ítems de configuración, controlar los cambios de esas características, registrar y reportar los cambios y su estado de implementación y verificar correspondencia con los requerimientos

(ANSI/IEEE 828, 1990)



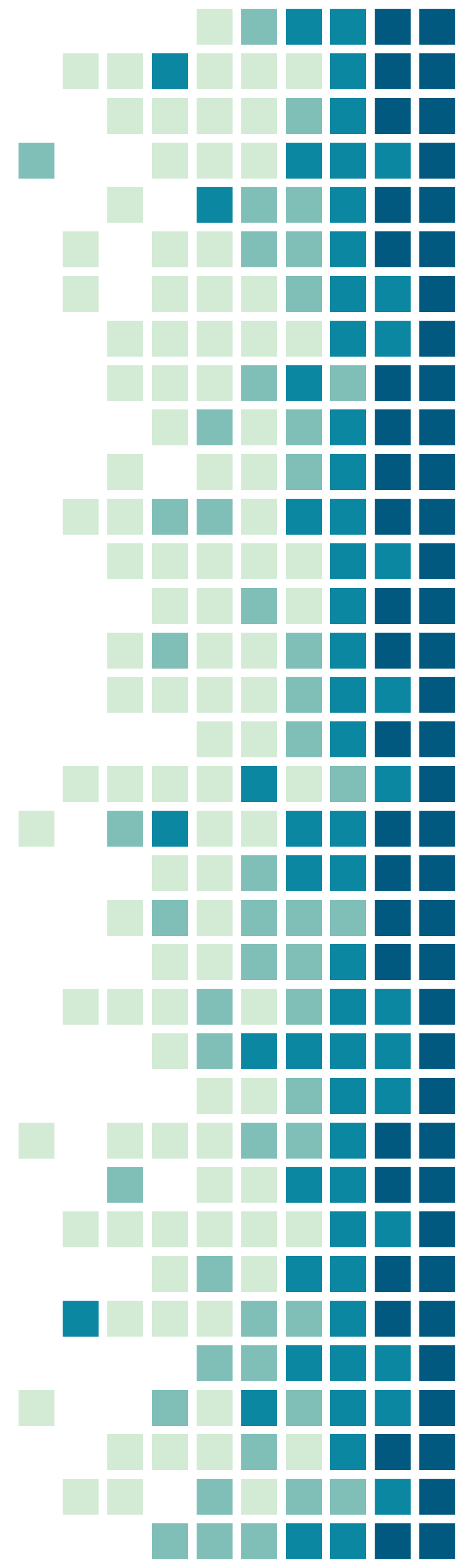
¿Por qué deberíamos gestionar la configuración del software?

Su propósito es establecer y mantener la integridad de los productos de software a lo largo de su ciclo de vida.

Involucra para la configuración:

- ❖ Identificarla en un momento dado
- ❖ Controlar sistemáticamente sus cambios
- ❖ Mantener su integridad y origen

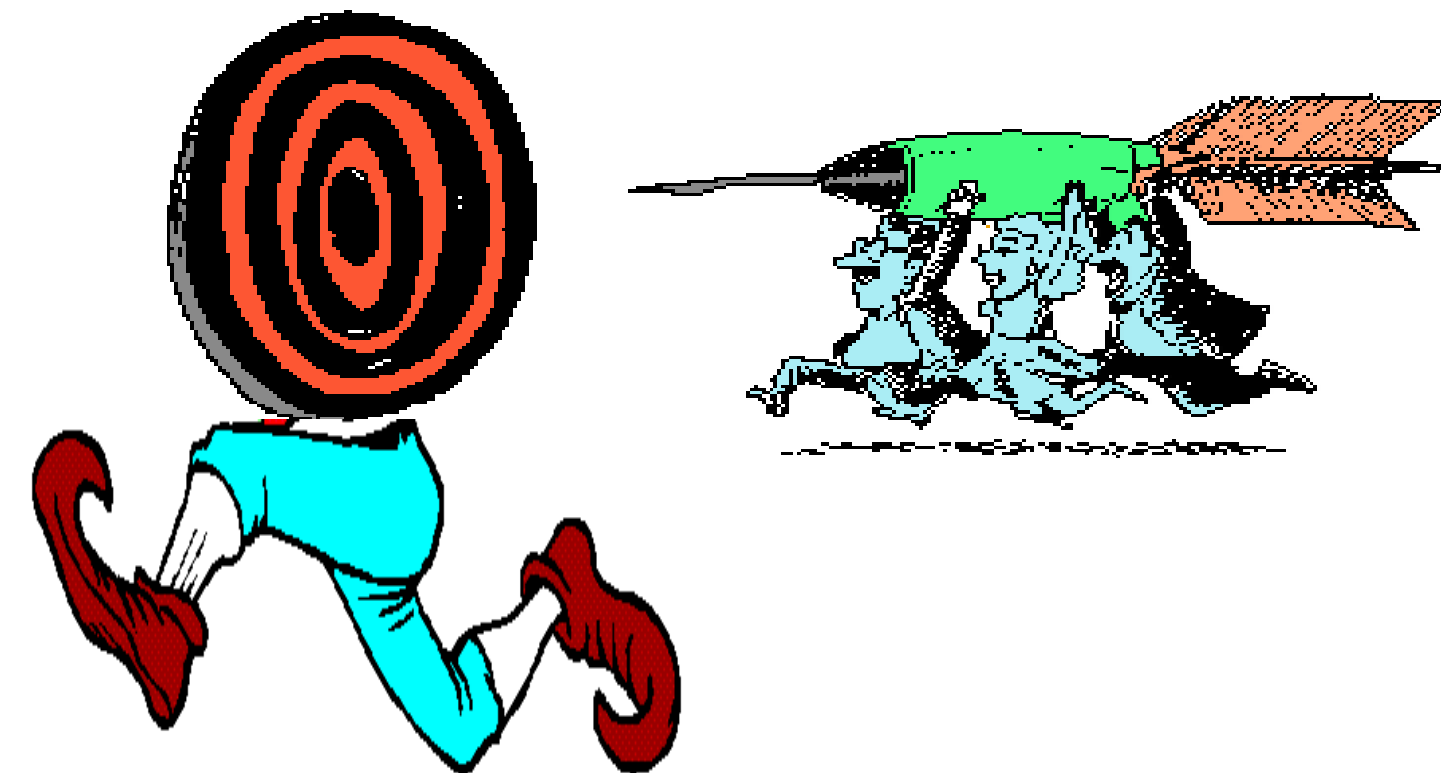
* "Responde a la trazabilidad" $\Rightarrow$ rastreabilidad



Integridad del Producto

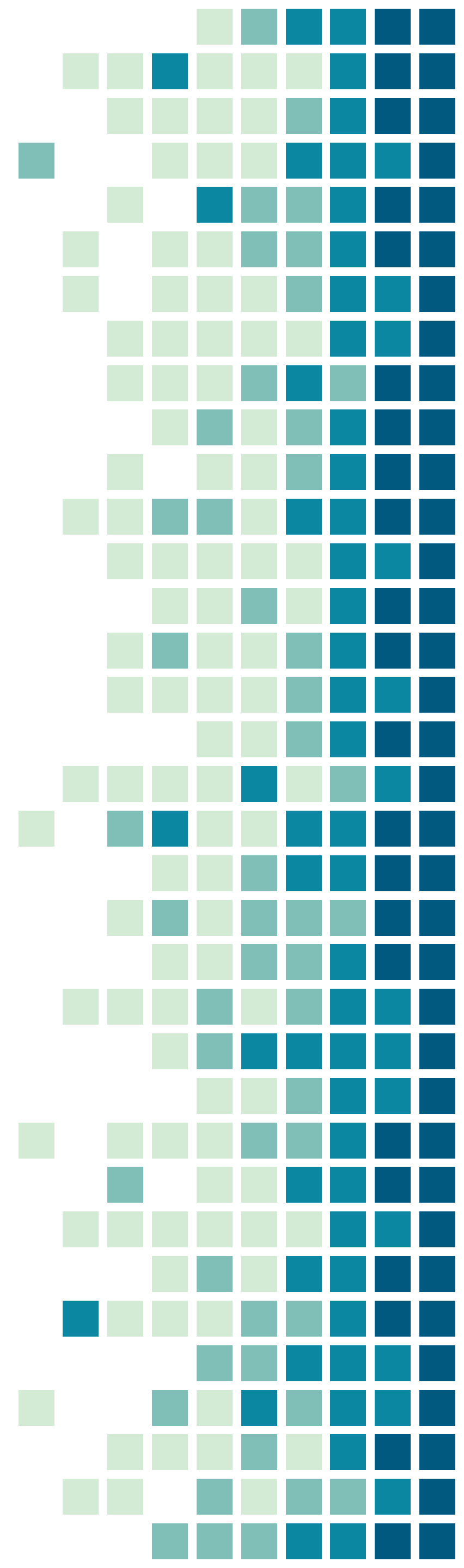
- satisface las necesidades del usuario
- puede ser fácil y completamente rastreado durante su ciclo de vida
- satisface criterios de performance
- cumple con sus expectativas de costo

El software: un blanco móvil



Problemas en el manejo de componentes

- ❖ Pérdida de un componente
- ❖ Pérdida de cambios (el componente que tengo no es el último)
- ❖ Sincronía fuente - objeto – ejecutable
- ❖ Regresión de fallas
- ❖ Doble mantenimiento
- ❖ Superposición de cambios
- ❖ Cambios no validados



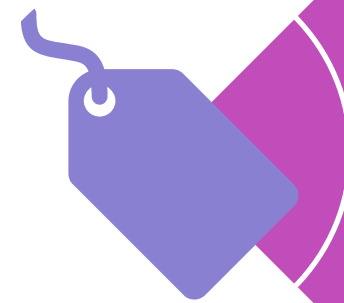
Algunos Conceptos Clave para la Gestión de Configuración de Software



Ítem de Configuración



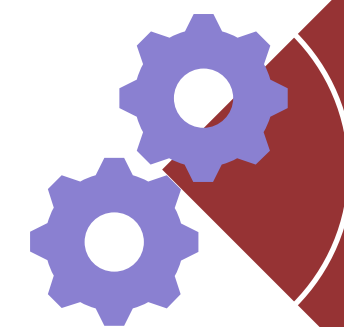
Repositorio



Línea Base



Ramas (Branch)

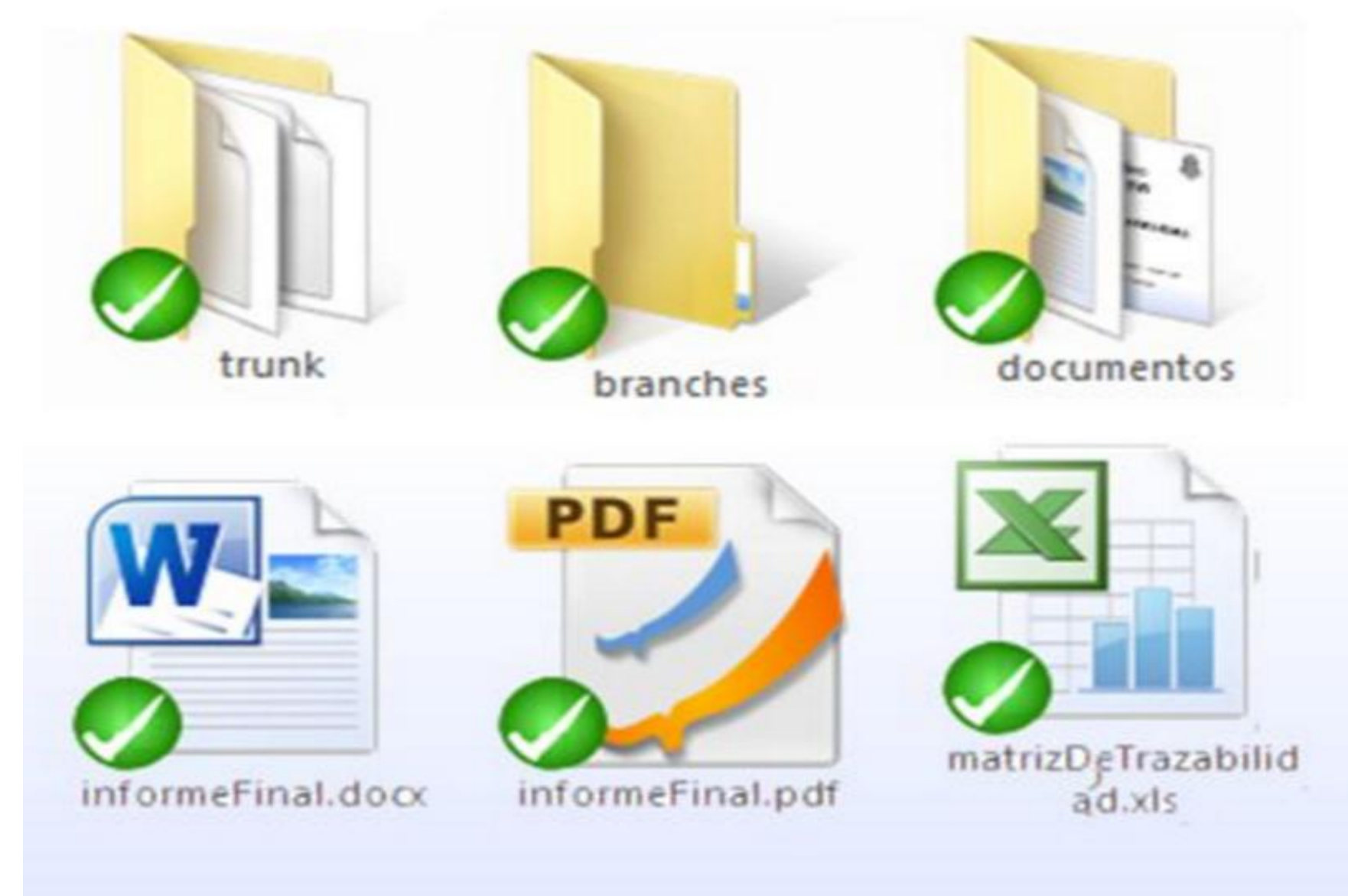


Configuración del Software

Ítem de Configuración de Software (SCI)

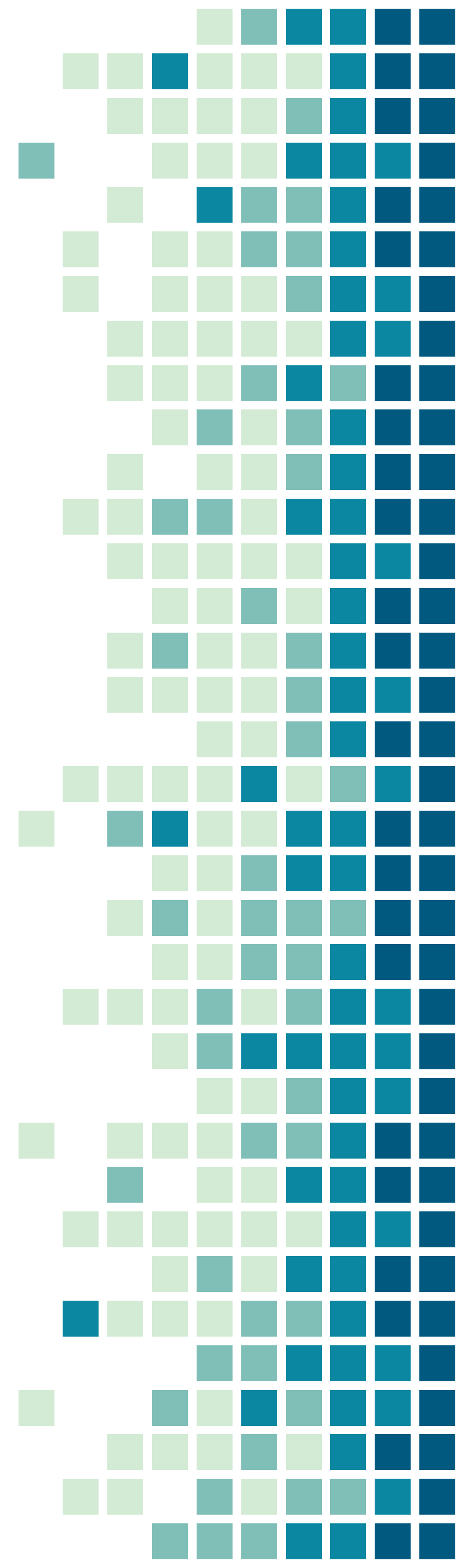
- ✓ Documentos de diseño, código fuente, código ejecutable, etc.

*Se llama **ítem de configuración (IC)** a todos y cada uno de los artefactos que forman parte del producto o del proyecto, que pueden sufrir cambios o necesitan ser compartidos entre los miembros del equipo y sobre los cuales necesitamos conocer su estado y evolución.*



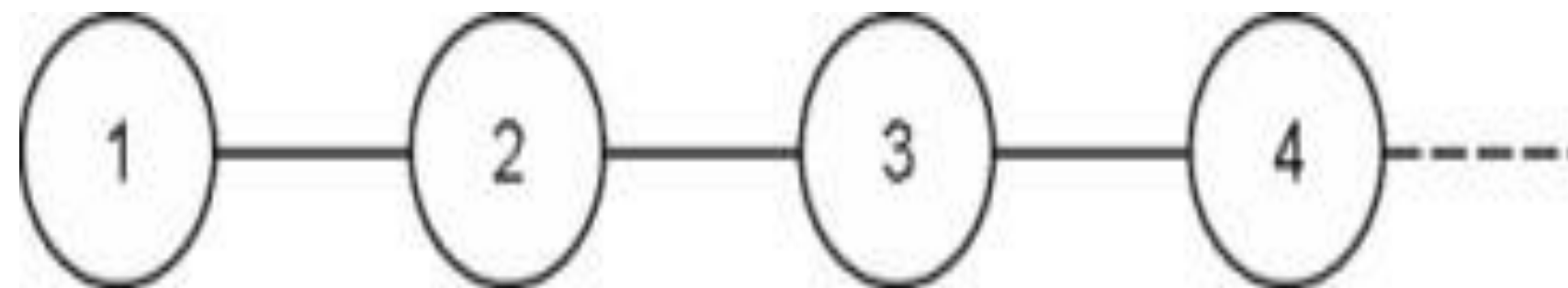
Algunos ejemplos de Ítems de Configuración

- ❖ Plan de CM
- ❖ Propuestas de Cambio
- ❖ Visión
- ❖ Riesgos
- ❖ Plan de desarrollo
- ❖ Prototipo de Interfaz
- ❖ Guía de Estilo de IHM
- ❖ Manual de Usuario
- ❖ Requerimientos
- ❖ Plan de Calidad
- ❖ Arquitectura del Software
- ❖ Plan de Integración
- ❖ Planes de Iteración
- ❖ Estándares de codificación
- ❖ Casos de prueba
- ❖ Código fuente
- ❖ Gráficos, iconos, ...
- ❖ Instructivo de ensamble
- ❖ Programa de instalación
- ❖ Documento de despliegue
- ❖ Lista de Control de entrega
- ❖ Formulario de aceptación
- ❖ Registro del proyecto

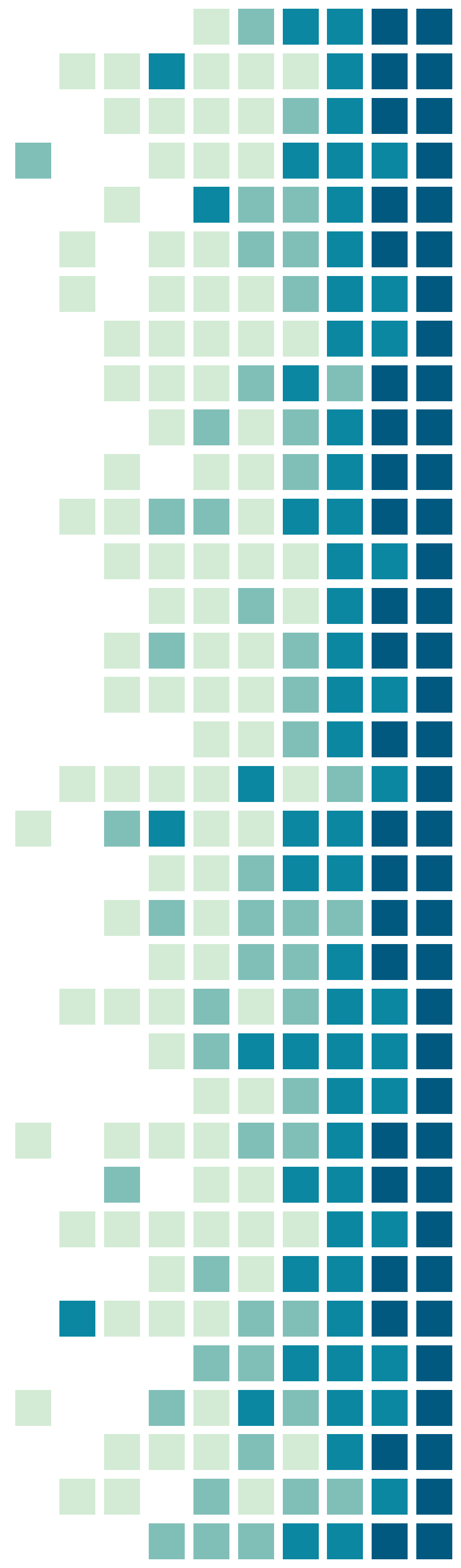


Versión

- Una versión se define, desde el punto de vista de la evolución, como la forma particular de un artefacto en un instante o contexto dado.
- El control de versiones se refiere a la evolución de un único ítem de configuración (IC), o de cada IC por separado.
- La evolución puede representarse gráficamente en forma de grafo.



Evolución lineal de un ítem de configuración



Variante

- ❖ Una variante es una versión de un ítem de configuración (o de la configuración) que evoluciona por separado.
- ❖ Las variantes representan configuraciones alternativas.
- ❖ Un producto de software puede adoptar distintas formas (configuraciones) dependiendo del lugar donde se instale.
- ❖ Por ejemplo, dependiendo de la plataforma (máquina + S.O.) que la soporta, o de las funciones opcionales que haya de realizar o no.

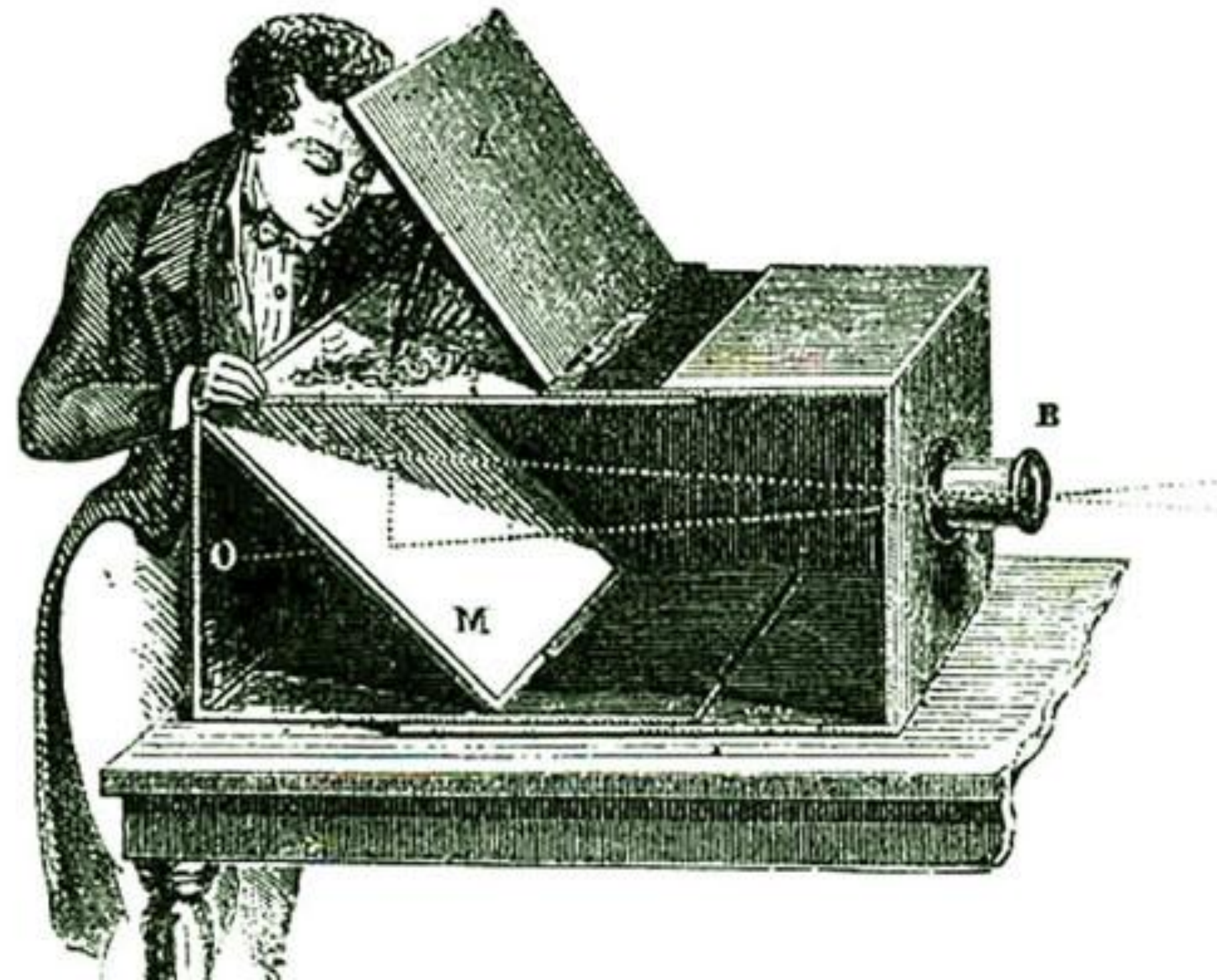


Variante de un ítem de configuración



La Configuración del Software

Un conjunto de ítems de configuración con su correspondiente versión en un momento determinado



¿Qué es un Repositorio?



- ❖ Un repositorio de información conteniendo los ítems de configuración (ICs)
- ❖ Mantiene la historia de cada IC con sus atributos y relaciones.
- ❖ Usado para hacer evaluaciones de impacto de los cambios propuestos.
- ❖ Pueden ser una o varias bases de datos



Ejemplo de Repositorio...

Vínculos favoritos

Documentos

Imágenes

Música

Más >>

Carpetas

Personal Trainer

project21

- .svn
- 2008
 - .svn
 - Exámenes finales
 - Material Entregado a Alumnos
 - Modalidad y planificación
 - Parciales
 - PPAI
- 2009
 - .svn
 - Exámenes finales
 - Material Entregado a Alumnos
 - Modalidad y planificación
 - Parciales
 - PPAI
- 2010
 - Apuntes y Material
- Ejercicios
 - .svn
 - Complejo de Cine
 - Escuela Musical
 - Estación de Servicios
 - Festival de Folklore

Nombre	Fecha modificación	Tipo	Tamaño	Etiquetas
.svn	27/05/2010 12:24 a...	Carpeta de archivos		
Complejo de Cine	26/04/2010 10:48 ...	Carpeta de archivos		
Escuela Musical	20/04/2010 12:22 a...	Carpeta de archivos		
Estación de Servicios	27/05/2010 12:24 a...	Carpeta de archivos		
Festival de Folklore	11/05/2010 01:02 a...	Carpeta de archivos		
Guarda Documental	27/05/2010 12:22 a...	Carpeta de archivos		
Liga de Fútbol	04/05/2010 04:27 a...	Carpeta de archivos		
Medidores de Agua	27/05/2010 12:23 a...	Carpeta de archivos		
Notificación Compras	27/05/2010 12:23 a...	Carpeta de archivos		
Seguimiento de Defect...	27/03/2010 02:02 ...	Carpeta de archivos		
Simulador de Compras	27/05/2010 12:27 a...	Carpeta de archivos		
Solicitud de Préstamo	27/05/2010 12:23 a...	Carpeta de archivos		
Tours	03/05/2010 10:29 ...	Carpeta de archivos		
Venta Digital	14/03/2010 07:26 ...	Carpeta de archivos		
Voto Electrónico	20/04/2010 12:23 a...	Carpeta de archivos		

15 elementos

Ejercicios

Bandeja de entrada ...

Curso de SCM(1) [M...

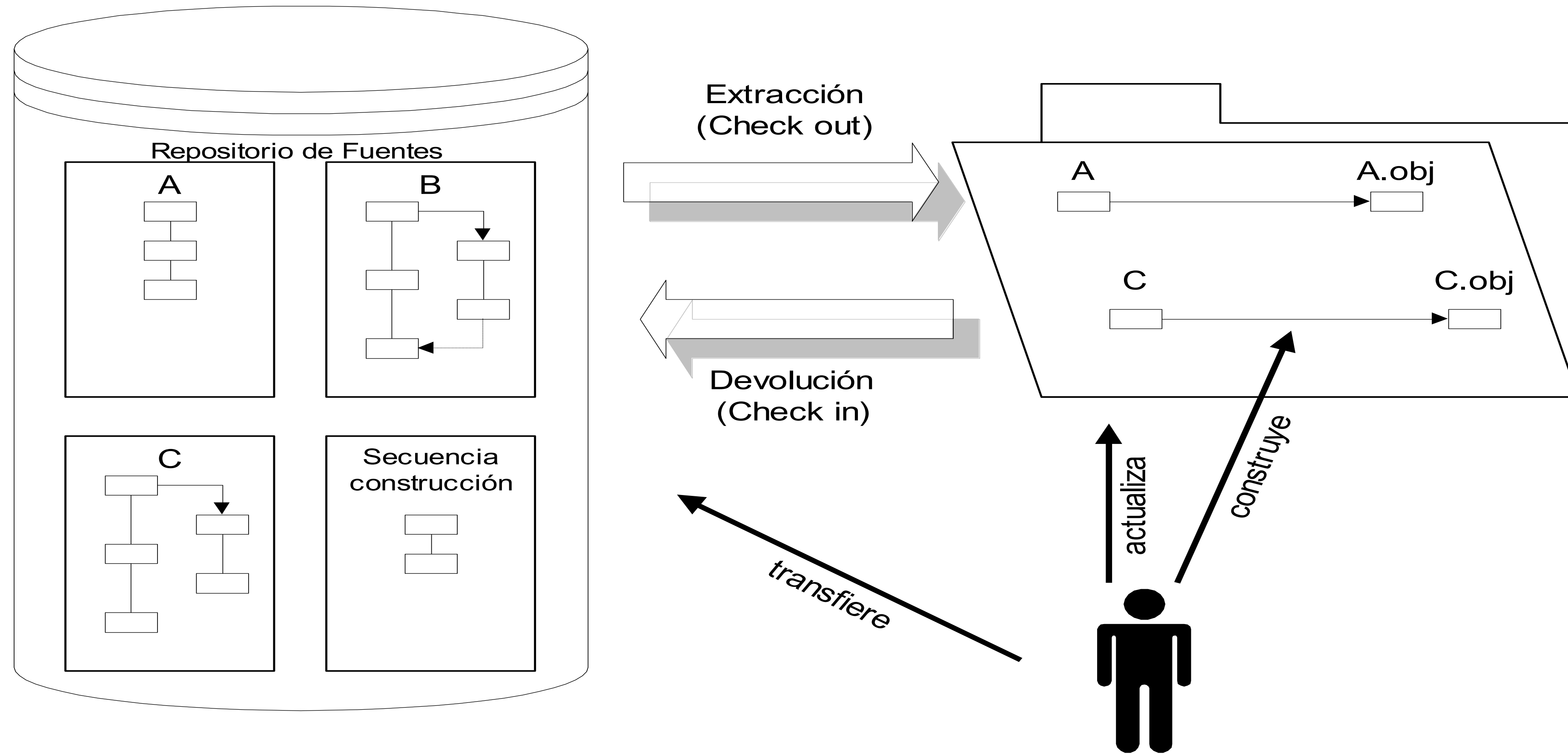
R&D SCM 05 2010 I...

R&D SCM 05 2010

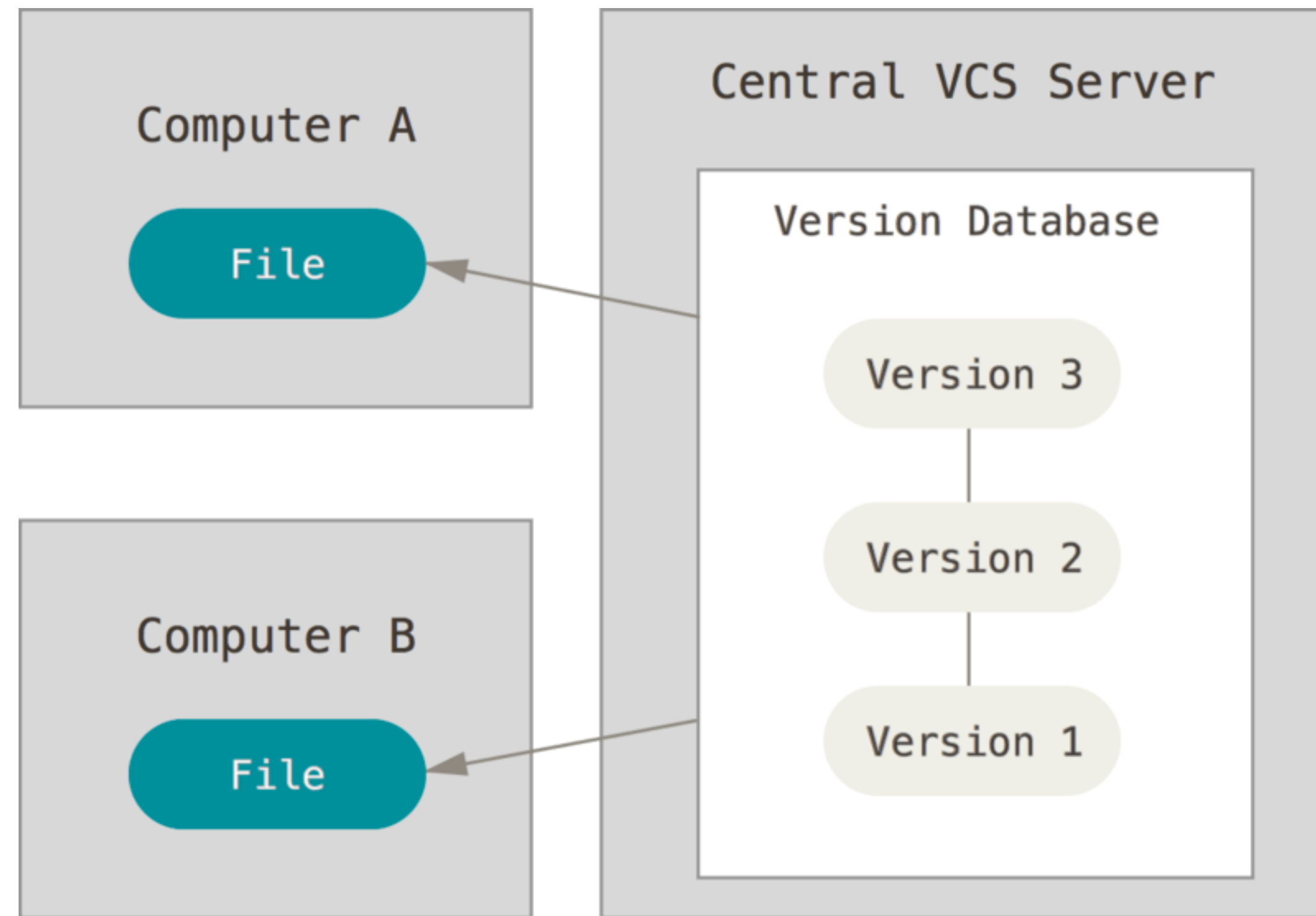
ES

11:08 a.m.

Funcionamiento del Repositorio

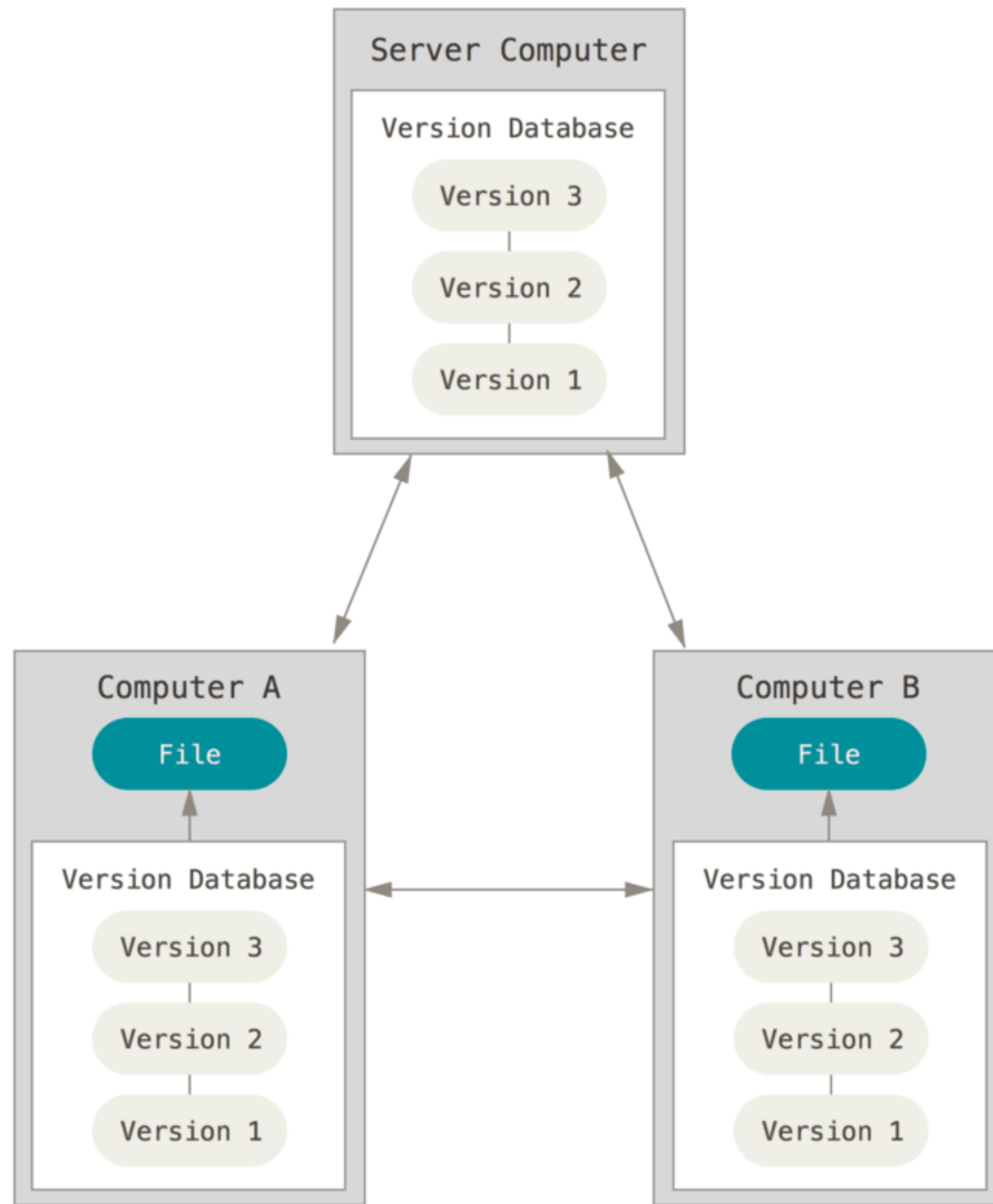


Repositorios Centralizados



- ❖ Un servidor contiene todos los archivos con sus versiones.
- ❖ Los administradores tiene mayor control sobre el repositorio.
- ❖ Falla el servidor y "estamos al horno".

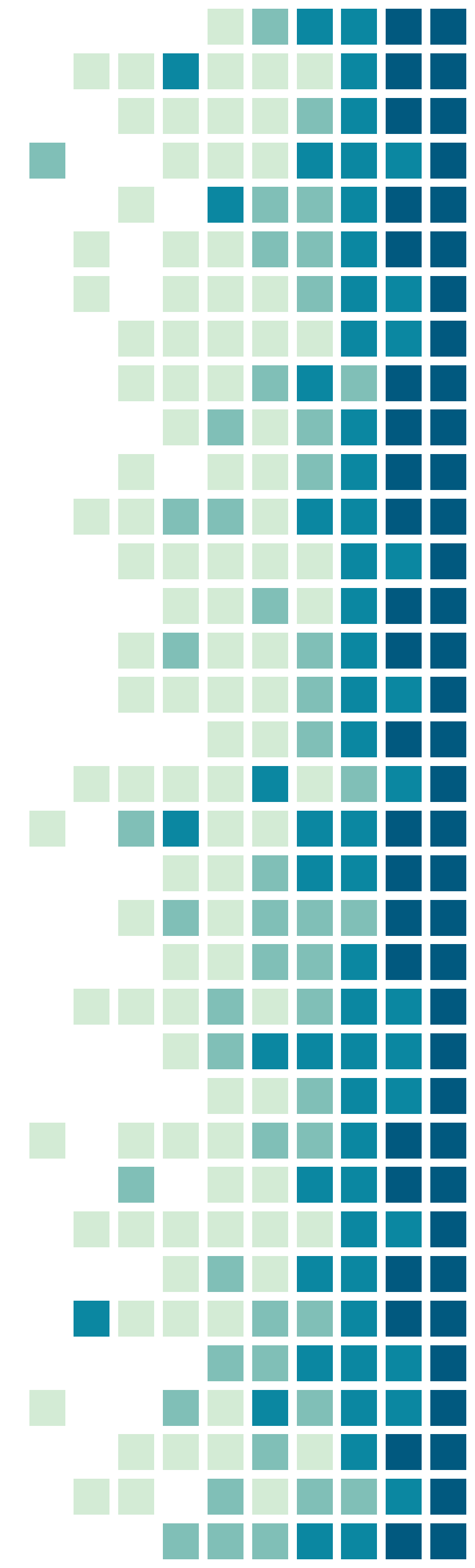
Repositorios Descentralizados



- ❖ Cada cliente tiene una copia **exactamente** igual del repositorio completo.
- ❖ Si un servidor falla sólo es cuestión de "copiar y pegar".
- ❖ Posibilita otros workflows no disponibles en el modelo centralizado.

Identificación de la Línea Base

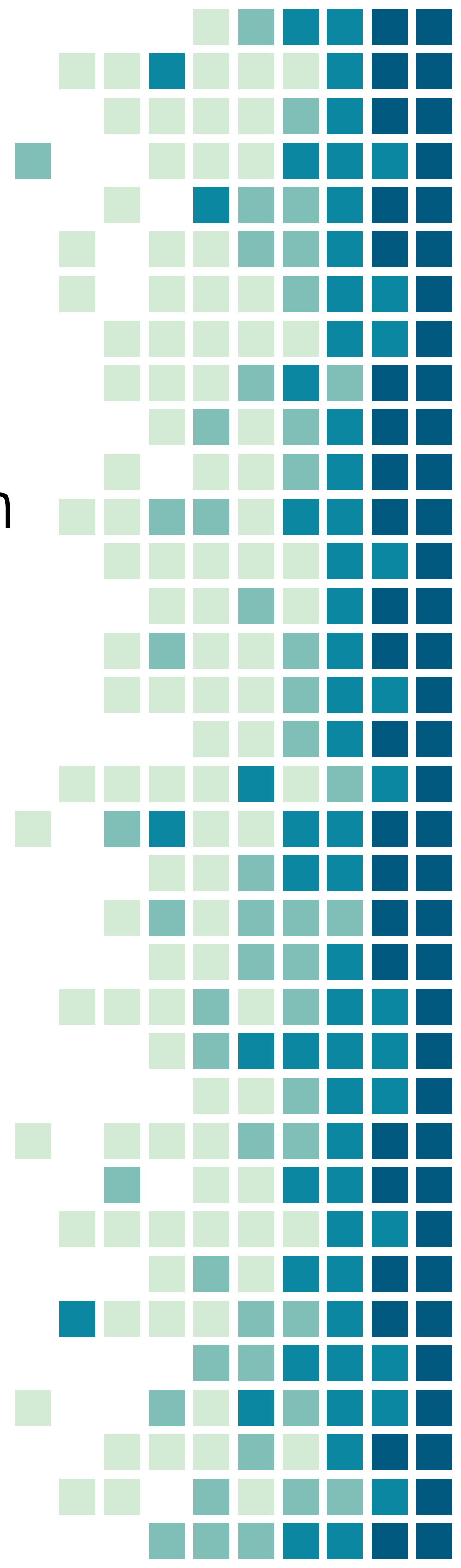
- ❖ Se utilizan etiquetas para “marcar” las baseline
- ❖ No confundir con la versión del Producto



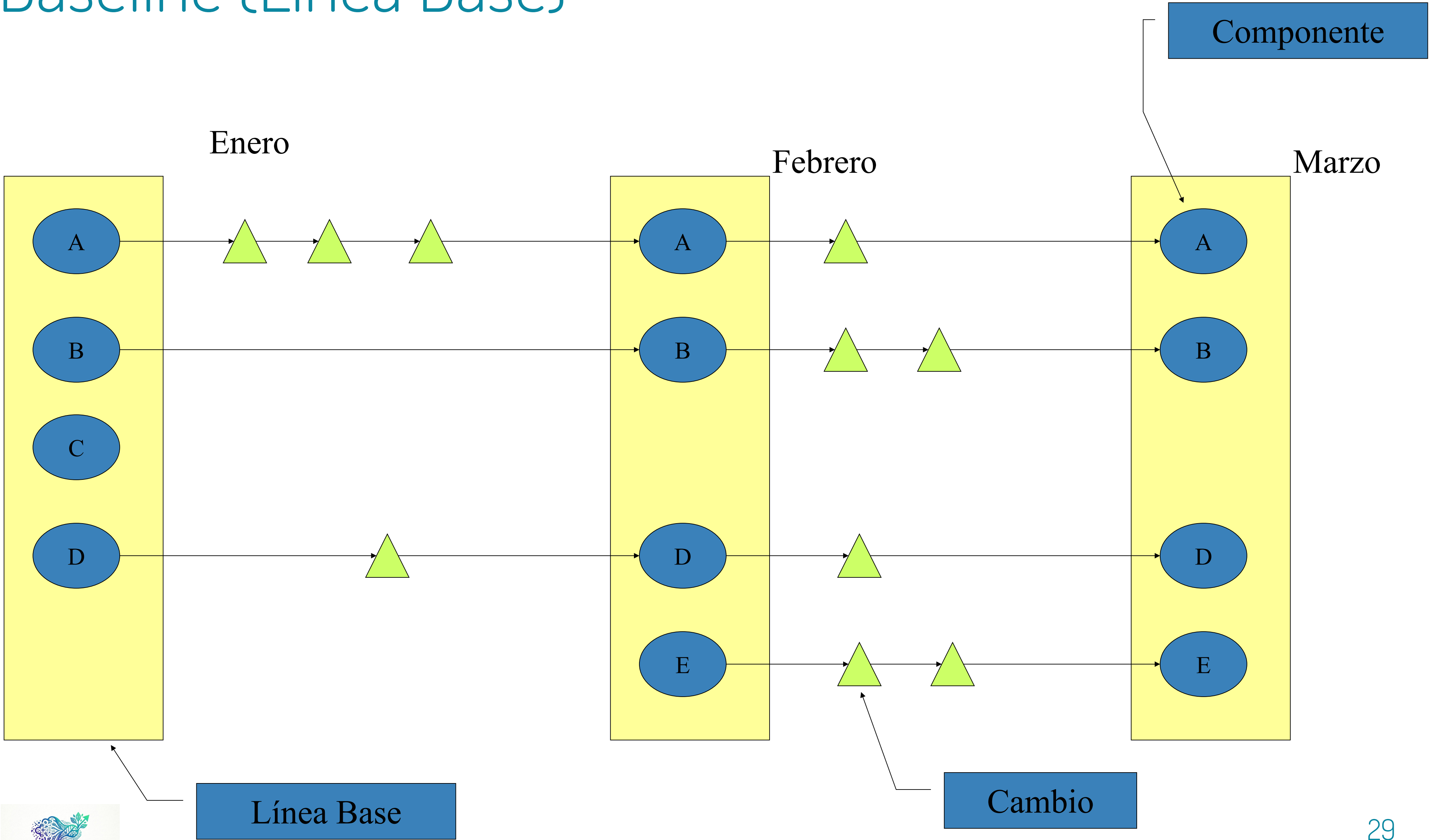
Líneas Base



- ❖ Una configuración que ha sido revisada formalmente y sobre la que se ha llegado a un acuerdo
- ❖ Sirve como base para desarrollos posteriores y puede cambiarse sólo a través de un procedimiento formal de control de cambios
- ❖ Permiten ir atrás en el tiempo y reproducir el entorno de desarrollo en un momento dado del proyecto



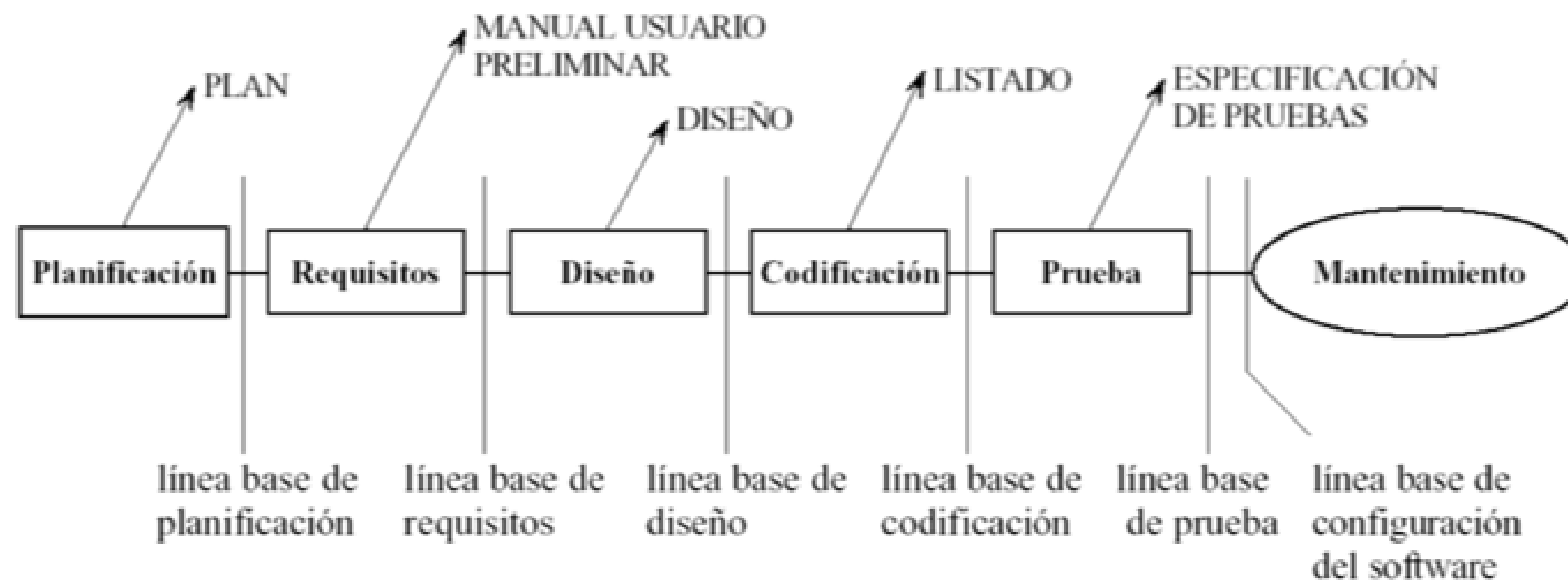
Baseline (Línea Base)



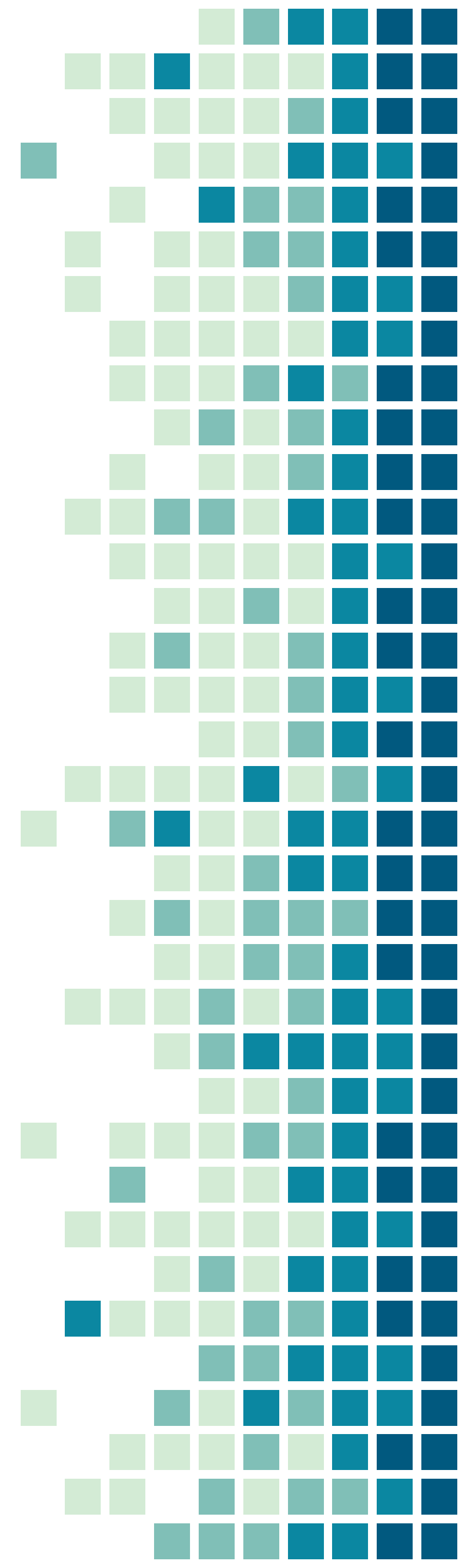
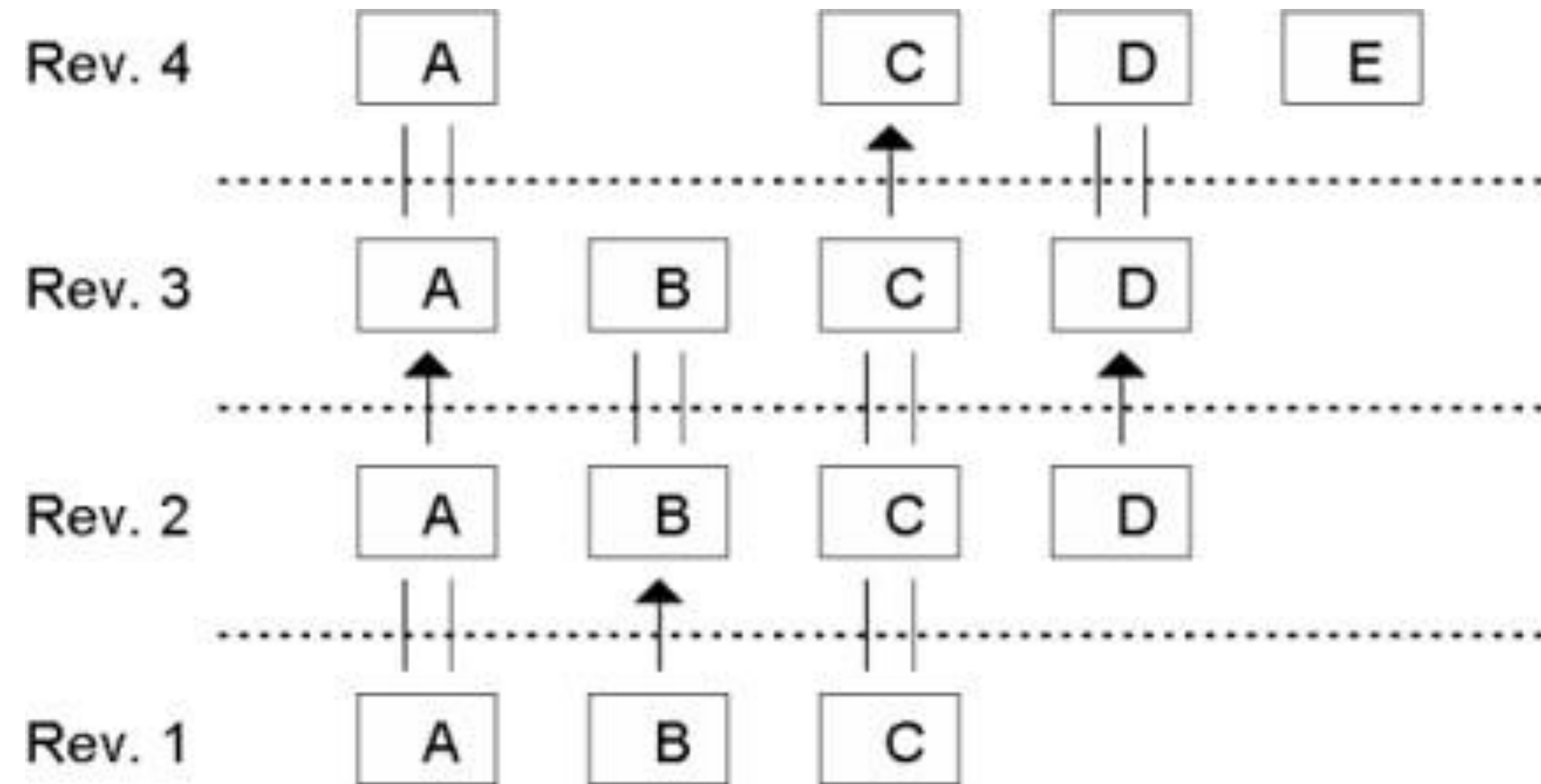
Representación de Líneas Base

Pueden ser:

- De especificación (Requerimientos, Diseño)
- De productos que han pasado por un control de calidad definido previamente



Evolución de una configuración

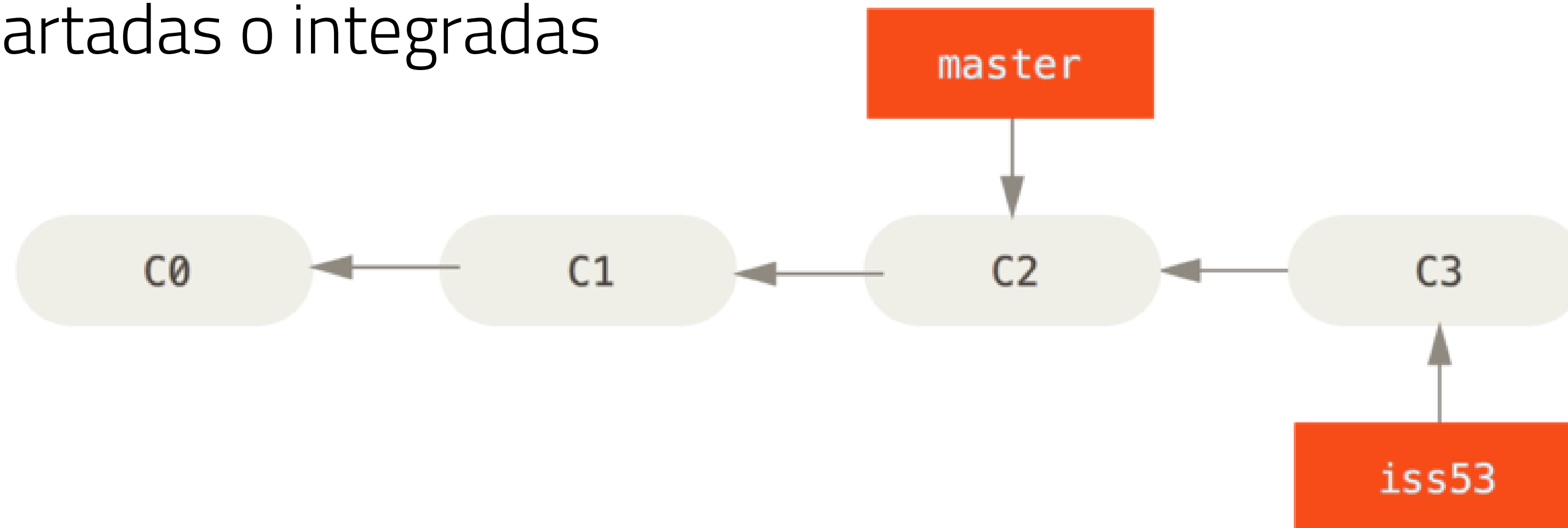




Ramas

Creación de ramas

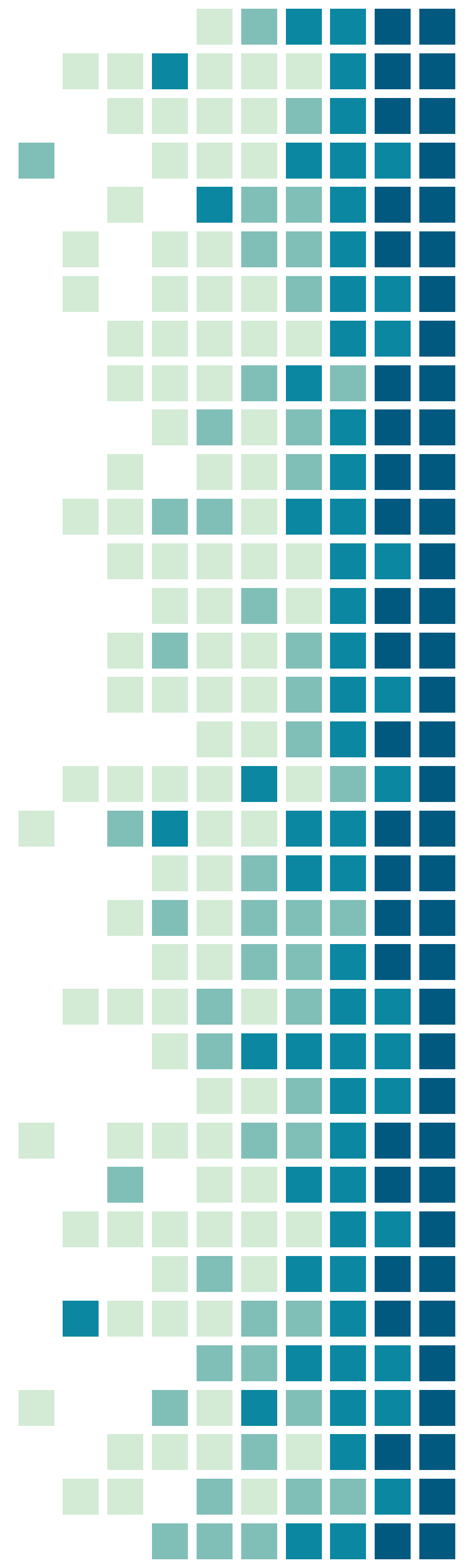
- ❖ Existe una rama principal (trunk, master)
- ❖ Sirven para bifurcar el desarrollo
- ❖ Pueden tener razones de creación con semántica
- ❖ Permiten la experimentación
- ❖ Pueden ser descartadas o integradas



Integración de ramas



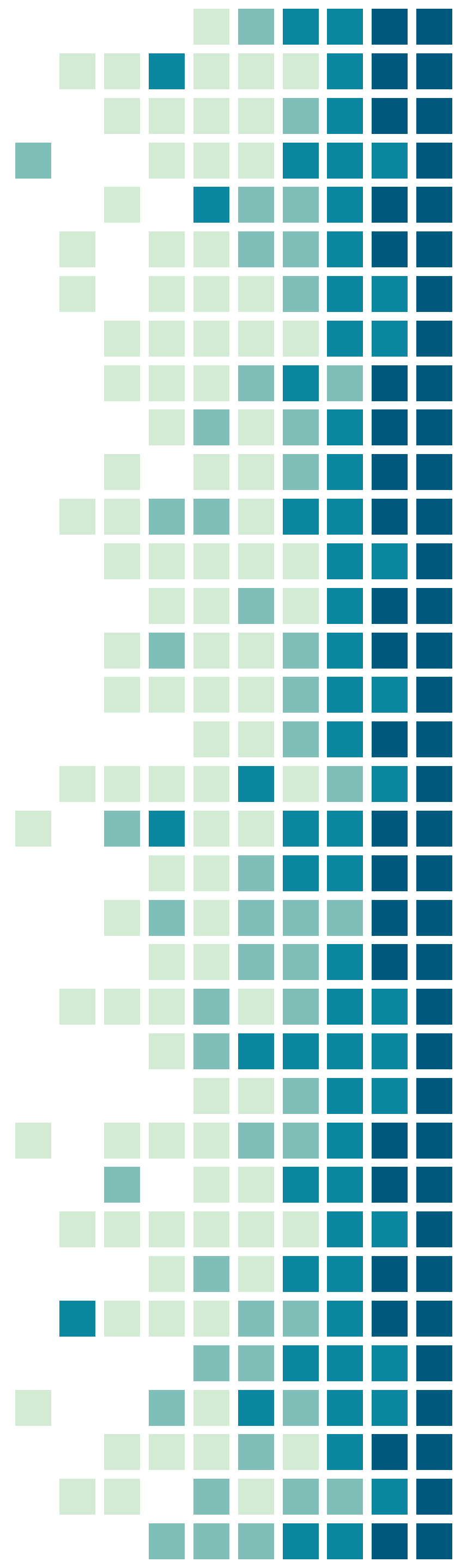
- ❖ La operación se llama merge
- ❖ Lleva los cambios a la rama principal
- ❖ Pueden surgir conflictos (resolvemos con diff)
- ❖ Todas las ramas deberían eventualmente integrarse a la principal o ser descartadas



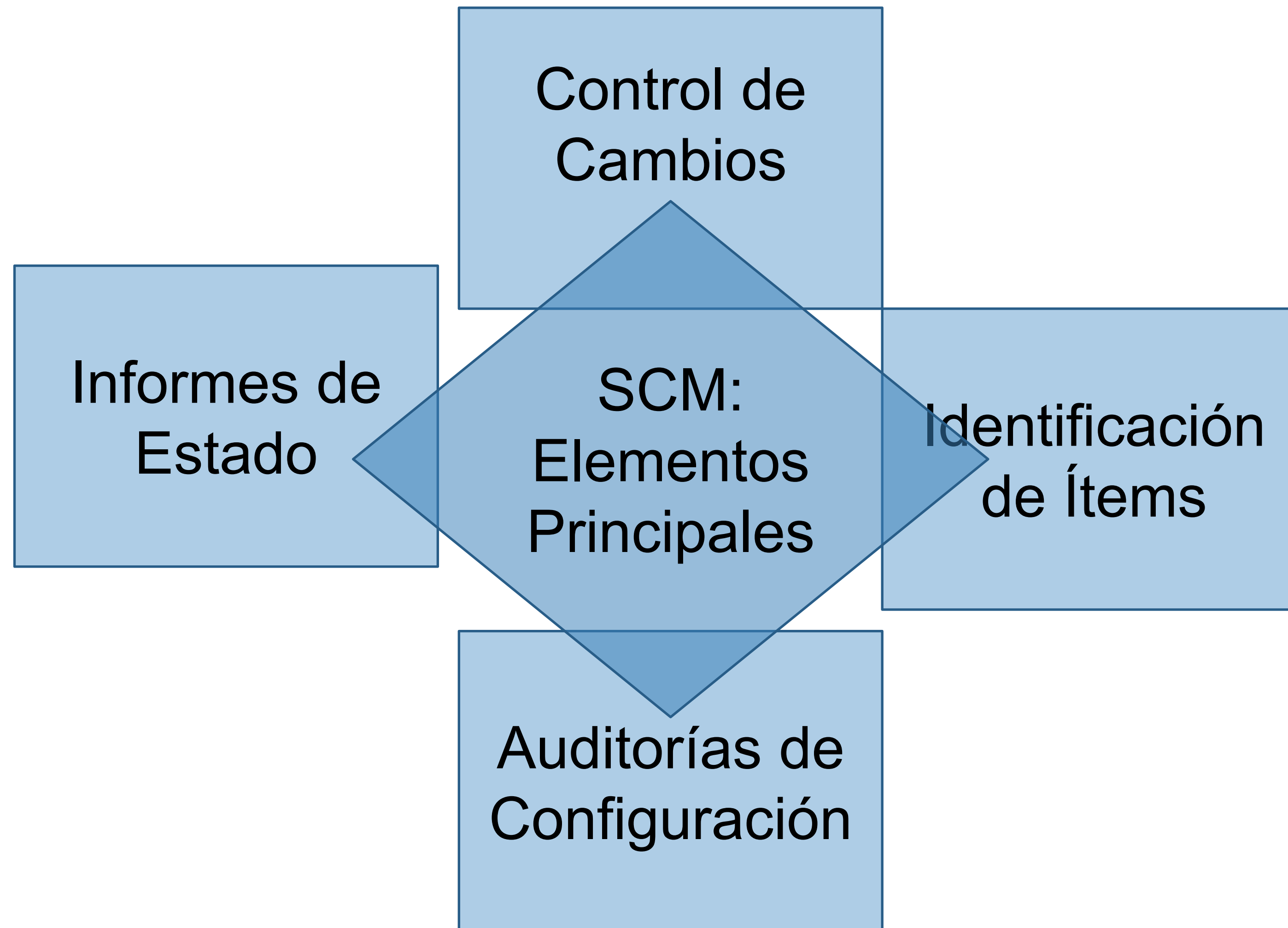
Definición de Gestión de Configuración de Software

Una disciplina que aplica dirección y monitoreo administrativo y técnico a: identificar y documentar las características funcionales y técnicas de los ítems de configuración, controlar los cambios de esas características, registrar y reportar los cambios y su estado de implementación y verificar correspondencia con los requerimientos

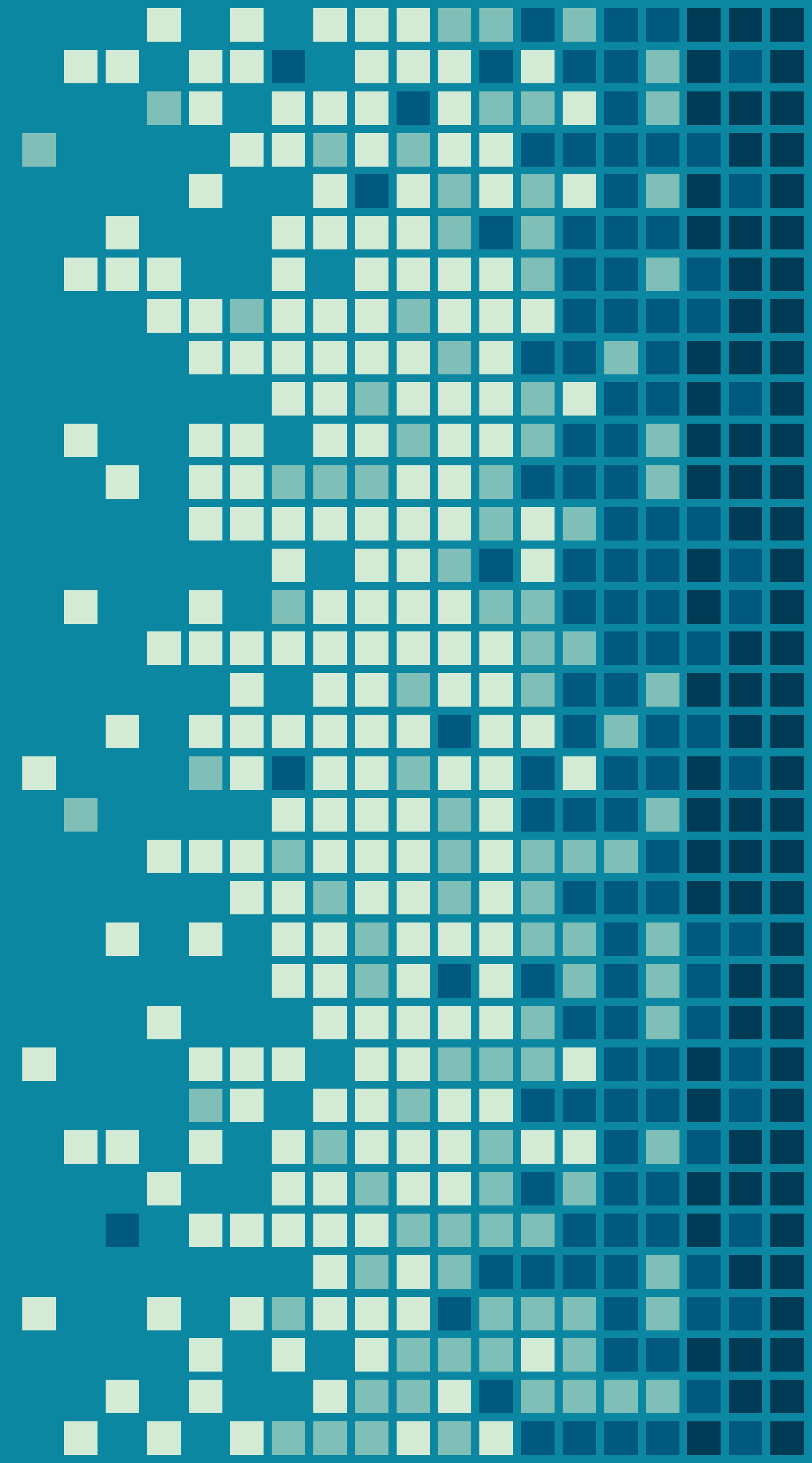
(ANSI/IEEE 828, 1990)



Actividades Fundamentales de la Gestión de Configuración de Software

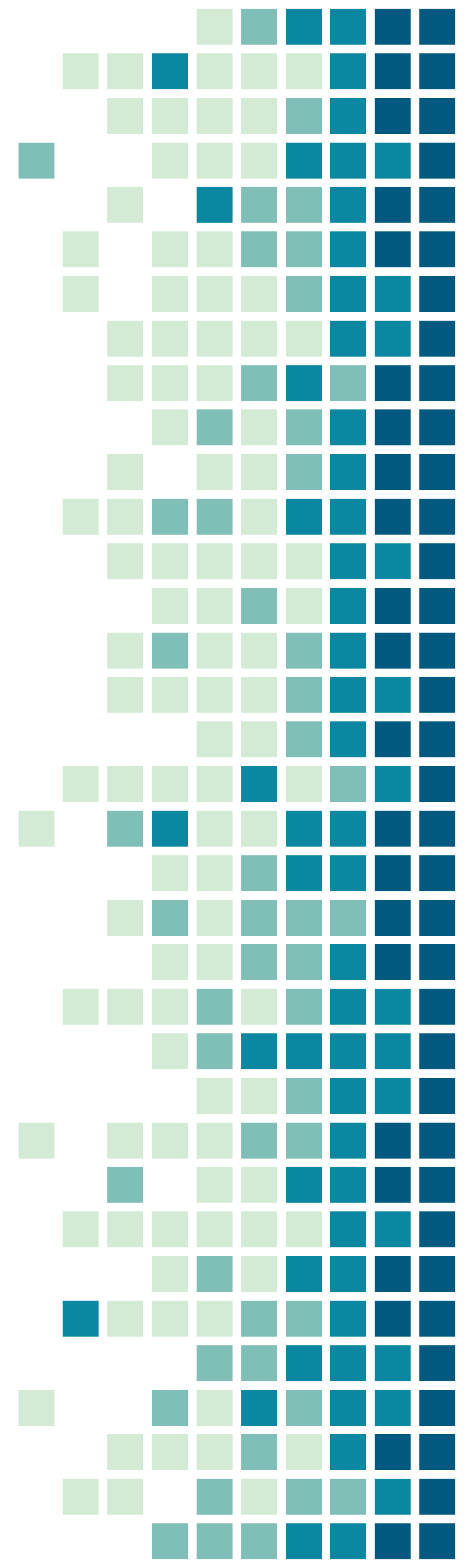


“ . *Identificación de ítems
de configuración*



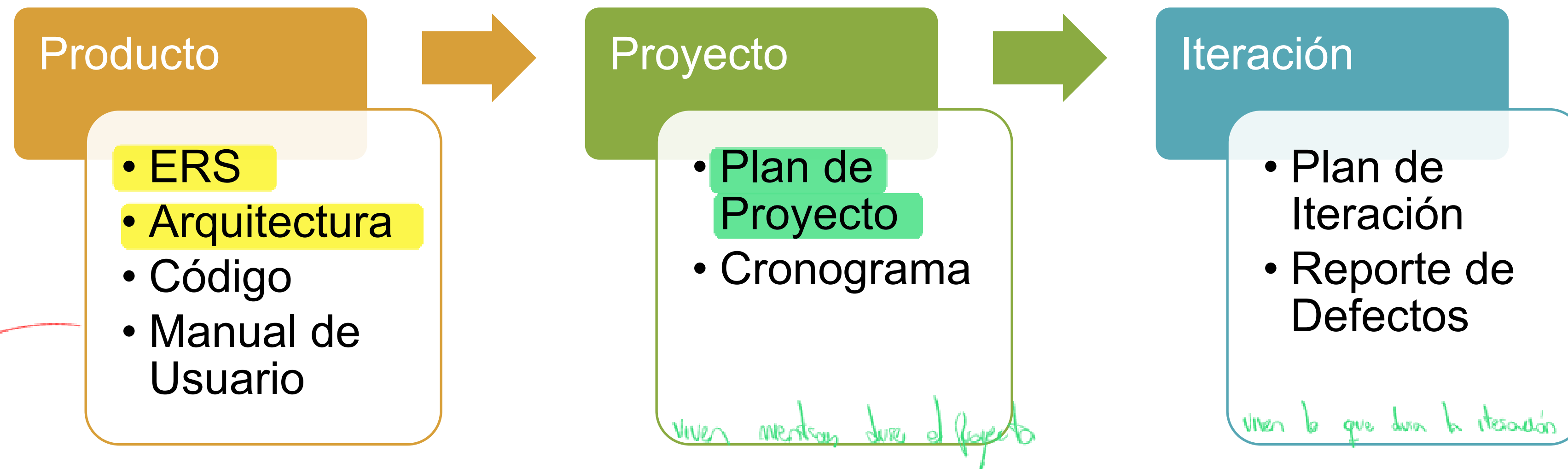
Identificación de ítems de configuración

- ❖ Identificación unívoca de cada ítem de configuración
- ❖ Convenciones y reglas de nombrado
- ❖ Definición de la Estructura del Repositorio
- ❖ Ubicación dentro de la estructura del repositorio



Ítems de Configuración para un proyecto de desarrollo de software

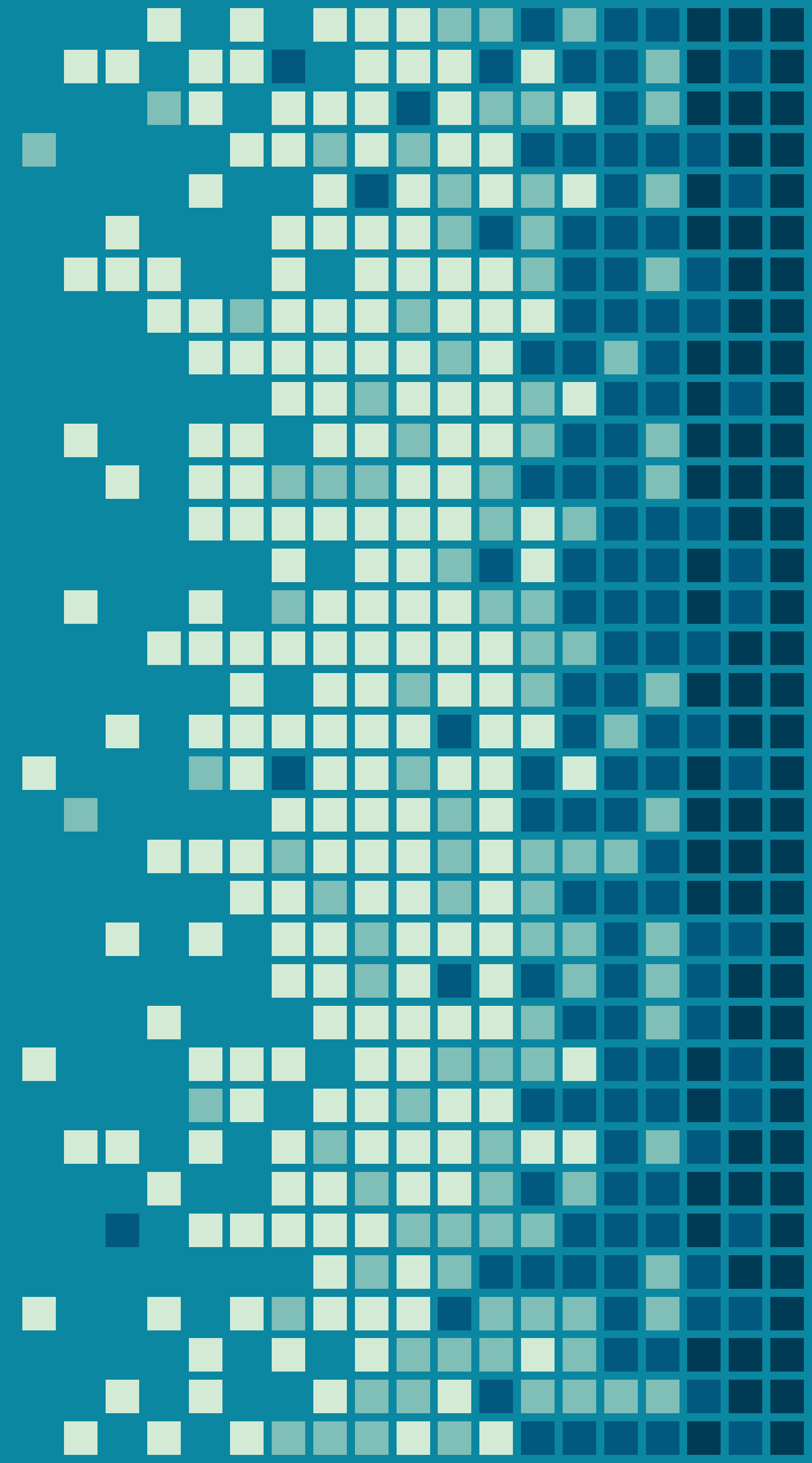
IC dependiendo del ciclo de vida



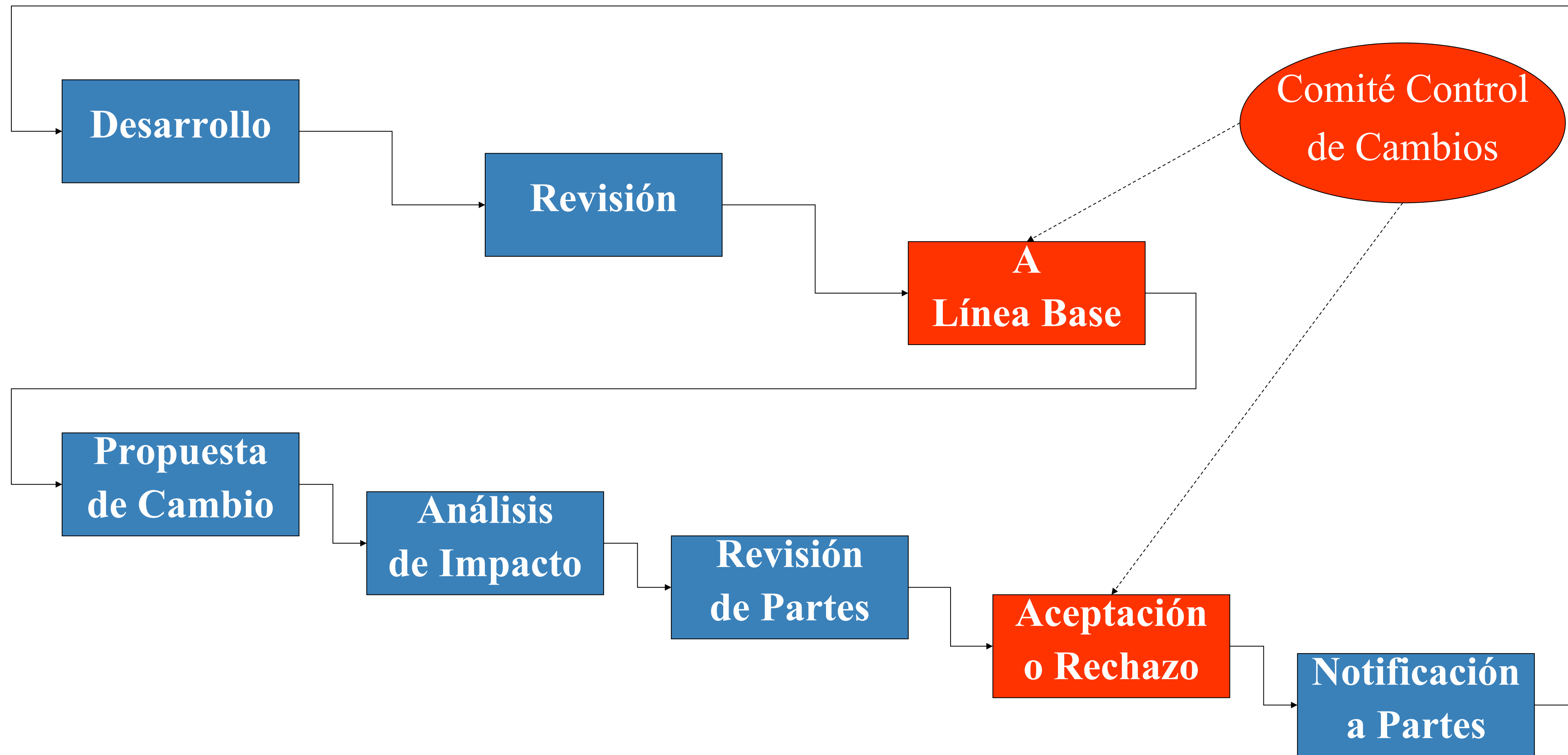
se van modificando a lo largo de muchos proyectos

* Tienen diferentes reglas de nombres

“ . *Control de Cambios*

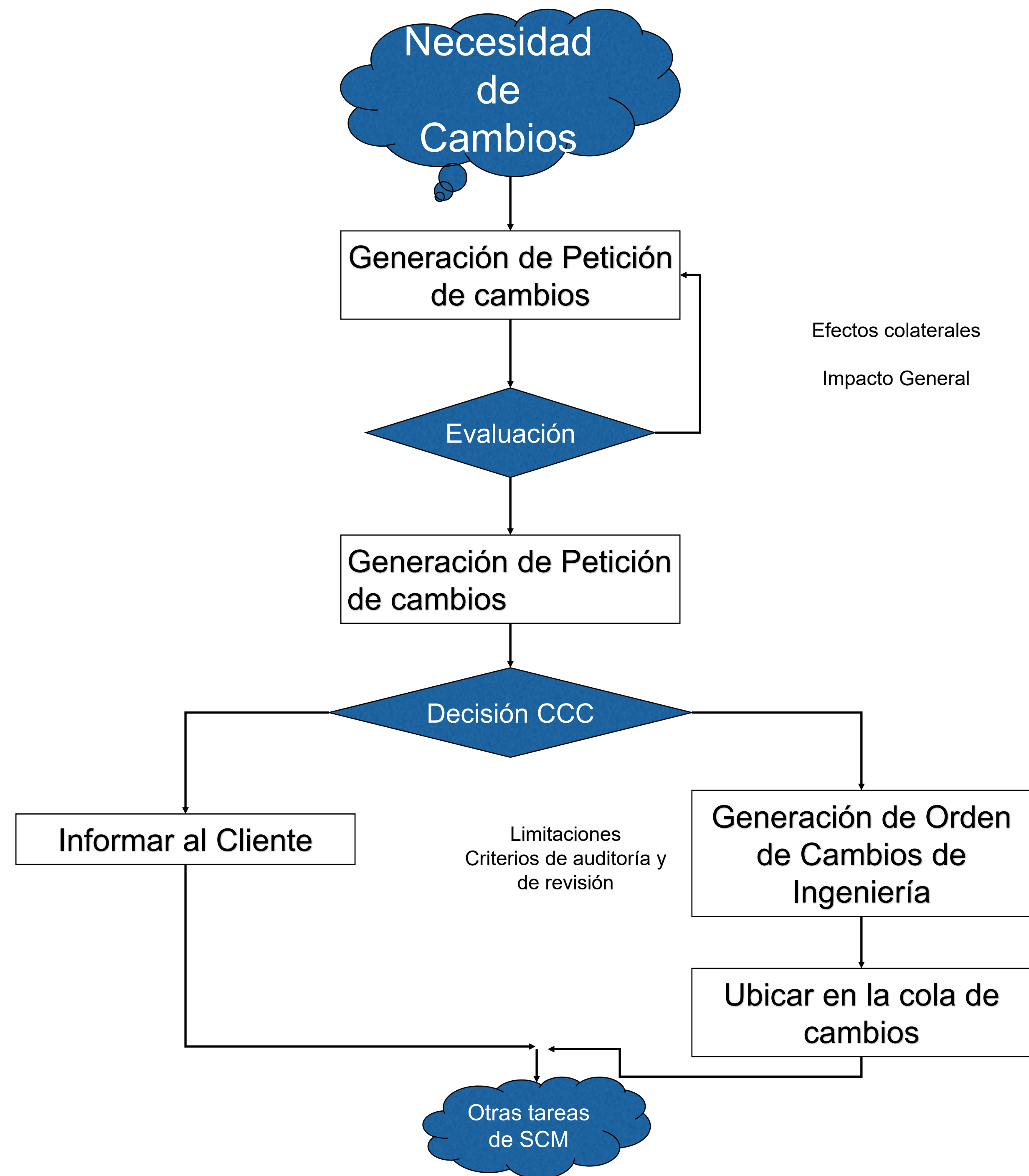


Proceso de Control de Cambios

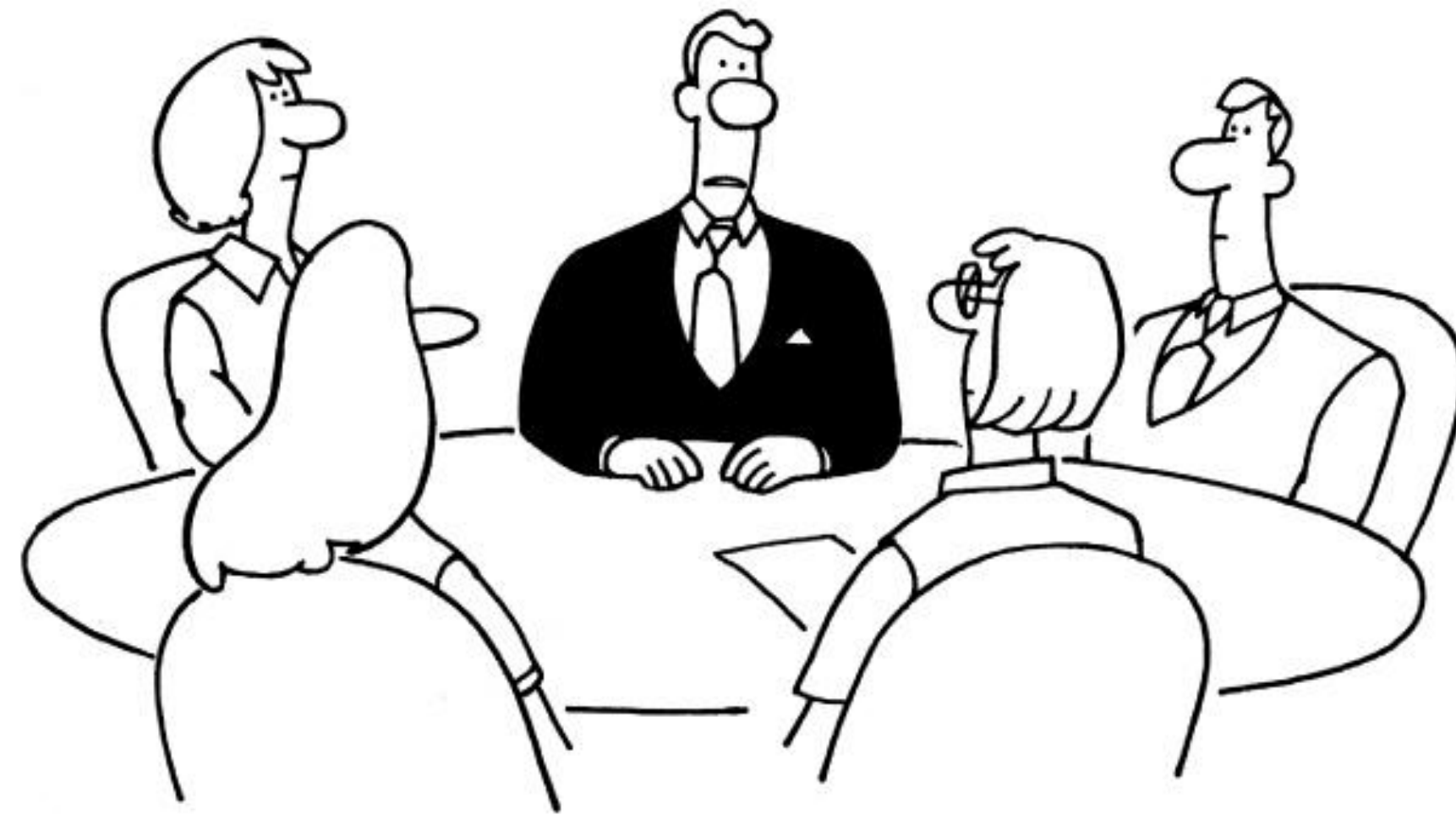


Control de Cambios

- ❖ Tiene su origen en un Requerimiento de Cambio a uno o varios ítems de configuración que se encuentran en una **línea base**.
- ❖ Es un Procedimiento formal que involucra diferentes actores y una evaluación del **impacto** del cambio



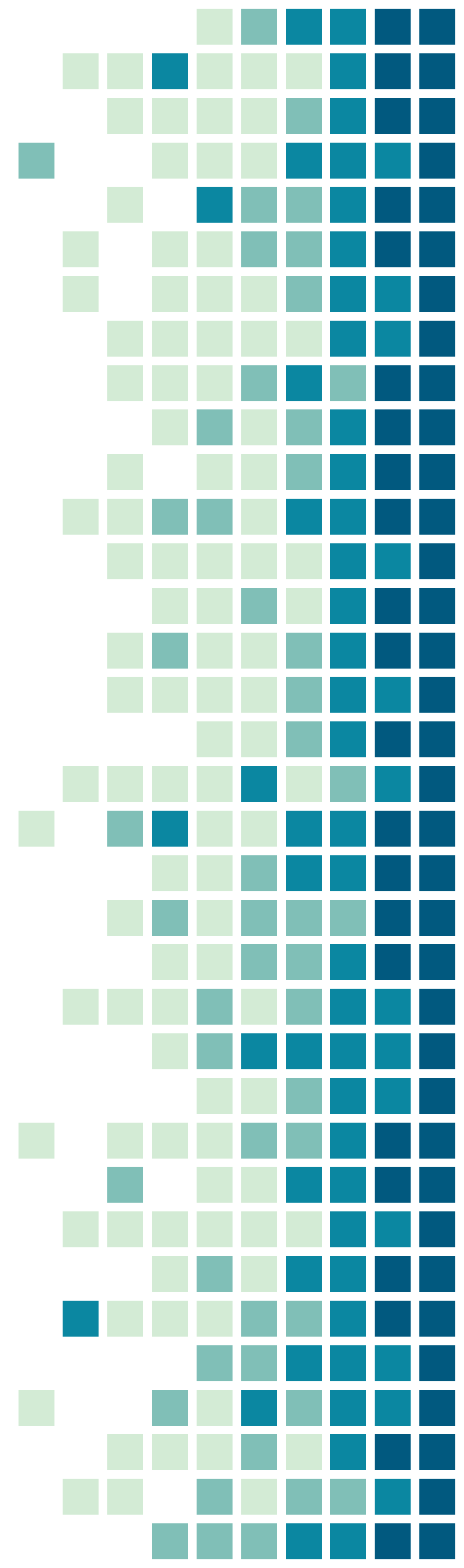
El Comité de Control de Cambios



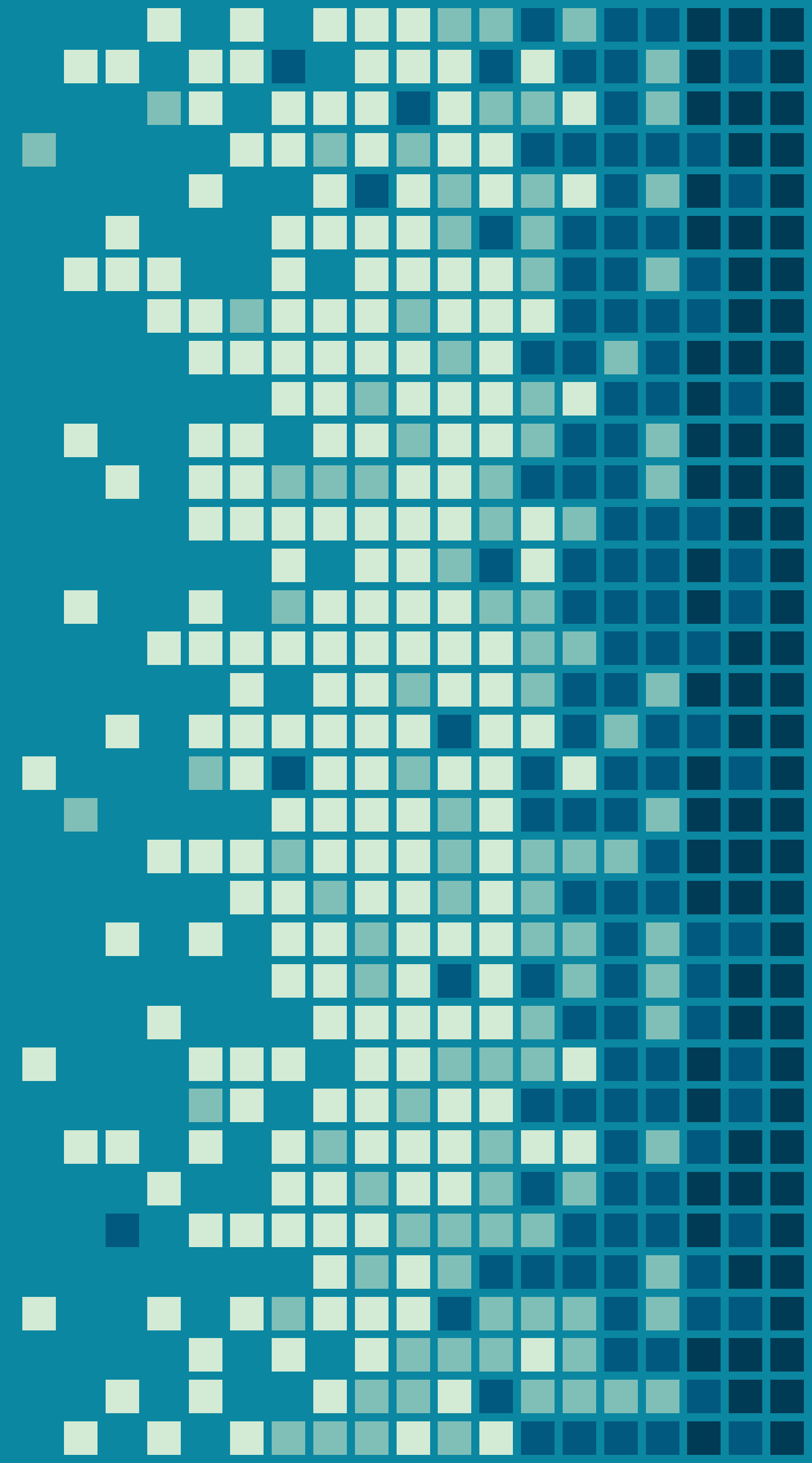
*"Whew! That was close!
We almost decided something!"*

Está formado por representantes de todas las áreas involucradas en el desarrollo:

- ❖ Análisis, Diseño
- ❖ Implementación
- ❖ Testing
- ❖ Otros interesados



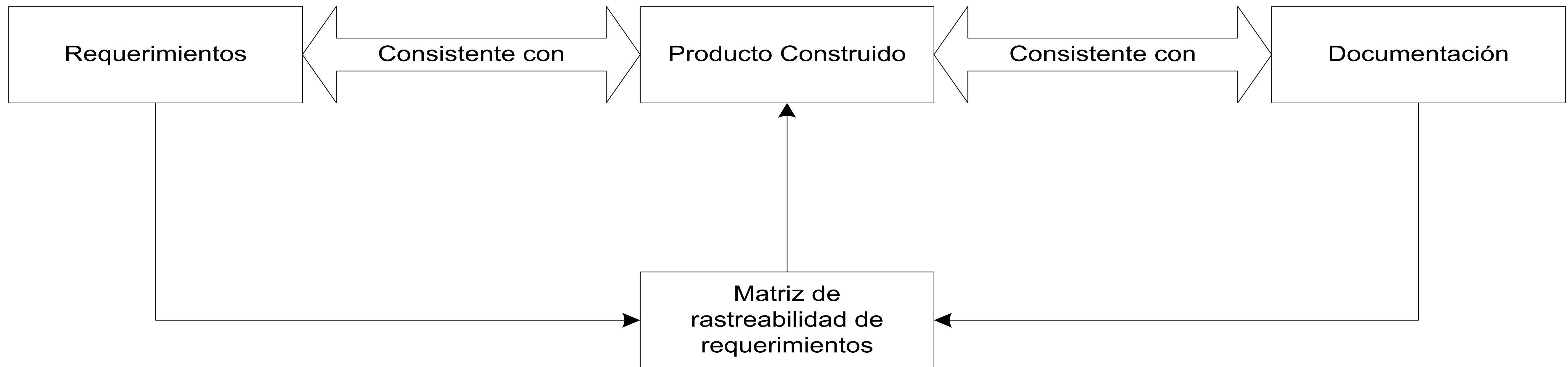
“ . Auditorías de Configuración de Software



Auditoría de Gestión de Configuración

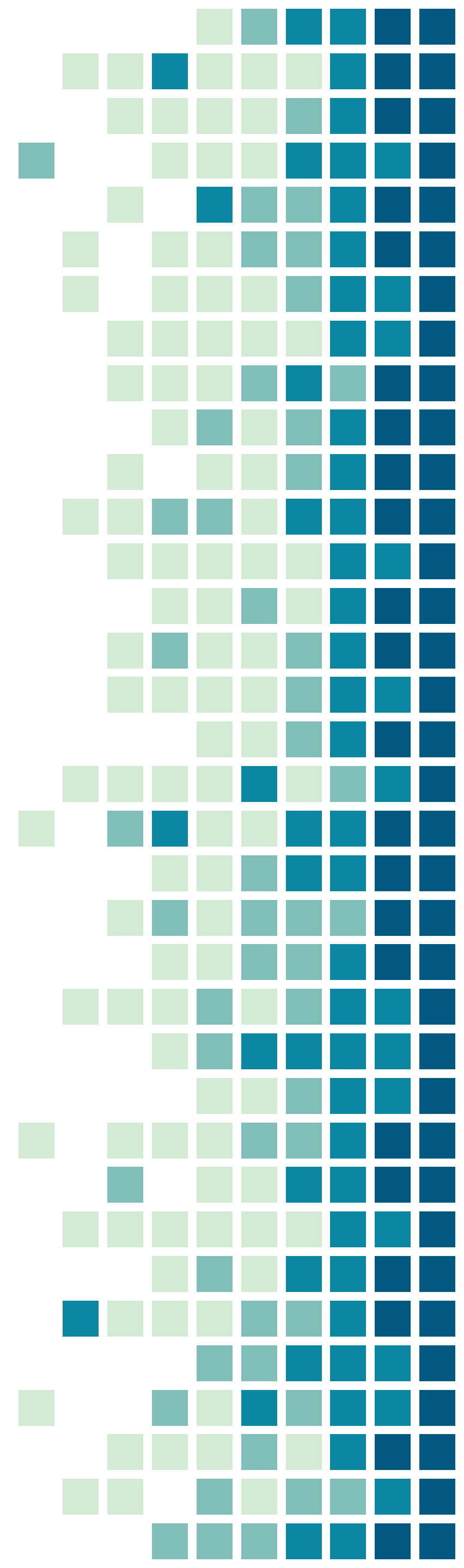
Auditoría Funcional de Configuración

Auditoría Física de Configuración



Auditorías de Configuración

- ❖ **Auditoría física de configuración (PCA)**
Asegura que lo que está indicado para cada ICS en la línea base o en la actualización se ha alcanzado realmente.
- ❖ **Auditoría funcional de configuración (FCA)**
Evaluación independiente de los productos de software, controlando que la funcionalidad y performance reales de cada ítem de configuración sean consistentes con la especificación de requerimientos.

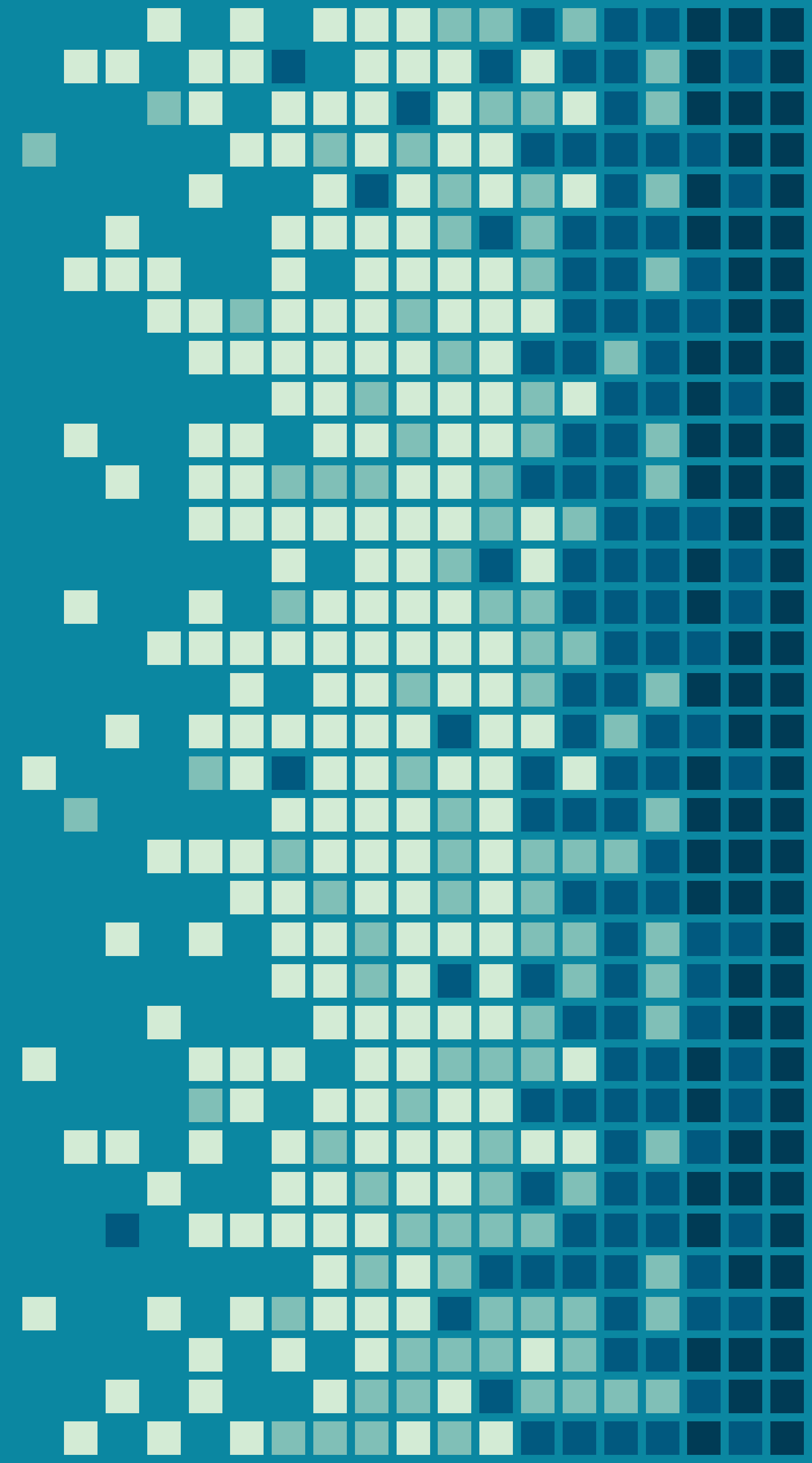


Auditoría de Gestión de Configuración y V&V

Sirve a dos procesos básicos: la validación y la verificación

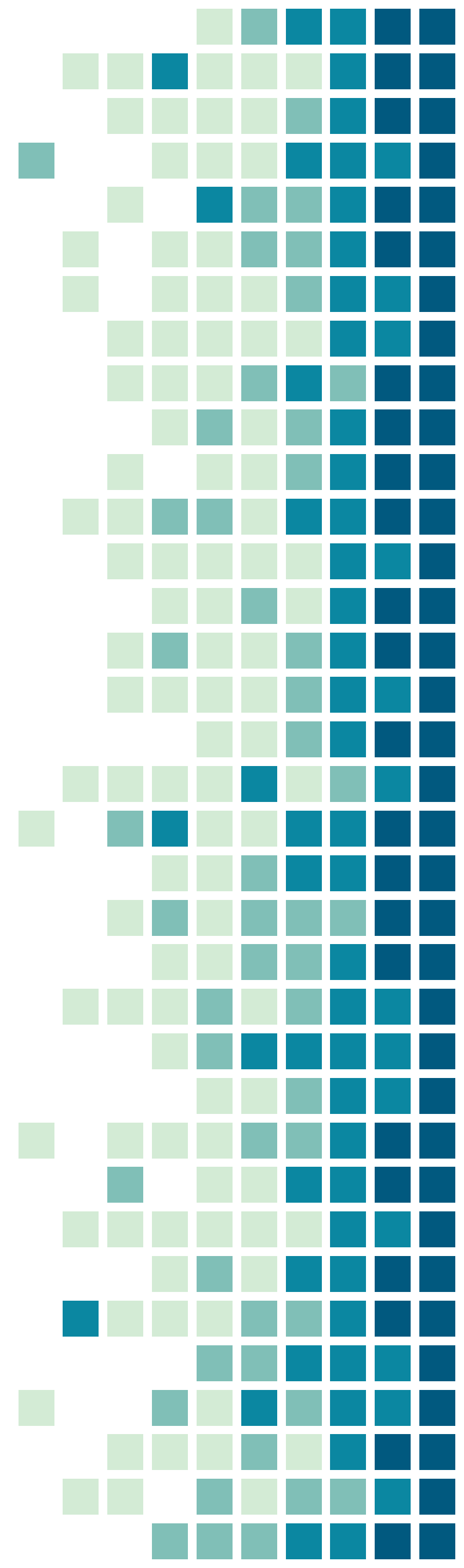
- ❖ **Validación:** el problema es resuelto de manera apropiada que el usuario obtenga el producto correcto.
- ❖ **Verificación:** asegura que un producto cumple con los objetivos preestablecidos, definidos en la documentación de líneas base (línea base). Todas las funciones son llevadas a cabo con éxito y los test cases tengan status "ok" o bien consten como "problemas reportados" en la nota de release.

“ *Informes de Estado*



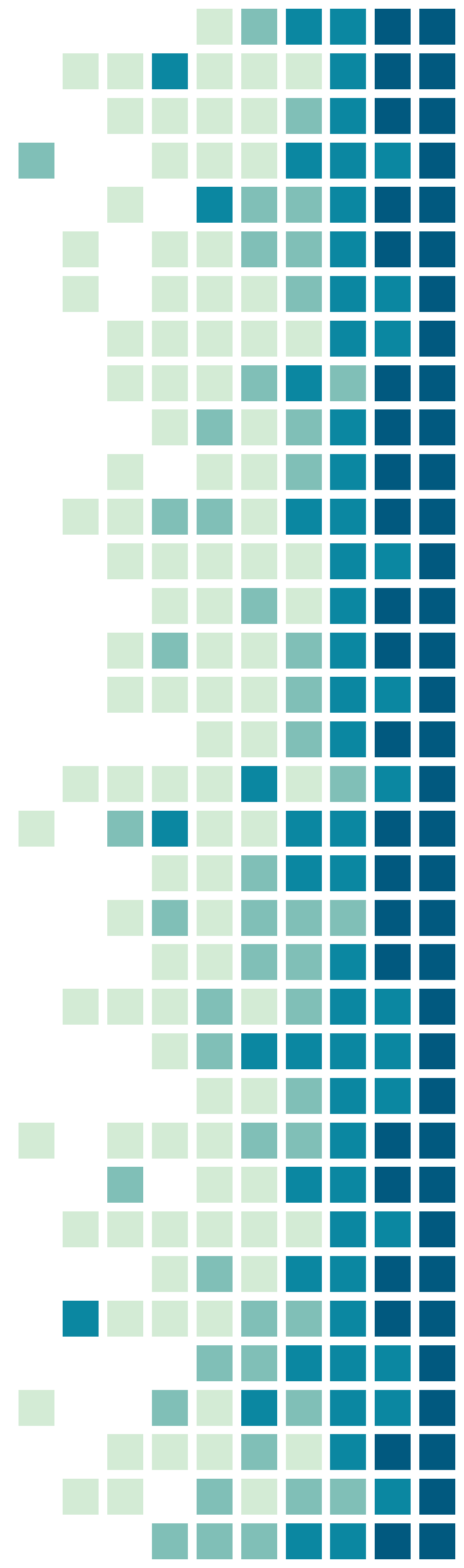
Registro e Informe de Estado

- ❖ Se ocupa de mantener los registros de la evolución del sistema.
- ❖ Maneja mucha información y salidas por lo que se suele implementar dentro de procesos automáticos.
- ❖ Incluye reportes de rastreabilidad de todos los cambios realizados a las líneas base durante el ciclo de vida.



Algunas preguntas que podría responder

- ❖ ¿Cuál es el estado del ítem?
- ❖ ¿Un requerimiento de cambio ha sido aprobado o rechazado por el CCB?
- ❖ ¿Qué versión de ítem implementa un requerimiento de cambio aprobado (saber cuál es el componente que contiene la mejora)?
- ❖ ¿Cuál es la diferencia entre una versión y otra dada?

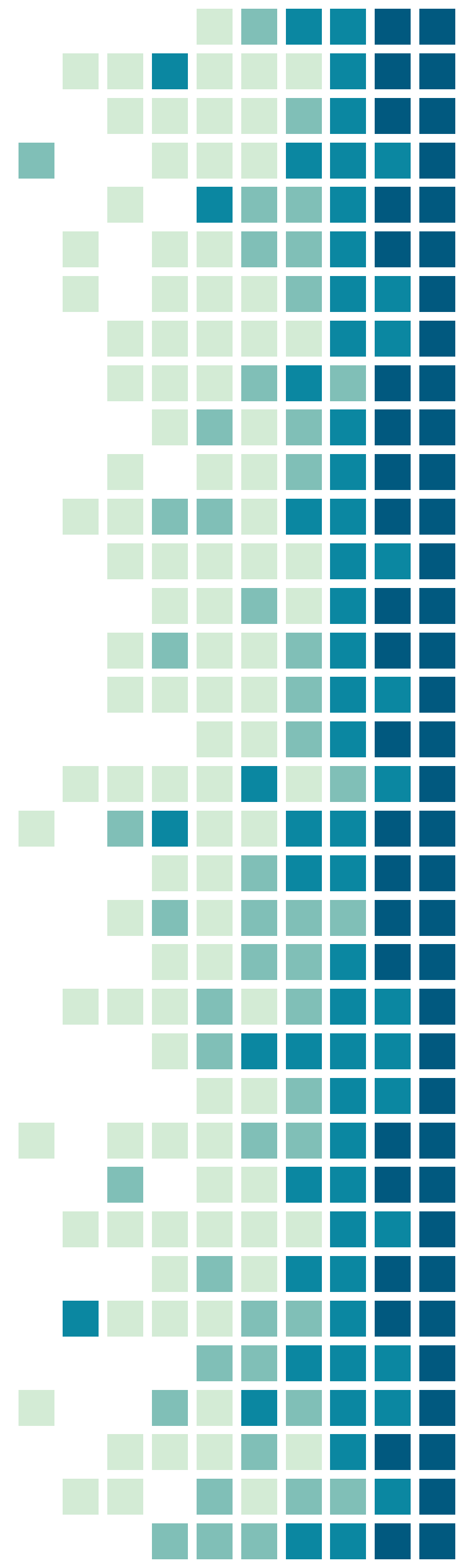




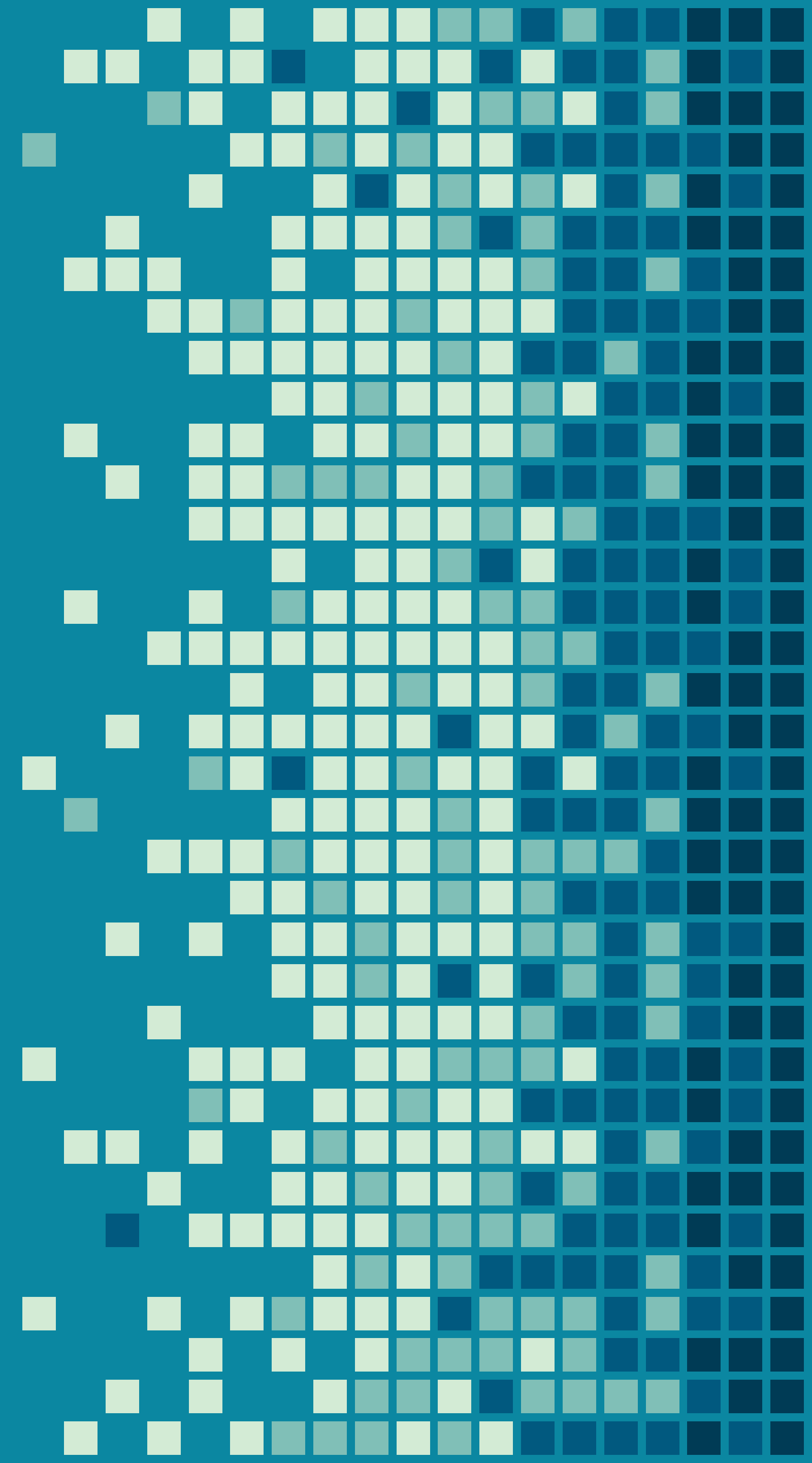
Plan de Gestión de Configuración

¡También se planifica! ¿Qué debería incluir el plan?

- ❖ Reglas de nombrado de los CI
- ❖ Herramientas a utilizar para SCM
- ❖ Roles e integrantes del Comité
- ❖ Procedimiento formal de cambios
- ❖ Plantillas de formularios
- ❖ Procesos de Auditoría



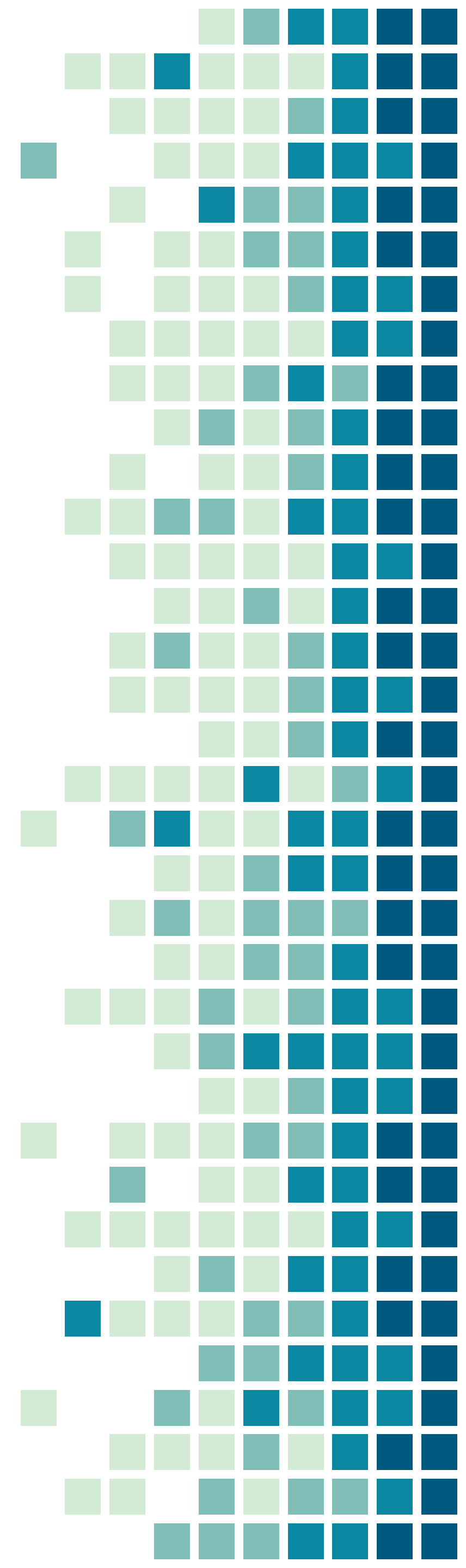
“ . *Gestión de Configuración
de Software en
ambientes Ágiles*



Recuerdan...

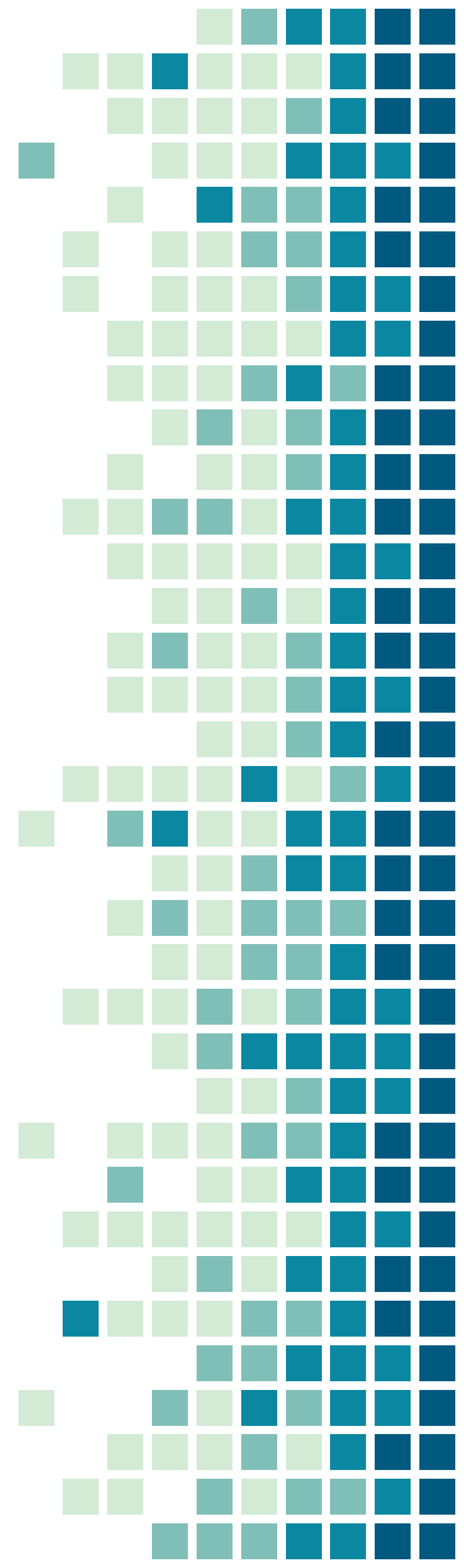
Manifiesto

Ágil



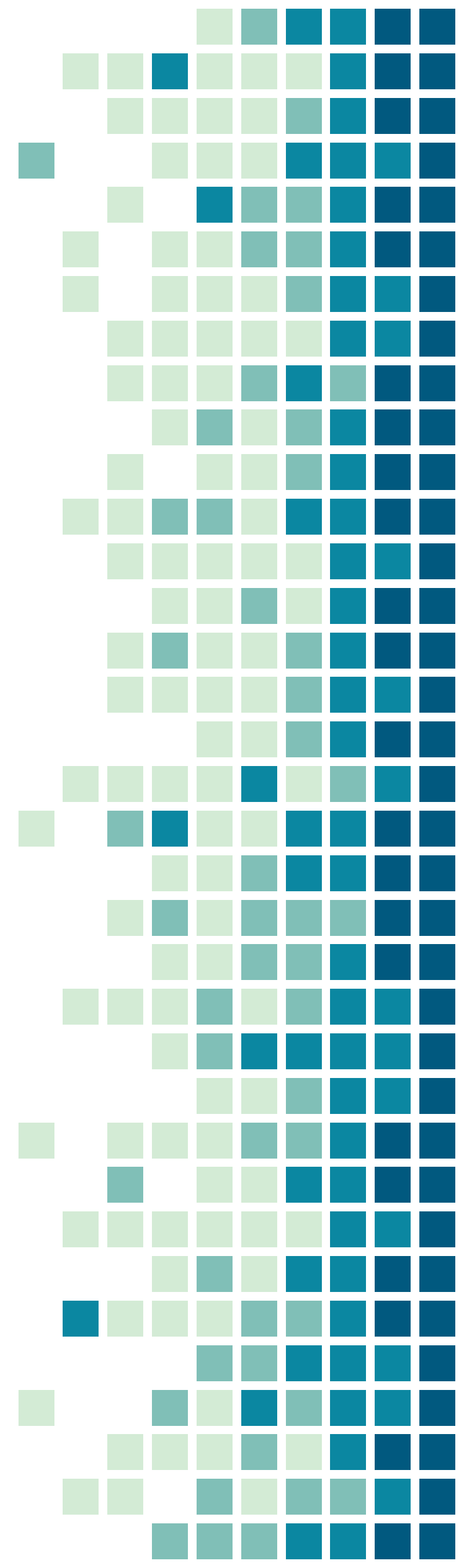
SCM en Agile

- ❖ Sirve a los practicantes (equipo de desarrollo) y no viceversa.
- ❖ Hace seguimiento y coordina el desarrollo en lugar de controlar a los desarrolladores.
- ❖ Responde a los cambios en lugar de tratar de evitarlos.
- ❖ Esforzarse por ser transparente y "sin fricción", automatizando tanto como sea posible.
- ❖ Coordinación y automatización frecuente y rápida.
- ❖ Eliminar el desperdicio - no agregar nada más que valor.
- ❖ Documentación Lean y Trazabilidad.
- ❖ Feedback continuo y visible sobre calidad, estabilidad e integridad



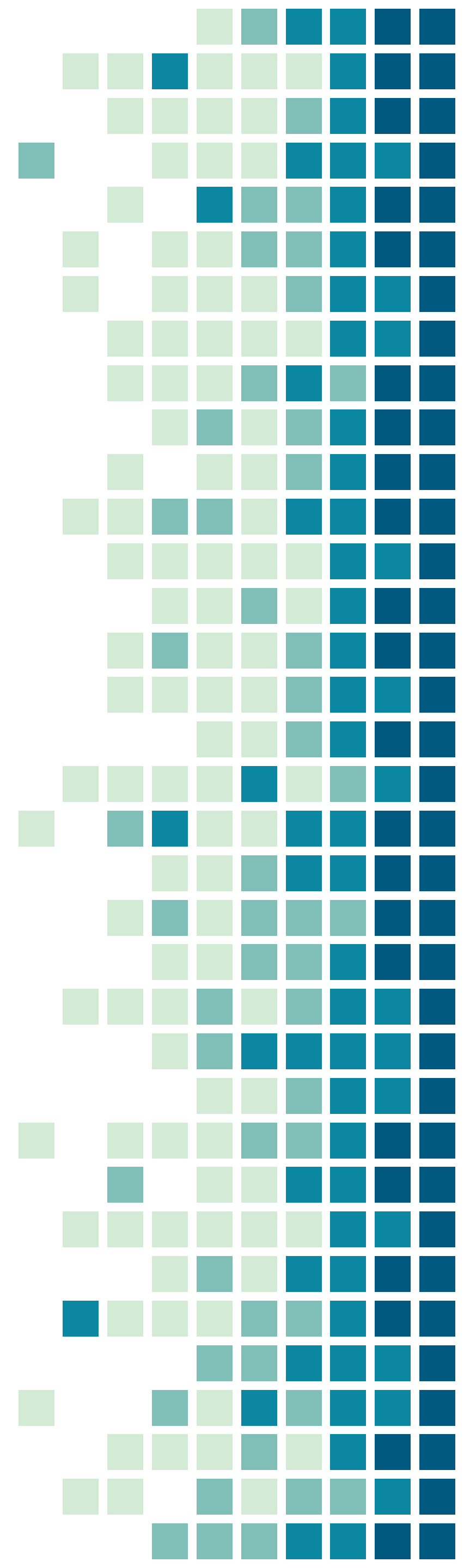
SCM en Agile, algunos tips....

- ❖ Es responsabilidad de todo el equipo.
- ❖ Automatizar lo más posible.
- ❖ Educar al equipo.
- ❖ Tareas de SCM embebidas en las demás tareas requeridas para alcanzar el objetivo del Sprint.



SCM en Agile, para debatir....

- ❖ ¿Qué pasa con el Comité de Control de Cambios?
- ❖ ¿Qué ítems de configuración podemos tener?
- ❖ ¿Qué pasa con las auditorías?
- ❖ ¿Qué pasa con los reportes de estado?



Referencias

- Bersoff, E.H., "Elements of Software Configuration Management",
- IEEE Transactions on Software Engineering, vol 10, nro. 1, enero 1984, pp 79-87
- Little Book of Configuration Management – <http://www.spmn.com>
- SCM & the Agile Manifesto - <http://www.scmpatterns.com/agilescm/>
- Harness.io - <https://harness.io/blog/continuous-verification/blue-green-canary-deployment-strategies/>

